ManyRVData Project Report

CachePool: Manycore Cluster of Customizable, Lightweight Scalar-Vector PEs for Irregular Data-Plane Workloads



Report by:
Diyou Shen, Ph.D. Student
Zexin Fu, Ph.D. Student
dishen, zexifu@iis.ee.ethz.ch

Supervised by:
Prof. Luca Benini
Prof. Alessandro Vanelli-Coralli
lbenini, avanelli@iis.ee.ethz.ch

Integrated Systems Laboratory,
ETH Zürich,
Zürich, Switzerland

October 24, 2026

Contents

1 Executive Summary **4**

2 Introduction **7**
2.1 State of the Arts . . . 7
2.1.1 On-Chip Memory Organization in Manycore Architectures . . . 7
2.1.2 Cache Coherence Schemes . . . 8
2.1.3 Scalable Non-Blocking Cache Miss Handling . . . 9
2.1.4 On-Chip Interconnection . . . 10
2.2 Project Objectives, Design Philosophy, and Contributions . . . 10
2.3 Reference Dataplane Workload Targets . . . 15

3 CachePool Architecture **16**
3.1 System Overview . . . 16
3.2 Core Complex . . . 17
3.2.1 Snitch Scalar Core . . . 17
3.2.2 Spatz Vector Coprocessor . . . 17
3.2.3 Core-Complex Configurations and Vector-Unit Sharing . . . 17
3.2.4 Core-Local Stack Memory . . . 18
3.2.5 Non-Blocking Memory Access and Dependency Tracking . . . 18
3.3 Tile Architecture . . . 20
3.3.1 Tile Organization . . . 20
3.3.2 Local L1 Access Network . . . 20
3.3.3 Atomic Operations . . . 21
3.4 Group Architecture . . . 21
3.5 Distributed and Partitionable L1 Data Cache . . . 22
3.5.1 Global Shared-L1 Organization . . . 22
3.5.2 Address Mapping and Software-Controlled Scrambling . . . 22
3.5.3 Cache Partitioning . . . 22
3.6 L1 Cache Controller . . . 23
3.6.1 Request Coalescer . . . 23
3.6.2 In-Situ Cache Overview . . . 23
3.6.3 Unified In-Cacheline Miss Handling . . . 24
3.6.4 Request and Refill Pipelines . . . 24
3.6.5 Way Prediction and Replacement . . . 25
3.6.6 Folded Cache Organization . . . 25
3.7 L1 Data Access Network . . . 27
3.7.1 Network Organization and Protocol . . . 27
3.7.2 Design Rationale and Network Decomposition . . . 27
3.7.3 Intra-Tile Access . . . 29
3.7.4 Intra-Group Access . . . 30
3.7.5 Inter-Group Narrow Meshes . . . 30
3.7.6 Non-Geometric Destination Routing . . . 30
3.7.7 L1 Network Latency . . . 30
3.8 Cache Refill Network and Main-Memory Connectivity . . . 31
3.8.1 Wide Refill Mesh . . . 32
3.8.2 HBM Connectivity . . . 33
3.8.3 Refill Routing and Protocol Conversion . . . 33
3.9 Instruction Memory Hierarchy . . . 35
3.9.1 Per-Core L0 Instruction Cache . . . 35
3.9.2 Tile-Shared L1 Instruction Cache . . . 35

3.9.3 Group-Shared L2 Instruction Cache . . . 35
3.10 System Control and Synchronization . . . 35
3.10.1 Memory-Mapped Control and Status Registers . . . 35
3.10.2 Hardware Barrier . . . 36
3.11 System Configuration and Scalability . . . 36
3.12 DRAM/HBM Co-Simulation . . . 36

4 Physical Implementation **40**
4.1 Methodology . . . 40
4.2 Floorplan . . . 40
4.3 Area Breakdown . . . 41

5 Software and Runtime Support **44**
5.1 Programming Model . . . 44
5.2 Topology Discovery and Core Identification . . . 44
5.3 Memory Allocation and Placement . . . 45
5.3.1 L1 Address Scrambling . . . 45
5.3.2 Cache Partitioning and Address Classification . . . 45
5.3.3 Cache Maintenance . . . 46
5.4 Synchronization and Work Distribution . . . 46
5.4.1 Memory Ordering and Fences . . . 46
5.4.2 Hardware and Software Barriers . . . 46
5.4.3 Atomic Operations . . . 47
5.4.4 Work Distribution . . . 47
5.5 Spatz Control . . . 47
5.6 Boot, Execution, and Profiling Support . . . 48
5.7 Hardware–Software Co-Design Guidelines . . . 48

6 GVSoC Simulation Framework **50**
6.1 CachePool System Model . . . 50
6.2 Configurable Architecture Exploration . . . 50
6.3 Memory-System Modeling . . . 52
6.4 Software Compatibility and End-to-End Simulation . . . 53
6.5 Functional Validation and RTL Compatibility . . . 53
6.6 Modeling Scope and Abstraction Choices . . . 54

7 Vector Kernel Characterization and Evaluation **55**
7.1 Vector Micro-Kernels . . . 55
7.1.1 Kernel Selection and Mapping . . . 55
7.2 Control Algorithm . . . 56
7.2.1 Reference Algorithm and Benchmark Formulation . . . 56
7.2.2 Fixed-Point Gain Evaluation . . . 57
7.2.3 Restructured Algorithm and Kernel Chain . . . 58
7.2.4 CachePool Mapping and Software Optimizations . . . 59

8 RLC/MAC Dataplane Implementation **61**
8.1 Target Workloads and Execution Model . . . 61
8.2 Software Architecture and Implementation . . . 61
8.2.1 Cooperative AM Downlink Pipeline . . . 61
8.2.2 Queues, Allocation, and Synchronization . . . 63
8.2.3 Private/Shared Placement and Work Mapping . . . 63
8.3 Uplink Receive Harness . . . 64

8.4 Functional Validation . . . 67

9 Evaluation **68**
9.1 Vector Kernel Benchmarking . . . 68
9.1.1 Roofline Analysis . . . 68
9.1.2 Control Kernel Characterization . . . 70
9.1.3 Control Algorithm End-to-End Results . . . 71
9.2 RLC Kernel Benchmarking . . . 72

10 Future Work **75**

11 Conclusion **77**

A Private Write-Through L0.5 Cache Exploration **82**
A.1 Motivation and Scope . . . 82
A.2 Hardware-Coherent Exploration . . . 82
A.3 Software-Managed Coherence . . . 82
A.3.1 Private L0.5 Organization . . . 83
A.3.2 Synchronization and Data Visibility . . . 83
A.3.3 Runtime Cache-Management Operations . . . 83
A.3.4 Kernel Integration . . . 84
A.4 Preliminary Performance Observations . . . 84
A.5 Discussion and Future Work . . . 85

B Physically-Aware 2D NoC Topology Exploration **86**
B.1 Motivation . . . 86
B.2 Topology Exploration . . . 86
B.3 Planar Torus Implementation . . . 87
B.4 Physical Cost of the 2D Torus . . . 87
B.5 Impact of Interconnect Pipelining . . . 87

C CachePool Runtime API Reference **89**

D Spatz Supported Vector Instructions **95**

1 Executive Summary

This document presents the final results of the *ManyRVData* project, a joint research effort between ETH Zürich and Huawei targeting scalable manycore architectures for wireless dataplane processing. The project investigates how hundreds of lightweight processing elements can be combined with vector acceleration and a cache-based shared-memory hierarchy while maintaining high performance, energy efficiency, and physical scalability. The main application focus is the Radio Access Networks (RAN) dataplane, including irregular and synchronization-intensive Radio Link Control (RLC) processing as well as vectorizable control and data-processing kernels.

The resulting architecture, referred to as *CachePool*, combines lightweight Snitch scalar cores with tightly coupled Spatz vector coprocessors in configurable Core Complexes. The system is organized hierarchically into Core Complexes, Tiles, and Groups, enabling the architecture to scale from small configurations to manycore systems with hundreds of scalar cores and vector lanes. Both 1S1V and 2S1V Core Complexes are supported, allowing one or two scalar cores to share a Spatz vector engine and providing a configurable trade-off between scalar-control throughput, vector capability, area, and utilization.

A central contribution of the project is the memory hierarchy. CachePool replaces an explicitly managed scratchpad-only organization with a hardware-managed, logically shared L1 data cache that is physically distributed across the system. In the reference large-scale configuration, the L1 provides 16 MiB of aggregate capacity distributed over 64 Tiles. Because the default organization exposes one shared memory without replicated private cache copies, no conventional hardware cache-coherence protocol is required.

The shared L1 is complemented by software-controlled address scrambling and cache partitioning. The interleaving granularity can be configured according to workload access patterns, while individual L1 banks can be assigned to globally shared or private regions. Private regions can be scoped to either a Tile or a Group, allowing the same hardware organization to trade global sharing for stronger locality and isolation when required. These mechanisms are implemented in the memory-routing layer and are therefore largely independent of the cache-controller microarchitecture.

To support irregular accesses and long memory latency, CachePool integrates the *In-Situ Cache* controller. The controller reuses cache-line resources for non-blocking read- and write-miss handling instead of relying on a separate fixed-size miss-status structure. Additional optimizations include request coalescing, dynamic miss-resource expansion, single-way lookup, and physical-design-oriented controller refinements. Together, these mechanisms target the high memory-level parallelism required by sparse, pointer-based, and data-dependent dataplane workloads.

Scalability of the shared memory is addressed through a hierarchical interconnect organization. Within each Tile, multiple decomposed crossbars provide low-latency access from the scalar and vector memory ports to local cache banks while reducing the physical complexity of a monolithic crossbar. Traffic between Tiles and Groups is carried by hierarchical local interconnects and multiple independent narrow meshes for fine-grained load/store accesses. Cache-line refill traffic is separated onto a dedicated 512 bit two-dimensional mesh connected to distributed external-memory endpoints. This separation allows the core-access and refill networks to be optimized independently for their respective traffic characteristics.

The project also extends the execution and synchronization infrastructure required to program the architecture efficiently. A hierarchical hardware barrier supports Tile-, Group-, and system-level synchronization, partial participation, and multiple independent barrier instances. Memory-mapped configuration registers expose cache partitioning, address mapping, cache maintenance, Spatz ownership, synchronization, and system-control functions. The accompanying runtime provides topology discovery, memory allocation, fences and atomics, hardware and software barriers, cache configuration and maintenance, Spatz-sharing control, and work-distribution

support.

To accelerate architectural exploration and software development, the project additionally develops a CachePool model in GVSoC. The simulator executes the same application binaries used by RTL simulation while allowing the system topology and memory configuration to be changed without RTL re-elaboration. The model represents the hierarchical CachePool organization and can use either a simplified fixed-latency memory backend or DRAMSys for more detailed external-memory timing. GVSoC therefore complements cycle-accurate RTL simulation with a faster platform for end-to-end functional validation and design-space exploration.

The architecture is evaluated using representative dataplane and compute workloads, including RLC-oriented kernels, synchronization and pointer-chasing microbenchmarks, and vector kernels such as dot product, matrix–vector multiplication, matrix multiplication, sparse linear algebra, and FFT. These workloads are used both to characterize the memory and synchronization requirements of the target domain and to evaluate the interaction between scalar execution, vector acceleration, cache locality, interconnect contention, and external-memory bandwidth. The final evaluation also includes physical implementation and power–performance–area analysis of representative CachePool configurations.

In addition to the main shared-L1 architecture, the project investigates several side design points. These include folded cache organizations and private write-through L0.5 caches with both hardware-managed and software-managed coherence. The latter exploration combines write-through private caching with explicit cache-maintenance operations and synchronization primitives, providing an alternative hardware/software trade-off for workloads with structured data sharing. These exploratory studies are documented separately from the main CachePool architecture.

Overall, the ManyRVData project demonstrates a scalable cache-based manycore architecture that combines lightweight scalar processing, vector acceleration, a distributed shared cache, hierarchical interconnects, hardware synchronization, and software-controlled memory placement. The project delivers synthesizable RTL, runtime software, representative benchmark kernels, system-level simulation models, and physical-design evaluation, providing a complete architecture-to-software platform for studying irregular and heterogeneous dataplane workloads.

The developed hardware and software components are released as open-source artifacts where applicable.

Report organization. The remainder of this report is organized as follows. Section 2 introduces the target dataplane workloads, reviews the relevant state of the art, and summarizes the design objectives, architectural principles, and major developments of the ManyRVData project. Section 3 presents the CachePool architecture, including the scalar–vector Core Complexes, distributed shared L1 cache, programmable cache organization, hierarchical interconnects, synchronization mechanisms, and cache-controller microarchitecture. Section 4 evaluates the architecture's physical implementation and analyzes its floorplan and post-place-and-route area characteristics for the 1S1V and 2S1V configurations. Section 5 describes the software and runtime support required to control memory placement, cache partitioning, synchronization, and scalar–vector resource sharing. Section 6 introduces the GVSoC model used for fast system-level simulation and workload exploration. Section 7 presents representative vector micro-kernels and the control-algorithm workload used to study interactions among vector execution, memory locality, and hardware–software optimization. Section 8 describes the implementation and mapping of the target RLC/MAC dataplane workload. Section 9 presents the experimental results, including vector-kernel roofline analysis, control-algorithm performance, RLC evaluation, and corresponding scalability and memory-system studies. Finally, Section 10 discusses the main directions for extending CachePool, and Section 11 summarizes the outcomes of the project.

The appendices complement the main report with exploratory studies and implementation references, including the private write-through L0.5 cache, physically aware 2D NoC topology

exploration, the CachePool runtime API, and the vector instructions supported by Spatz.

2 Introduction

Wireless baseband dataplane processing is a prime domain for energy-efficient parallel computing: the RAN market is vast, and the throughput and efficiency requirements are escalating. As 5G evolves toward 5.5G and 6G, per-cell computational complexity is projected to grow far beyond what technology scaling alone can deliver, demanding architectural and implementation advances across the stack. This project addresses that challenge by focusing on RAN dataplane processing, particularly tasks in the RLC layer. Service Data Units (SDUs) from higher layers are queued and assembled into Protocol Data Units (PDUs) with sequence headers for retransmission management before being passed to MAC, an interaction that tightly couples performance and reliability constraints.

As a computing workload, RLC has three defining features. First, scale: modern deployments must sustain on the order of 10 million packets/s overall, with each RLC instance exceeding 10^3 packets/s and thousands of instances active concurrently. Second, latency: the Transition Time Interval (TTI) spans 62.5 µs to 1 ms (typically 500 µs), setting a hard real-time envelope for all sub-tasks. Third, irregularity: per-user tasks are independent but highly variable, with a small code footprint but large and complex data structures that create sparse, non-contiguous accesses over broad address ranges. Together, these properties result in many concurrent processing contexts accessing dispersed state with poor spatial locality.

To ground our design choices, the remainder of this section surveys state-of-the-art manycore memory organizations, cache-coherence approaches, and on-chip interconnects, and evaluates their suitability for the RLC workload's stringent throughput, TTI-scale latency, and irregular memory-access behavior. We then introduce *ManyRVData*'s design objectives and architectural principles and summarize the project's main contributions.

2.1 State of the Arts

2.1.1 On-Chip Memory Organization in Manycore Architectures

Modern manycore processors use either hardware-managed caches or software-managed scratchpads as their L1 memory. Scratchpad designs give each core or tile a shared banked memory without coherence. For example, the MemPool architecture is a manycore architecture with hundreds of individually programmable cores sharing a low-latency L1 Scratchpad Memory (SPM) [1] and TeraPool scales further beyond a thousand cores with a shared SPM [2]. Such systems achieve very low and predictable global access latency (e.g. 5 cycles) but require explicit Direct Memory Access (DMA) transfers and data placement by software.

In contrast, some other manycore designs use hardware-managed caches. In general purpose CPUs, recent systems include *AmpereOne* (96–192 Arm cores) with 64 KB L1D and 2 MB private L2 per core plus a 64 MB system-level cache on a mesh interconnect [3]; AMD's 128-core *EPYC Bergamo*, which composes eight 16-core CCDs around shared L3 slices for a large last-level cache [4]; and Intel's *Sierra Forest* Xeon, which tiles E-cores into clusters that share L2/L3 slices over a mesh fabric [5]. These manycore CPU designs target control-flow-heavy code with irregular and discontinuous memory access. Compared to compute-intensive accelerators, cache hierarchies offer better performance for workloads with unpredictable data access and potential for temporal locality and reuse. In GPGPUs, hardware-managed caches are also employed to improve programmability and performance under diverse workloads. NVIDIA's Hopper GPUs, for example, feature a unified per-SM memory block that serves as both L1 cache and shared memory [6]. This physical memory can be partitioned at runtime: one portion operates as a hardware-managed L1 cache, while the other functions as software-managed shared memory. This configurable design allows the architecture to adapt to both regular, contiguous data access patterns (where shared memory excels), and irregular, unpredictable access patterns (where L1 caching provides robustness without manual data management). It is also very inspiring to

configure the same block of memory into different management schemes.

In summary, dataflow processors and ASIC-based accelerators tend to favor scratchpads and static scheduling for their highly predictable access patterns, whereas general-purpose CPUs and GPUs utilize hardware-managed caches to handle control and irregular workloads more efficiently. Inspired by these architectural trade-offs, our project targets the RLC workload, which involves frequent control flow, linked-list traversal, pointer chasing, and synchronization mechanisms such as spin locks. These characteristics result in dynamic and unpredictable memory access patterns that are better served by a cache-based memory hierarchy. Therefore, we adopt a hardware-managed cache design as the L1 memory.

2.1.2 Cache Coherence Schemes

Traditional CPU coherence. Conventional multiprocessors maintain cache-line coherence system-wide via snooping (broadcast on a shared bus/ring) or directory-based protocols. While effective up to tens of cores, both approaches encounter fundamental scaling barriers in manycore, data-intensive settings: snooping saturates interconnect bandwidth and power with broadcast traffic; directories incur metadata that grows with sharers (and indirection latency), plus invalidation/ownership traffic that contends on the NoC under fine-grained sharing and write bursts. These effects manifest as coherence hotspots, rising miss/ack latencies, and energy overheads that compete with useful bandwidth, motivating alternative schemes in emerging designs [7, 8, 9].

Emerging schemes for heterogeneous and manycore systems. To accommodate accelerators and extreme parallelism, several directions relax granularity, timing, or scope. (i) *Coarse-grained (region) coherence* amortizes metadata/traffic by tracking permissions over larger regions, particularly effective for CPU–GPU sharing where data moves in bulk [10, 11]. (ii) *Timestamp-based coherence* (e.g., Tardis) replaces invalidations with logical time, eliminating sharer vectors and broadcasts while retaining performance comparable to directories on large core counts [9, 12]. (iii) *Scoped/domain-limited coherence* confines coherence to domains to cap protocol traffic and metadata. In ARM systems, shareability attributes define Non-shareable, Inner Shareable, and Outer Shareable domains; ACE/CHI interconnects and CMN-class meshes honor these domains for coherence snoop and maintenance behavior, enabling cluster-local coherence while avoiding system-wide broadcast [13, 14, 15]. HSA formalizes a complementary scoped memory model (work-group/agent/system) so software can request visibility at the smallest required scope [16, 17]. At the host–device boundary, CXL-cache and CCIX protocols create link-local coherent domains between CPUs and attached accelerators, maintaining coherence only within the CXL/CCIX fabric rather than the entire platform [18, 19, 20, 21]. These designs exemplify domain scoping: keep strong coherence where it pays (clusters, coherent links), and rely on software/fences when crossing domains.

Software-managed or no-coherence designs. Several research/industrial manycores deliberately omit global coherence, shifting responsibility to software/runtimes. Cerebras' Swarm/SwarmX interconnects all cores and systems for software-managed data movement without traditional caches or global hardware coherence [22]. Many RISC-V accelerator clusters (e.g., MemPool/TeraPool) adopt a shared, multi-banked L1 SPM (Tightly Coupled Data Memory (TCDM)) and rely on DMA and software synchronization rather than cache protocols [1, 23]. Intel's 48-core Single-Chip Cloud Computer (SCC) deliberately omits coherence; coherence is maintained via message passing and explicit cache management [24]. Industrial manycores such as Kalray MPPA organize cores into clusters that share a local multi-bank memory with software-managed coherence and no global coherence domain [25]. In the accelerator space, GPUs historically employ scoped visibility with fences and atomics; recent work [26] explores extending coherence

across hierarchical multi-GPU systems without full broadcast. These approaches reduce hardware complexity and traffic, but shift responsibility for correctness and data movement onto the runtime, an attractive trade-off when per-flow state is private and communication is structured.

The target RLC workload involves linked list traversal, branch-heavy control logic, and frequent synchronization using spinlocks and atomics, all within tight TTI-scale deadlines. In this project, we design the L1 memory to be cache-based to handle irregular and sparse memory access. We further simplify the cache coherence design by making all L1 cache partitions shared by default. At the same time, when certain data structures (e.g., local stack, per-user buffers or hot data) are accessed heavily by a single tile and are sensitive to contention, the system allows configuring selected L1 partitions as private to each tile, with software responsible for maintaining cache coherence. Carefully partitioning shared and private regions helps reduce interference while preserving flexibility and programmability.

2.1.3 Scalable Non-Blocking Cache Miss Handling

Hardware-managed caches remove the need for explicit data movement under irregular access patterns, but their effectiveness in a manycore system depends strongly on the ability to tolerate cache misses. When many cores and accelerators independently access sparse or data-dependent structures, numerous misses can be outstanding simultaneously. A blocking or weakly non-blocking cache quickly serializes these accesses and exposes the full latency of the lower memory hierarchy to the processing elements.

Conventional non-blocking caches track outstanding misses using dedicated Miss Status Handling Registers (MSHRs) or related miss-handling structures. Increasing the number of entries improves memory-level parallelism, but the associated storage, associative lookup, arbitration, and dependency-tracking logic grow with the number of concurrent misses. This scaling cost becomes particularly important in lightweight manycore clusters, where the cache controller and its miss-handling structures compete for area and power with the processing elements and the cache SRAM itself. Prior work has therefore explored more scalable MSHR organizations, including set-associative or hashed structures [27, 28] and dynamically allocated miss entries [29].

An alternative direction is to reuse existing cache capacity for miss tracking rather than provisioning a separate fixed-size miss-handling structure. This observation is motivated by the fact that a substantial fraction of cache lines may exhibit little or no reuse before eviction [30, 31, 32]. MiCache [33], for example, unifies cache and MSHR entries within a shared storage structure, demonstrating that cache capacity can be repurposed dynamically for outstanding misses.

However, existing approaches still leave several limitations for the type of manycore memory system considered in this project. Many primarily target read-miss tracking, while a shared cache serving multiple scalar and vector requesters must also efficiently handle concurrent and potentially partial write misses. Miss-handling capacity can also remain constrained by fixed per-miss metadata organization, particularly when many requests target the same cache line under long refill latency. Finally, much prior work focuses on architectural simulation or isolated cache structures, while a manycore design must also consider timing closure, SRAM organization, interconnect integration, and physical implementation.

These observations motivate the *In-Situ Cache* architecture adopted in CachePool. In-Situ Cache eliminates a dedicated MSHR structure by repurposing cache-line resources to represent outstanding read and write misses directly within the cache. This allows miss-handling capacity to scale with available cache resources while keeping the controller compact, making it particularly suitable for the high memory-level parallelism generated by irregular dataplane workloads.

2.1.4 On-Chip Interconnection

The on-chip interconnection is a critical component in connecting Processing Elements (PEs) with memory subsystems in a cluster. Existing approaches can be broadly categorized into (1) low-latency crossbars (Xbars) and (2) router-linked meshes.

Xbars provide low-latency access by enabling direct connections between all endpoints, but their scalability is limited by quadratic growth in routing complexity and wiring overhead. Multi-level memory hierarchies can act as coalescers to reduce the number of Xbar endpoints [34], but at the cost of increased latency and data duplication across hierarchy levels. Hierarchical Xbars also help relieve routing pressure, but the effective bandwidth per core decreases as the system scales [35, 36]. Software techniques such as task scheduling [37, 38] and dynamic address reallocation [39] can reduce interconnect pressure, but they cannot fundamentally overcome the bandwidth limitations of Xbar-based designs and impose additional programming complexity.

Meshes, by contrast, offer better scalability, with latency increasing linearly in the number of hops. The *Celerity* manycore system [40] employs a 2D-mesh Nework on Chip (NoC) with a Partitioned Global Address Space (PGAS) memory system to connect 496 cores, but relies heavily on software to tolerate the high communication latency, complicating programmability. The *HammerBlade* architecture [41] uses a half-Ruche NoC [42] to connect cores and cache banks, but still struggles to sustain non-blocking memory accesses at scale, even with support for up to 63 outstanding requests.

More recent designs attempt to combine the low-latency of Xbars with the scalability of meshes through hybrid interconnection schemes. TensTorrent's *Wormhole* architecture [43] uses Xbars locally to connect five PEs and memory banks within a cluster, and a router-linked 2D-torus to connect 80 clusters globally. Similarly, *ET-SoC-1* [44] clusters 32 cores using Xbars and scales further to 1088 cores through a 2D-mesh NoC. The *TeraNoc* architecture [45] follows a comparable approach, replacing the hierarchical Xbars in *TeraPool* with a hybrid Xbar–NoC interconnect to balance latency and scalability.

Overall, interconnection fabrics in manycore systems expose a fundamental trade-off between latency, scalability, and programmability. Xbars deliver low-latency access but face quadratic growth in complexity, meshes provide scalable layouts but incur higher and variable latency, and hybrid designs attempt to balance the two. While software techniques can mitigate some of these challenges, they increase programming complexity and do not fundamentally resolve the bandwidth limitations. This underscores the need for interconnect solutions that sustain high utilization at scale without prohibitive hardware or software overhead.

2.2 Project Objectives, Design Philosophy, and Contributions

The *ManyRVData* project targets scalable manycore processing for irregular and synchronization-intensive dataplane workloads. These workloads combine fine-grained scalar control, data-parallel computation, sparse and pointer-based memory accesses, and frequent communication among many concurrent processing elements. Such access patterns are difficult to serve efficiently with explicitly managed scratchpad-memory hierarchies, where software techniques such as tiling and double buffering rely on regular and predictable data movement. At the same time, conventional private-cache hierarchies introduce substantial coherence and interconnect complexity when scaled to hundreds of processing elements.

To address these challenges, ManyRVData develops *CachePool*, a scalable scalar–vector manycore architecture centered around a logically shared but physically distributed cache hierarchy. The design seeks to retain a simple shared-memory programming model while exposing sufficient control over data placement, memory sharing, synchronization, and execution resources to adapt the architecture to different workload characteristics.

The main design principles of CachePool are summarized below.

1. *Tightly coupled scalar–vector execution with configurable resource sharing.*

CachePool combines lightweight Snitch scalar cores with compact Spatz vector coprocessors. Scalar cores handle control-intensive and irregular execution, while Spatz accelerates data-parallel regions using multiple vector memory ports and SIMD arithmetic lanes. The tight accelerator interface allows scalar and vector execution phases to alternate at fine granularity without requiring a separate offload runtime. CachePool additionally supports both 1S1V and 2S1V Core Complexes. In the latter configuration, two Snitch cores share one Spatz instance, allowing the scalar-to-vector resource ratio to be adapted to workload requirements while reducing the area cost of replicated vector units. The Core Complex organization and vector-unit-sharing mechanism are described in Section 3.2.

2. *A logically shared, physically distributed L1 cache.*

   The L1 data cache is exposed as one shared memory space while its banks are physically distributed across Tiles and Groups. This organization avoids replication of independently cached copies and therefore eliminates the need for a conventional hardware cache-coherence protocol in the default configuration. The physically distributed organization allows aggregate capacity and bandwidth to scale with system size, while the hierarchical interconnect provides access to both local and remote banks. The resulting architecture therefore combines a shared-memory abstraction with a physically non-uniform access topology. The distributed L1 organization is detailed in Section 3.5.

3. *Software-controlled data placement and cache partitioning.*

   Because physical access distance matters even within a logically shared cache, CachePool exposes programmable address mapping and partitioning mechanisms. Software-controlled address scrambling adjusts the interleaving granularity used to distribute cache lines across banks, Tiles, and Groups. This allows highly parallel kernels to spread traffic for bandwidth, while locality-sensitive kernels can keep larger contiguous regions closer to their consumers. The same interconnect also implements configurable private/shared cache partitions independently of the cache controller. Any subset of the four banks in a Tile can be assigned to a private region, with private scope selected at either Tile or Group granularity, while the remaining banks stay globally shared. The fully private configuration can consequently decompose the machine into smaller logical clusters when stronger locality or isolation is beneficial. The hardware mechanisms are described in Sections 3.5.2 and 3.5.3, while their runtime control is described in Sections 5.3.1 and 5.3.2.

4. *Scalable non-blocking cache control.*

   CachePool integrates the In-Situ Cache controller as the shared L1 cache controller. Rather than relying on dedicated miss-status and write-buffer storage that grows separately from the cache capacity, In-Situ Cache repurposes cache-line resources to track outstanding read and write misses. This allows miss-handling capacity to scale with available cache resources while maintaining a compact hardware organization. The controller is complemented by request coalescing, single-way lookup, physical-design-oriented replacement and metadata optimizations, and a wide cache-line refill interface. Together, these features target the large number of concurrent independent misses generated by sparse and irregular dataplane workloads. The cache-controller organization and miss-handling mechanisms are described in Sections 3.6 and 3.6.3 to 3.6.5.

5. *Hierarchical interconnects matched to access granularity.*

   CachePool separates fine-grained core-generated memory traffic from cache-line refill traffic and uses different interconnect organizations for the two. Within a Tile, multiple parallel all-to-all crossbars connect corresponding scalar and vector memory ports from the Core Complexes to the local L1 banks. Decomposing a monolithic crossbar by memory-port class reduces routing complexity while preserving parallel bandwidth. Within and across Groups,

fine-grained load/store traffic is carried through hierarchical local forwarding and multiple independent narrow meshes. Several parallel 32 bit mesh paths can be assigned to each Tile position, allowing bandwidth and routing resources to be adapted to physical-design constraints. Cache-line refills use a separate 512 bit two-dimensional mesh connected to distributed off-chip memory endpoints. Separating the narrow core-access network from the wide refill network avoids forcing both traffic classes onto a single interconnect optimized for neither. The L1 data-access hierarchy and refill network are described in Sections 3.7 and 3.8.

6. *Hierarchical synchronization and lightweight runtime control.*

   CachePool provides hardware support for synchronization at the Tile, Group, and system levels. The hierarchical barrier supports partial participation as well as multiple independent barrier slots, enabling synchronization domains to match the physical and logical decomposition of the machine. Memory-mapped configuration registers expose cache partitioning, address mapping, cache maintenance, barrier participation, Spatz ownership, and other architectural controls to software. A lightweight runtime abstracts these interfaces and provides topology discovery, synchronization, memory allocation, cache-management, and work-distribution primitives. The hardware synchronization and control mechanisms are described in Sections 3.10 and 3.10.2, and the corresponding software support is described in Section 5.

Based on these principles, the current ManyRVData implementation makes the following main contributions.

1. **A scalable multi-Group CachePool architecture.**

   We developed and integrated a hierarchical manycore architecture composed of Groups, Tiles, and configurable scalar–vector Core Complexes. The current large-scale configuration supports a 4×4 Group mesh with four Tiles per Group and four Core Complexes per Tile, providing up to 256 scalar cores in the 1S1V organization and supporting larger hart counts with the 2S1V organization. The complete hierarchy and system organization are described in Sections 3, 3.1 and 3.4.

2. **Configurable scalar–vector Core Complexes.**

   We integrated Snitch and Spatz into configurable 1S1V and 2S1V Core Complexes. The dual-scalar organization includes hardware support for safe Spatz sharing, including free and software-controlled lock modes and explicit ownership transitions. The Core Complex and its memory interfaces are detailed in Section 3.2.

3. **Distributed and partitionable shared-L1 organization.**

   We implemented a 16 MiB logically shared L1 cache physically distributed across the full system. The L1 interconnect supports programmable address scrambling and mixed private/shared bank configurations, including Tile-private and Group-private regions. These mechanisms are implemented in the address-routing layer and remain independent of the cache-controller microarchitecture. The organization is described in Sections 3.5, 3.5.2 and 3.5.3.

4. **In-Situ non-blocking L1 cache controller.**

   We integrated and extended the In-Situ Cache architecture for CachePool. The controller reuses cache-line storage for unified read- and write-miss handling, supports dynamic expansion of miss metadata, and incorporates physical-design-oriented optimizations such as single-way lookup and simplified replacement. This provides scalable miss concurrency without requiring a dedicated MSHR structure sized independently of the cache. The controller is described in Sections 3.6 and 3.6.3 to 3.6.5.

5. **Hierarchical L1 data-access and refill networks.**

   We developed a multi-level interconnect architecture consisting of decomposed Tile-local crossbars, Group-local communication, multiple parallel narrow inter-Group meshes for fine-grained accesses, and a dedicated 512 bit refill mesh for cache-line traffic. The architecture is parameterized to adapt the number of routing resources and external-memory endpoints to different system scales and physical-design constraints. The two traffic classes and their corresponding networks are described in Sections 3.7 and 3.8.

6. **Hierarchical synchronization and system control.**

   We implemented a hierarchical hardware barrier supporting Tile-, Group-, and system-level synchronization, partial participation, and multiple independent barrier slots. We also introduced a memory-mapped control interface for cache configuration, maintenance, address mapping, Spatz sharing, boot control, and execution status. These mechanisms are detailed in Sections 3.10, 3.10.1 and 3.10.2.

7. **Runtime and programming support.**

   We developed a lightweight runtime that exposes the CachePool topology and architectural control mechanisms to software. The runtime provides core/Tile/Group identification, hardware and software barriers, atomic operations, memory fences, dynamic memory allocation, cache-maintenance operations, address-scrambling and cache-partition configuration, Spatz ownership control, and topology-aware work-distribution support. The runtime is described in Section 5.

8. **Fast system-level simulation with GVSoC.**

   We developed a configurable GVSoC model of CachePool for fast end-to-end simulation and architectural exploration. The model executes the same application binaries as the RTL implementation and exposes major topology parameters at simulation time, allowing larger configuration sweeps without RTL re-elaboration. It includes models of the distributed cache and interconnect hierarchy and can optionally be coupled to DRAMSys for detailed off-chip memory timing. The simulator and its validation are described in Sections 6, 6.1 and 6.5.

9. **Dataplane-oriented workload and kernel development.**

   We developed and adapted representative workloads including RLC dataplane kernels, linked-list and synchronization micro-kernels, dense and sparse linear-algebra kernels, and vectorized compute kernels. These workloads are used to characterize memory access behavior, guide architectural development, and evaluate the interaction between scalar control, vector execution, cache locality, synchronization, and memory-system bandwidth. The RLC workload model and software organization are described in Sections 8, 8.1 and 8.2.

10. **Architectural side explorations.**

    In parallel with the main shared-L1 architecture, we investigated several alternative design points, including private write-through L0.5 caches with both hardware-managed and software-managed coherence, folded-cache organizations, and alternative cache/interconnect organizations. These studies help quantify the trade-offs between locality, coherence complexity, miss latency, interconnect traffic, and implementation cost. The folded-cache organization is introduced in Section 3.6.6, while the private-cache exploration is documented separately in the appendix.

Major developments since the stage report. The previous stage report established the initial CachePool direction, including tightly coupled scalar–vector execution, a shared cache-based L1 hierarchy, the In-Situ non-blocking cache concept, preliminary software-controlled private partitions, and a hierarchical interconnect. During the second project stage, these concepts were extended into a complete scalable system and refined according to the observed requirements of the target dataplane workloads. The resulting architecture is described in detail in Section 3.

The architecture was scaled from the preliminary Tile-centric organization to a hierarchical multi-Group system with explicit Tile-, Group-, and inter-Group communication structures. The final system integrates 16 Groups arranged as a 4×4 mesh, with four Tiles per Group and four Core Complexes per Tile. This scaling work replaces the stage-report design, in which cross-Tile communication and the higher-level interconnect were still future extensions, with a complete system-level topology and physically distributed shared-L1 organization; the resulting hierarchy is detailed in Sections 3 and 3.7.

The memory system was extended with programmable address scrambling and a generalized bank-level partitioning mechanism. Where the stage report considered private L1 partitioning primarily as a future Tile-local optimization, the final architecture supports mixed shared/private configurations from zero to four private banks per Tile, with the private scope selectable at either Tile or Group granularity. The resulting mechanism allows software to trade globally distributed bandwidth against stronger locality and isolation without modifying the cache-controller microarchitecture. This development directly targets RLC behavior, where some data structures are globally shared while per-user state, hot buffers, and synchronization-sensitive structures can benefit from more localized placement. The corresponding cache-organization mechanisms and their software usage are described in Sections 3.5.3 and 5.7.

The interconnect was also substantially redesigned as the architecture moved beyond a single Tile. The stage-report refill path was based on a crossbar-oriented organization, and the cross-Tile fabric had not yet been implemented. The final system instead combines decomposed Tile-local crossbars, Group-local forwarding, multiple parallel 32 bit inter-Group mesh planes for fine-grained L1 data accesses, and a separate 512 bit two-dimensional refill network for cache-line traffic. The resulting hierarchy separates traffic classes by transfer granularity and provides a physically scalable path from local low-latency accesses to system-wide communication. The narrow L1 data-access network and the wide refill network are described separately in Sections 3.7 and 3.8.

The scalar–vector subsystem was extended from the original 1S1V organization to support 2S1V Core Complexes in which two Snitch cores share one Spatz vector coprocessor. The final design includes explicit hardware arbitration and software-controlled ownership modes for safe Spatz sharing. Core-side latency tolerance was also integrated more explicitly with the distributed memory hierarchy through reorder buffers, scoreboard-based dependency tracking, multiple outstanding memory accesses, and Spatz instruction-level parallelism. These changes allow the scalar-to-vector resource ratio and memory-level parallelism to be better matched to mixed control and data-processing phases. The configurable Core Complex organizations and their sharing mechanisms are presented in Section 3.2.3, while their software-level implications are discussed in Section 5.7.

The In-Situ Cache controller progressed from the preliminary cache-line-based miss-handling concept reported at the project midpoint to a fully integrated controller within the scaled CachePool hierarchy. The final implementation supports unified read- and write-miss handling, dynamic expansion of miss state, request coalescing, pipelined request and refill processing, and physical-design-oriented lookup and replacement optimizations. This maturation is particularly relevant to sparse and pointer-based dataplane accesses, where many independent memory operations can remain outstanding simultaneously. The final controller organization is described in Section 3.6.2.

Table 1: Reference RLC dataplane workload targets used throughout the report.

Target	Users	DL Throughput	Packet Size	DL Packets/Slot
Single-user peak	1	20 Gbit/s	1350 B	1157
Multi-user peak	48	50 Gbit/s	800 B	4882
Multi-user average	4800	20 Gbit/s	800 B	1963

Synchronization and runtime control were expanded into first-class system features. The final architecture implements a hierarchical hardware barrier spanning Tile, Group, and system levels, with partial participation and multiple independent barrier slots. The runtime exposes topology discovery, memory allocation, fences and atomics, cache maintenance, address scrambling, cache partitioning, barrier control, Spatz ownership, and topology-aware work distribution. These facilities allow software to exploit the architectural mechanisms according to the locality, sharing, synchronization, and scalar–vector characteristics of each kernel. The corresponding runtime mechanisms and hardware–software co-design guidelines are discussed in Sections 5 and 5.7.

Finally, the final project stage broadened the evaluation infrastructure beyond the RTL-centric flow of the stage report. A configurable CachePool model was developed in GVSoC for fast end-to-end simulation and architectural exploration using the same application binaries as the RTL implementation. The project also expanded the kernel suite, system-level physical implementation, and performance analysis, and investigated additional architectural directions including folded-cache organizations and private write-through L0.5 caches. The resulting evaluation methodology and workload results are presented in Section 9, while the physical implementation of the complete system is discussed in Section 4. Together, these developments transform the preliminary architecture reported at the project midpoint into the complete CachePool platform evaluated in this final report.

2.3 Reference Dataplane Workload Targets

To provide a consistent reference for architecture sizing and workload evaluation, the project considers three representative RLC dataplane traffic scenarios provided by the project partner. These scenarios capture different combinations of per-user throughput, aggregate traffic, and concurrency, and are used throughout the report as common workload targets.

The **single-user peak** target stresses the maximum packet-processing rate associated with one active user and serves as a reference for per-flow latency and sustained processing throughput. The **multi-user peak** target represents a moderate number of simultaneously active users under the highest aggregate downlink throughput and therefore emphasizes parallel processing capacity, synchronization, and shared-memory bandwidth. The **multi-user average** target represents a much larger population of users operating at lower average per-user rates, emphasizing scalability, concurrency, memory footprint, and resource sharing across many independent RLC entities.

These targets provide a common reference for the architecture, software, and evaluation sections of the report. The detailed workload implementation and final measurement methodology are introduced later together with the corresponding RLC evaluation.

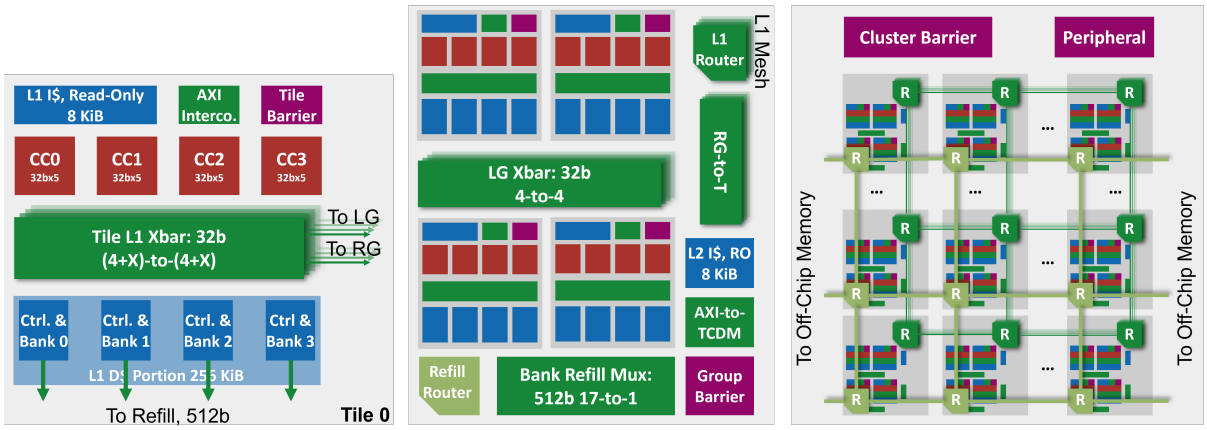


Figure 1: Top-level organization of CachePool, showing the hierarchical arrangement of Groups, Tiles, and Core Complexes together with the distributed shared L1 memory system and the main interconnect structure.

3 CachePool Architecture

CachePool is the manycore architecture developed within the ManyRVData project. It combines heterogeneous scalar–vector processing elements with a physically distributed, software-configurable shared L1 data cache. The current system is organized hierarchically into 16 Groups connected through a 2D mesh. Each Group contains 4 Tiles, and each Tile integrates 4 Core-Complexs (CCs). This hierarchy provides a scalable organization for both the compute resources and the memory interconnect, while constraining long-distance communication to maintain physical implementation feasibility.

The memory system distinguishes between fine-grained core-generated memory accesses and cache-line refill traffic. Load and store requests are transported through a hierarchy of local crossbars and multiple narrow 2D meshes, whereas cache-line refills are carried by a separate 512 bit-wide 2D mesh connected to a configurable number of HBM endpoints. The L1 data cache is physically distributed across all Tiles but forms a single logical memory in the default configuration. Software can further partition the cache at bank granularity into globally shared and Tile- or Group-private regions, allowing the effective memory-sharing domain to be adapted to the workload.

3.1 System Overview

The current CachePool configuration comprises 16 Groups, with 4 Tiles per Group and 4 CCs per Tile. Its physical compute hierarchy can therefore be summarized as

$$\text{System} \rightarrow 16 \times \text{Group} \rightarrow 4 \times \text{Tile} \rightarrow 4 \times \text{CC}.$$

Each CC combines one or more scalar Snitch cores with a Spatz vector coprocessor, as detailed in Section 3.2.

Each Tile contains four L1 data-cache banks with a combined capacity of 256 KiB. Across the 64 Tiles in the system, these banks form a physically distributed 16 MiB L1 data cache. In the default configuration, all banks belong to a single globally shared address space and are accessible by every core in the system. Because the L1 is shared rather than replicated in private coherent copies, CachePool does not require a hardware cache-coherence protocol between cores.

The terms *Group* and *Tile* refer to fixed levels of the physical implementation hierarchy. In contrast, throughout this report, we use *cluster* to denote a logical set of cores sharing one L1 memory domain. This distinction becomes relevant when cache partitioning is enabled.

Software can restrict selected L1 banks to a Tile or Group, thereby changing the effective cluster organization without modifying the underlying physical Group–Tile topology.

3.2 Core Complex

3.2.1 Snitch Scalar Core

ManyRVData employs single-stage 32 bit RISC-V *Snitch* cores [46] as scalar host processors, supporting the RV32I base Instruction Set Architecture (ISA). Vector instructions are offloaded to the Spatz vector coprocessor, while optional 32 bit floating-point support can be enabled depending on the selected system configuration.

As in MemPool and TeraPool, the Snitch Load Store Unit (LSU) integrates a scoreboard to track outstanding memory operations and to guarantee forward progress in the absence of read-after-write (RAW) hazards. The LSU supports multiple outstanding requests, allowing Snitch to issue a sequence of loads and stores without waiting for earlier responses to return [47]. This capability is particularly important in CachePool, where accesses may target local banks, remote Tiles, remote Groups, or off-chip memory through the cache hierarchy. As a result, memory responses may return with non-uniform latency, while the core still preserves the required architectural ordering.

Although Snitch uses a compact in-order scalar pipeline, its memory interface is non-blocking and can track multiple outstanding accesses. A scoreboard tracks dependencies between instructions, while a 16-entry reorder buffer decouples memory-request issue from potentially variable response latency. The detailed memory-side behavior is described in Section 3.2.5.

3.2.2 Spatz Vector Coprocessor

The streamlined 32 bit vector coprocessor *Spatz* [48] is tightly coupled to one or two Snitch scalar cores within a CC. In the current ManyRVData configuration, each Spatz instance contains 4 vector lanes and exposes four independent 32 bit memory ports, providing an aggregate load/store width of 128 bit per cycle before interconnect contention.

Spatz implements RVV 1.0 through a generic accelerator interface and uses a compact latch-based Vector Register File (VRF). Its controller employs a scoreboard to track dependencies and to support vector chaining across instructions. The dedicated Vector Load Store Unit (VLSU) can issue multiple outstanding memory requests, enabling the vector unit to tolerate the variable access latency introduced by the distributed L1 and the remote memory hierarchy. Optional 32 bit floating-point support can be enabled for both scalar and vector floating-point operations.

Spatz is designed to expose both memory-level and instruction-level parallelism. Its four independent VLSU ports support multiple outstanding memory operations, while a four-entry instruction window allows independent vector instructions targeting different functional units to overlap in execution. Dependency tracking and vector chaining enable this parallelism without requiring strictly sequential completion of vector instructions. The corresponding execution and memory-side mechanisms are detailed in Section 3.2.5.

3.2.3 Core-Complex Configurations and Vector-Unit Sharing

CachePool supports two CC organizations that trade scalar-core density against vector-resource replication. The baseline 1S1V configuration couples one Snitch scalar core to one Spatz vector coprocessor, while the 2S1V configuration allows two Snitch cores to share one Spatz instance. The selected organization is a compile-time system parameter and affects both the compute-resource ratio and the number of independent narrow memory ports that must be routed from each CC to the local L1 banks.

1S1V configuration. In the 1S1V organization, one Snitch core is tightly coupled to one four-lane Spatz coprocessor, as shown in Figure 2a. The Snitch core provides one independent 32 bit scalar memory port, while Spatz provides four independent 32 bit vector load/store ports. Each CC therefore contributes five narrow memory ports to the Tile-local L1 access network. This organization provides a dedicated vector engine for every scalar core and is suitable when vector execution constitutes a substantial fraction of the workload.

2S1V configuration and vector-unit sharing. In the 2S1V organization, two Snitch cores share one four-lane Spatz coprocessor, as shown in Figure 2b. The organization increases the number of independently executing scalar threads without replicating the complete vector datapath and VRF, allowing the scalar-to-vector resource ratio to be adapted to workloads in which vector resources would otherwise be underutilized. The two Snitch cores each retain an independent 32 bit scalar memory port, while the shared Spatz contributes its four 32 bit vector memory ports. A 2S1V CC therefore exposes six independent narrow memory ports toward the Tile-local L1 access network.

Because both Snitch cores can issue accelerator instructions to the same Spatz instance, the 2S1V configuration includes a dedicated sharing controller at the accelerator interface. The controller supports two operating modes for different software execution scenarios.

- **Free Mode:** requests from the two Snitch cores are selected in a round-robin manner. Because Spatz can execute and retire vector instructions out of order internally, ownership of the accelerator interface can switch only after all previously accepted instructions from the current sender have completed. Free Mode is selected by default after reset.
- **Lock Mode:** one Snitch core can explicitly claim ownership of Spatz through software. While the lock is active, accelerator requests are accepted only from the owner core. Requests issued by the non-owner core are rejected immediately and return an error response.

Transitions between Free Mode and Lock Mode may not complete immediately because vector instructions from the previous owner can still be in flight inside Spatz. Two intermediate transition states are therefore used to drain outstanding instructions and prevent ownership races during mode switching. The resulting control Finite State Machine (FSM) is shown in Figure 2c.

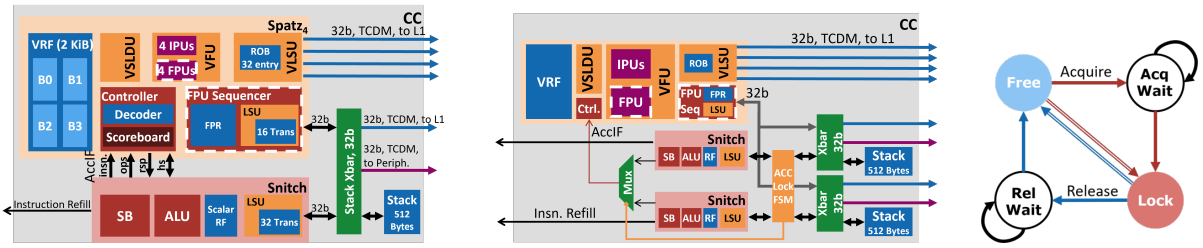
3.2.4 Core-Local Stack Memory

One stack implementation provides each Snitch core with a dedicated 1024 B SRAM for low-latency private stack accesses. All cores observe the same virtual address mapping for this SRAM, meaning that a stack address always resolves to the local SRAM associated with the issuing core rather than to a globally unique physical SRAM instance.

When the private stack capacity is insufficient, stack accesses can be redirected to the shared L1 cache. The backing region in L1 is computed as a function of the core identifier, so that each core is assigned a distinct overflow region. This organization preserves the convenience of a private stack abstraction while allowing deeper stack usage when required. The address mapping and overflow handling are transparent to the programmer.

3.2.5 Non-Blocking Memory Access and Dependency Tracking

Both Snitch and Spatz are designed to tolerate the variable memory latency introduced by the physically distributed L1 hierarchy. Their memory interfaces support multiple outstanding load and store operations, allowing subsequent instructions to continue execution while earlier memory transactions remain in flight, provided that no data dependency prevents progress.



(a) 1S1V Core Complex with one Snitch core and one dedicated four-lane Spatz coprocessor.

(b) 2S1V Core Complex with two Snitch cores sharing one four-lane Spatz coprocessor.

(c) FSM controlling Spatz sharing and ownership transitions in the 2S1V configuration.

Figure 2: Supported Core-Complex organizations and vector-resource-sharing mechanism. The 1S1V configuration dedicates one Spatz coprocessor to each Snitch core, whereas the 2S1V configuration increases scalar-core density by sharing one Spatz between two Snitch cores. The sharing FSM controls accelerator-interface arbitration and safe transitions between free and software-controlled ownership.

Snitch uses a reorder buffer at its memory interface to decouple instruction execution from memory-response latency. The CachePool configuration provides 16 reorder-buffer entries on the integer side. This depth is chosen together with the relatively small scalar register file of 32 $\times$ 32 bit, since the available architectural registers ultimately limit the number of independent memory operations that can be kept useful in flight. Memory dependencies are tracked by the Snitch scoreboard, which stalls only when a subsequent instruction depends on an outstanding result or would otherwise violate a required dependency.

Spatz applies the same principle at a larger scale. The scalar floating-point path contains a 16-entry reorder buffer, while each of the four independent VLSU memory ports provides a 32-entry reorder buffer. The four VLSU ports can therefore maintain a large number of independent memory transactions concurrently, helping the vector engine tolerate both cache misses and the additional latency of accesses to physically remote L1 banks. As in Snitch, dependency hazards are tracked by a scoreboard so that independent operations can proceed while dependent instructions wait for the required operands or memory responses.

Non-blocking memory access is complemented by instruction-level parallelism within Spatz. Spatz maintains an instruction window of four vector instructions and allows instructions targeting different functional sub-units to execute out of order with respect to one another. The main sub-units include the Vector Functional Unit (VFU) for arithmetic operations, the VLSU for vector memory accesses, the Vector Slide Unit (VSLDU) for slide and data-rearrangement operations, and the controller for vector configuration and CSR operations. Different sub-units can operate concurrently, while each individual sub-unit processes at most one vector instruction at a time.

Spatz additionally supports vector chaining across different functional units. A consumer instruction can therefore begin processing elements produced by an earlier instruction without waiting for the complete vector result to be written back first, provided that the corresponding dependency and structural constraints are satisfied. Together with the four-entry instruction window and the independently non-blocking VLSU ports, this allows memory, arithmetic, and data-movement operations to overlap substantially.

The scoreboard is responsible for enforcing dependencies across these concurrently executing operations. This allows both Snitch and Spatz to expose memory-level and instruction-level parallelism without requiring the programmer to explicitly serialize independent accesses. The mechanism is particularly important in CachePool because L1 access latency is non-uniform:

local accesses can complete quickly, while requests to remote Tiles or Groups may remain in flight for substantially longer. The reorder buffers and scoreboards allow such long-latency transactions to coexist with independent execution instead of blocking the processor pipeline immediately.

3.3 Tile Architecture

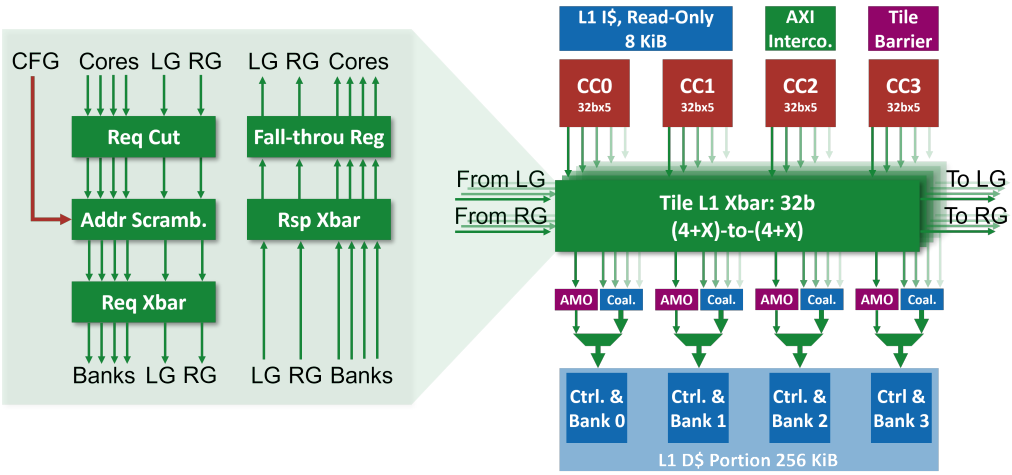


Figure 3: Tile-level organization of CachePool, showing the four Core Complexes, the local L1 data-cache banks, and the Tile-local interconnect resources.

3.3.1 Tile Organization

Each Tile integrates 4 CCs and 4 L1 data-cache banks with a combined capacity of 256 KiB. The Tile also contains the local levels of the instruction-cache hierarchy together with the interconnect resources required to access its local L1 banks and to forward requests toward neighboring Tiles.

In the 1S1V configuration, each CC contributes one Snitch memory port and four Spatz memory ports. In the 2S1V configuration, each CC contributes two Snitch memory ports and four Spatz memory ports. These memory ports remain logically separated throughout the Tile-local access network to preserve parallelism and avoid concentrating all traffic on a single shared datapath.

3.3.2 Local L1 Access Network

Local L1 accesses are routed through multiple parallel all-to-all crossbars connecting the CC memory ports to the four L1 banks within the Tile. Each logical memory-port position is assigned an independent crossbar that connects the corresponding port of every CC to the four local banks. Consequently, the Tile-local network contains 5 parallel crossbars in the 1S1V configuration and 6 parallel crossbars in the 2S1V configuration.

This decomposition avoids concentrating all requests into a single wide monolithic crossbar and provides independent arbitration for the Snitch and Spatz memory ports. For a crossbar with fixed data width, the switching complexity scales approximately as $\mathcal{O}(N_{\text{in}}N_{\text{out}})$. A flat organization with $N_{\text{cc}}N_{\text{port}}$ request inputs and $N_{\text{port}}N_{\text{bank}}$ output ports therefore scales as

$$\mathcal{O}_{\text{flat}} = \mathcal{O}\big(N_{\text{cc}}N_{\text{port}}^2N_{\text{bank}}\big)\,.$$

In CachePool, the request ports are instead decomposed into N_{port} independent crossbars, each connecting N_{cc} inputs to N_{bank} local banks. The resulting aggregate complexity scales as

$$\mathcal{O}_{\text{decomp}} = \mathcal{O}(N_{\text{port}}N_{\text{cc}}N_{\text{bank}})\,,$$

removing one factor of N_{port} compared with the flat organization. Besides reducing switching and arbitration complexity, this decomposition improves physical routing feasibility by distributing the wiring across several narrower structures.

3.3.3 Atomic Operations

Although the globally shared L1 organization removes the need for a hardware cache-coherence protocol, concurrently executing cores still require synchronization and memory-consistency primitives. An atomic unit is therefore inserted on the scalar-core memory path before the L1 data-cache controller to provide Atomic Memory Operations (AMOs) in compliance with the RISC-V "A" extension. Vector requests bypass this unit because atomic operations are currently supported only on the scalar path.

The atomic unit was originally designed for SPM-based systems with predictable access latency. In CachePool, its control logic is adapted to use handshake-driven request and response handling so that it can tolerate the variable latency introduced by cache hits, cache misses, and remote accesses.

Spatz memory requests instead bypass the atomic unit and are forwarded directly to the request coalescer, described in Section 3.6.1, before entering the cache controller.

3.4 Group Architecture

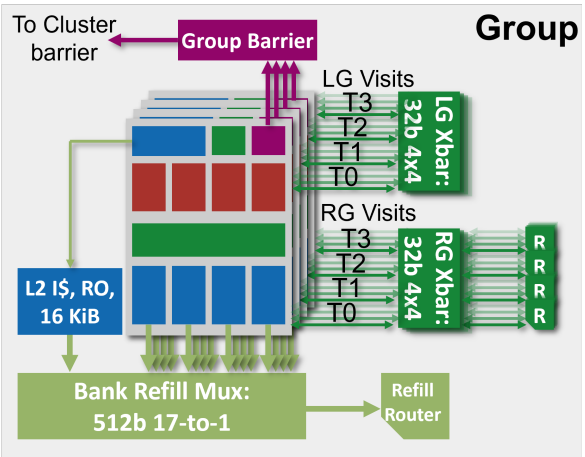


Figure 4: Group-level organization of CachePool, showing the four Tiles, the Group-local L1 interconnect, the inter-Group narrow network, and the wide refill NoC.

Each Group contains 4 Tiles and forms the intermediate physical hierarchy between the Tile-local resources and the global inter-Group network. The default configuration contains 4 Tiles per Group, while the number of Tiles remains parameterizable for reduced system configurations and faster verification runs. The Group provides local communication resources that allow accesses to be forwarded between its constituent Tiles without entering the inter-Group mesh.

The four Tiles occupy fixed relative positions within the Group. These positions are preserved across the global 2D mesh and define the geometric correspondence used by the narrow inter-Group networks. A remote request can therefore reach the Tile at the same relative position directly through the corresponding mesh, while requests targeting another Tile position require an additional Group-local forwarding stage.

The Group also defines one of the supported visibility domains for private L1 cache partitions. When Group-private partitioning is enabled, banks assigned to the private region are accessible by all cores within the Group but remain inaccessible to cores in other Groups.

The Group additionally contains the shared L2 instruction-cache level, which serves the instruction caches of all Tiles in the Group before refill traffic is forwarded to the global 512 bit refill mesh.

3.5 Distributed and Partitionable L1 Data Cache

3.5.1 Global Shared-L1 Organization

The CachePool data cache forms a single logical L1 memory while being physically distributed across all Tiles in the system. Each Tile contains four L1 banks with a combined capacity of 256 KiB, resulting in a total capacity of 16 MiB across the 64 Tiles.

In the default configuration, all L1 banks are globally accessible by every core in the system. Because the architecture does not maintain mutually incoherent private copies of cached data, no hardware cache-coherence protocol is required between cores. Instead, all cores access the same logical L1 memory through the hierarchical load/store interconnect described in Section 3.7.

3.5.2 Address Mapping and Software-Controlled Scrambling

The distributed L1 banks are interleaved through a software-controlled address-scrambling mechanism. Software can configure the scrambling granularity according to the locality and bandwidth requirements of individual workloads. The scrambling granularity must be larger than the 64 B cache-line size so that all bytes in one cache line map consistently to the same destination cache bank.

The programmable mapping provides a trade-off between locality and traffic distribution. A coarser scrambling granularity keeps larger contiguous address regions within the same Tile or Group and can therefore preserve spatial locality. A finer scrambling granularity distributes consecutive regions across more Tiles and banks, increasing the available aggregate memory bandwidth for highly parallel accesses. The scrambling parameters are programmed through memory-mapped system configuration registers, as described in Section 5.

3.5.3 Cache Partitioning

CachePool supports software-controlled partitioning of the four L1 banks in each Tile into *private* and *shared* regions. Partitioning is performed at the granularity of complete cache banks, resulting in the configurations summarized in Table 2.

Table 2: Supported private/shared L1 bank partitioning configurations within each Tile.

Private banks	Shared banks	Configuration
0	4	Fully shared (default)
1	3	Mixed
2	2	Mixed
3	1	Mixed
4	0	Fully private

Banks assigned to the shared region remain globally accessible by all cores in the system. For banks assigned to the private region, software additionally selects one of two visibility scopes:

- **Tile-private:** private banks are accessible only by cores located in the corresponding Tile.
- **Group-private:** private banks are accessible only by cores located in the corresponding Group.

The fully private configuration decomposes the system into multiple smaller logical clusters, where each cluster comprises the cores sharing the corresponding private L1 memory. Mixed configurations retain a globally accessible shared region for communication and shared data structures while providing private cache capacity and bandwidth to individual Tiles or Groups.

Cache partitioning is implemented through the address-mapping and scrambling logic in the L1 interconnect rather than within the cache controller itself. The mechanism is therefore independent of the particular cache-controller implementation used behind the L1 crossbar.

The partitioning scheme is controlled through memory-mapped configuration registers. One register selects the number of private and shared banks and the visibility scope of the private region. A second register defines the address boundary between the private and shared portions of the memory space. The address comparison determines whether an access is routed to the private or shared bank subset according to the configured partitioning scheme. The address-boundary setting is only active for mixed private/shared configurations and has no effect when all banks are configured as either fully shared or fully private.

3.6 L1 Cache Controller

3.6.1 Request Coalescer

A request coalescer is placed in front of the L1 cache controller to aggregate multiple narrow 32 bit memory requests targeting the same cache line into a wider request. This reduces redundant accesses to the cache controller and improves utilization of the 512 bit cache-line datapath.

The coalescer operates over a configurable request window. In the default configuration, the window contains two request slots, while the width can be adjusted at design time according to the expected access pattern and implementation constraints.

3.6.2 In-Situ Cache Overview

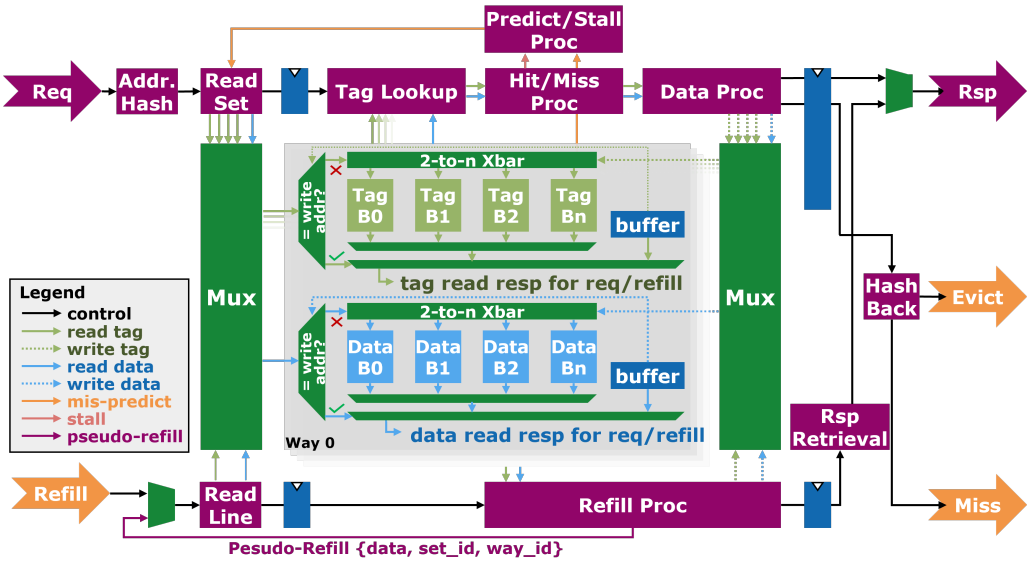


Figure 5: Block diagram of the In-Situ Cache controller, showing the request, tag/data access, refill, response, and eviction paths.

The L1 banks are managed by the *In-Situ Cache* controller [49], a scalable non-blocking cache architecture designed to tolerate long and variable refill latencies without relying on large dedicated miss-handling structures. Instead of storing outstanding-miss state in a conventional MSHR array and separate write buffer, In-Situ Cache repurposes otherwise available cache lines to track misses directly within the cache structure.

The controller extends the conventional invalid/valid cache-line state model with dedicated pending states for read and write misses. This enables unified handling of outstanding read misses, sub-cacheline write misses, and request merging through the same tag lookup mechanism. Because the miss-handling state is stored in cache-line resources, the available tracking capacity naturally scales with the cache itself rather than being fixed by a statically provisioned MSHR structure.

The current implementation further incorporates a set of microarchitectural and physical-design optimizations, including dynamic expansion of read-miss metadata, single-way data lookup through hash-based way prediction, pseudo-random replacement, and a folded cache organization. The folded-cache implementation and its physical-design implications are discussed separately in Section 3.6.6.

3.6.3 Unified In-Cacheline Miss Handling

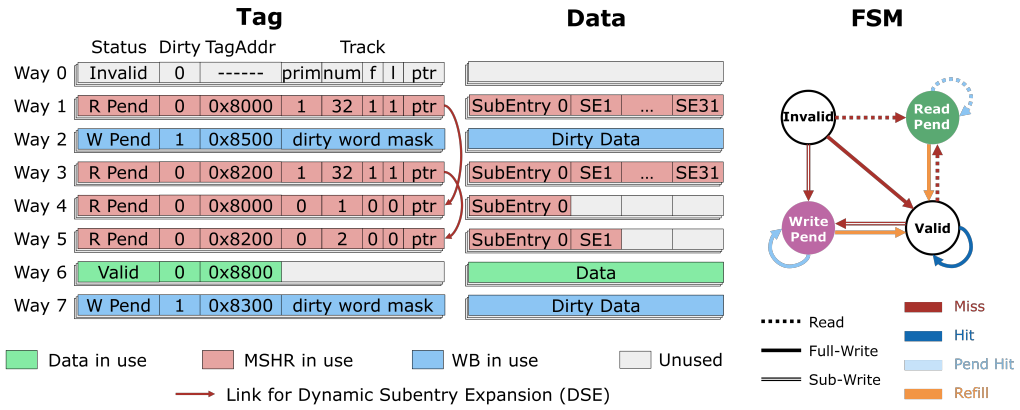


Figure 6: Cache-line organization and state transitions of the In-Situ Cache controller, including valid, read-pending, and write-pending states.

In-Situ Cache extends the conventional invalid/valid cache-line state model with two additional states, *read-pend* and *write-pend*, allowing outstanding read and write misses to be represented directly within cache lines. A hit on a pending line indicates that a request can be merged with an already in-flight miss, while a miss on all valid and pending states allocates a replacement line for miss tracking.

For a read miss, the selected cache line transitions to the read-pend state and records the metadata required to return the refill data to the requesting processing element. Subsequent requests to the same cache line can merge into the existing pending entry rather than issuing redundant refill requests. When the refill returns, the line transitions to the valid state and responses are generated for all merged requesters.

Sub-cacheline write misses are handled similarly through the write-pend state. The cache line stores the partially updated data together with a dirty-word mask that records which words have been modified while the refill is outstanding. Once the missing cache block arrives, the pending writes are merged with the refill data and the line transitions to the valid state.

The design therefore unifies read-miss tracking and write-miss coalescing within the cache itself, eliminating the need for separate dedicated MSHR and write-buffer storage.

3.6.4 Request and Refill Pipelines

The In-Situ Cache controller is organized as a three-stage pipeline with separate control paths for processing-element requests and memory refills. The request path performs address hashing

and way prediction in the first stage, tag comparison and prediction checking in the second stage, and hit response or miss generation in the final stage.

The refill path writes returned cache-line data directly to the location selected when the miss was allocated. For read-pending lines extended through DSE, the refill controller traverses the linked expansion lines and generates pseudo-refill operations to recover all request metadata and invalidate the expansion entries. This allows refill completion and response generation without requiring a separate miss-response buffer.

To avoid deadlock and guarantee forward progress, refill traffic is given bounded access to the internal structures and is only admitted when the required landing resources are available. Tag and data memories are implemented as pseudo-dual-port structures constructed from single-port SRAM macros, with forwarding and fixed-priority arbitration used to resolve same-bank conflicts.

3.6.5 Way Prediction and Replacement

A conventional set-associative cache reads all data ways on every lookup, which increases SRAM access energy and routing activity. In-Situ Cache instead uses a hash-based way-prediction mechanism that selects one data way for the initial access while all tag ways are still checked in parallel.

If the predicted way is incorrect, the request is replayed to access the actual hit way or another eligible replacement way. The controller combines this mechanism with pseudo-random replacement, avoiding the additional state and update logic required by an LRU policy while also excluding pending lines from replacement.

This organization reduces data-SRAM activity and simplifies replacement logic, while the occasional replay introduces only a small additional latency for mispredicted accesses.

3.6.6 Folded Cache Organization

CachePool's folded data-bank organization provides narrower SRAM interfaces while preserving cache capacity, associativity, and the software-visible line size. Four-way folding transforms 512-bit-wide data arrays into 128-bit-wide arrays with four times the depth. Cyclic placement distributes each line's parts across physical columns, supporting parallel access to a selected line when accesses do not contend. Folding controls the physical layout inside each cache controller; private/shared partitioning separately controls the controllers to which an address is routed.

Capacity-preserving transformation. Let W denote the ways, B the pseudo-dual banking factor, D the rows per way and pseudo-dual bank, and L the cache-line width in bits. Here B counts internal SRAM banks per way; private/shared L1 partitioning separately assigns cache controllers. The unfolded implementation uses WB single-port data SRAMs, each with D rows of L bits. Folding by F instead gives each SRAM FD rows of L/F bits. The macro count remains WB and capacity is preserved:

$$WB\,D\,L = WB\,(FD)\,(L/F). \tag{1}$$

For four ways, a 512 bit line, $F = 4$, and $B = 2$, the eight data SRAMs per controller change from 512 bit-wide arrays to 128 bit-wide arrays with four times the depth. Metadata arrays remain separate. The transformation preserves both the stored bits and the number of SRAM macros while providing a fourfold narrower interface.

Skewed placement across ways. CachePool distributes a line's parts across physical columns using a cyclic skew, enabling parallel selected-line access through narrower SRAM interfaces. Ways form groups of F; for $w = gF + \bar{w}$, part p, and original row r, the mapping is

$$c = gF + ((\bar{w} + p) \bmod F), \qquad r_{\text{fold}} = Fr + p. \tag{2}$$

The pseudo-dual bank index is preserved. Thus the F parts of one selected way occupy different columns and can be accessed in parallel when there is no competing operation. Figure 7 connects the capacity-preserving transformation to the physical placement of line parts. For the highlighted Way 1, parts P0–P3 occupy columns C1, C2, C3, and C0 at rows $4r$, $4r+1$, $4r+2$, and $4r+3$, respectively. The figure shows one fixed pseudo-dual bank index; its four SRAM macros are replicated for the second index when $B = 2$.

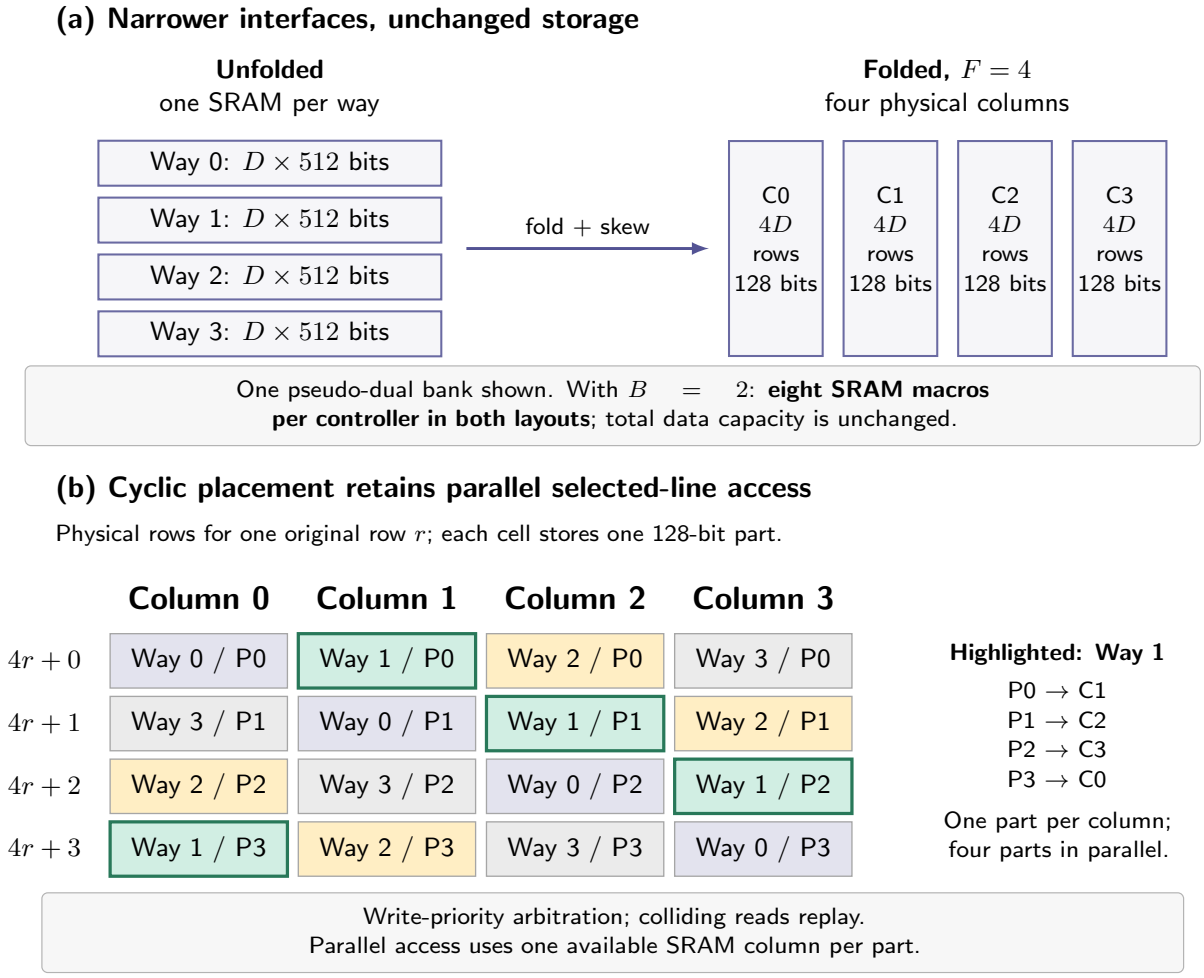


Figure 7: Capacity-preserving folded SRAM organization for four ways, $F = 4$, and a 512-bit line. One pseudo-dual bank is shown; the layout is replicated for the second bank. Colors identify logical ways. The highlighted way places its four parts in distinct physical columns at rows $4r + p$, enabling parallel access when those columns are available. This is a physical data-array transformation; cache partitioning is a separate address-routing mechanism.

Controller integration and arbitration. The implementation resides in the Tile's skewed data-bank network. `UseFoldedDataBanks` and `FoldWayGroup` select the layout, and the resulting factor propagates to the controller as `DataPartSplit`. The automatic fold-way group is four for the normal four-way cache. The factor must divide both the number of ways and the words per line. Narrow accesses identify a part from the line offset; refill, eviction, and maintenance retain the full-line semantics needed by the cache state machine. The request coalescer groups requests within each part-width SRAM access window.

Different ways can contend for the same physical column and pseudo-dual bank. The column arbiter prioritizes writes over reads, then the first requesting part within a priority class. A read colliding with another way's write replays until it receives a grant for its own address. Grant

propagation, part-qualified byte enables, and the forwarding buffer preserve data correctness across these accesses. Hash-based single-way selection supplies the selected line's data while retaining four-way associativity; replay resolves unsuccessful accesses. This organization uses single-way data lookup, with all-ways-active data reads excluded by the implementation checks.

SRAM organization and physical integration. Folding separates the local SRAM interface width from the logical cache-line width. Selection multiplexers, skew routing, and column arbitration connect the narrower macros to the full-line cache operations. The resulting aspect-ratio transformation gives physical integration narrower, deeper SRAMs while retaining cache capacity, associativity, and parallel selected-line access. System-level area and timing results characterize the combined cache and interconnect organization.

3.7 L1 Data Access Network

The L1 data access network carries fine-grained memory requests generated by the scalar and vector cores to the physically distributed L1 data-cache banks. It is distinct from the cache refill network described in Section 3.8, which transports wide cache-line transactions between the cache hierarchy and the external-memory interfaces.

Routing within the L1 data access network is performed hierarchically according to the physical location of the destination cache bank. The interconnect first determines whether the target bank resides in the issuing Tile, elsewhere in the same Group, or in a remote Group. Only the network resources required for the selected access class are traversed, avoiding unnecessary propagation through higher levels of the hierarchy.

3.7.1 Network Organization and Protocol

The L1 data access network is organized hierarchically into three access classes: intra-Tile, intra-Group, and inter-Group. Fine-grained scalar and vector memory accesses are transported using a lightweight TCDM-style request/response protocol rather than a full AXI fabric.

Each requester acts as a master and each L1 data-cache bank acts as a slave endpoint. A memory transaction therefore consists of a request routed from the issuing core toward the destination cache bank and a corresponding response returned to the requester. The request carries the target address, operation type, write data when required, access size, and metadata used for response ordering. The response returns read data or completion status together with the metadata required to associate it with the originating requester.

The hierarchical organization avoids requiring every requester to connect directly to every cache bank in the system. Local accesses terminate within the Tile, accesses to another Tile in the same Group traverse the Group-local interconnect, and accesses to a remote Group additionally traverse the inter-Group mesh. This preserves low latency for nearby accesses while allowing the same shared L1 address space to scale across the full system.

The local crossbars described in Section 3.3.2 handle accesses targeting banks within the issuing Tile. Requests targeting another Tile in the same Group are forwarded through Group-local crossbar resources. Accesses to remote Groups are transported through multiple narrow 2D meshes and are optionally followed by an additional Group-local forwarding stage when the destination Tile is not geometrically aligned with the incoming mesh.

3.7.2 Design Rationale and Network Decomposition

The L1 data access network is designed around both the fine-grained nature of core-generated memory traffic and the physical constraints of a large manycore implementation. Scalar and vector memory accesses are naturally word-granular, and the corresponding requester interfaces operate on 32 bit data words. Using substantially wider links for this traffic would increase wiring

Table 3: TCDM-style request/response interface used between processing elements and the L1 memory system.

Request		
Field	**Bits**	**Description**
addr	32	Target byte address
write	1	Load/store selector
amo	4	Atomic-operation encoding
data	32	Store request data
strb	4	Byte-enable mask for stores
size	3	Transfer-size encoding preserved from AXI transactions
user	15/16	User-defined metadata
q_valid	1	Request-valid handshake
p_ready	1	Response-ready handshake
Response		
Field	**Bits**	**Description**
data	32	Load response data
error	1	Error indicator
write	1	Store-response indicator
user	15/16	User-defined metadata
p_valid	1	Response-valid handshake
q_ready	1	Request-ready handshake
User Metadata		
Field	**Bits**	**Description**
core_id	2/3	Identifier of the sending scalar core within a Tile
tile_id	6	Identifier of the sending Tile
is_amo	1	Marks responses associated with atomic operations
req_id	5	Request identifier used exclusively for response reordering
is_fpu	1	Indicates requests originating from the floating-point path

cost and require additional adaptation between the core-level protocol and the network without providing a proportional benefit for typical fine-grained accesses. CachePool therefore preserves the native word width and scales aggregate bandwidth by instantiating multiple parallel narrow channels.

The hierarchy combines local crossbars with a multi-channel inter-Group mesh. Within a Tile and Group, crossbars provide low-latency communication over short physical distances. Extending such crossbars over the complete system would lead to rapidly increasing wiring complexity and long global connections. Conversely, routing every access through a flat mesh would increase the round-trip latency even for physically nearby cache banks. CachePool therefore uses a hybrid organization in which local communication remains crossbar-based while longer-distance inter-Group traffic is carried by a regular two-dimensional mesh. This follows the same general design principle as TeraNoC, where narrow word-width mesh channels are used for long-distance core-to-L1-bank communication while small crossbars preserve low local-access latency.

The number of parallel inter-Group channels is primarily constrained by physical implementation rather than selected solely from an architectural bandwidth target. CachePool targets a moderate standard-cell utilization of approximately 50–60% so that sufficient routing resources

remain available for the dense Tile-local interconnects, clock distribution, and global wiring. This requirement limits the practical physical size of a Group and consequently the amount of communication infrastructure that can be placed around its boundary.

At the Group edge, the feasible number of NoC ports is limited by several coupled implementation effects. First, physical pin density constrains how many independent network interfaces can be exposed along the boundary. Second, each additional port consumes routing tracks not only outside the Group but also internally, because all signals must be connected from the boundary interface to the corresponding router and local interconnect. Third, each NoC interface requires associated buffering and interface logic. To keep the wiring between these cells and the physical pins short, this logic is concentrated near the Group boundary, creating localized cell-density pressure. Increasing the number of channels therefore raises pin density, routing demand, internal fan-in wiring, and edge-localized cell concentration simultaneously. The selected channel count is consequently a physical-design trade-off between aggregate bandwidth and implementability.

Within this physical implementation envelope, CachePool instantiates multiple independent 32 bit mesh channels to increase aggregate inter-Group bandwidth while preserving regular and physically manageable wiring. The multi-channel organization also aligns naturally with the Spatz vector memory interface. Spatz exposes four independent vector load/store ports that can issue accesses concurrently, and these ports typically populate the four vector mesh channels simultaneously during vector execution. The available inter-Group bandwidth is therefore naturally exercised by vector workloads without requiring an additional router-remapping mechanism to redistribute traffic dynamically among the channels.

Each request remains associated with its corresponding network plane from source to destination. Cross-plane routing is intentionally not supported. This restriction reduces routing and arbitration complexity and preserves regular physical wiring, since each plane can be treated as an independent communication resource. The trade-off is that unused bandwidth in one plane cannot be dynamically borrowed by traffic assigned to another plane. This limitation is most visible for traffic generated only by the scalar cores, where some of the vector-oriented network capacity may remain idle, whereas vector execution naturally activates the parallel planes together.

Request and response traffic are transported on separate network paths. Requests travel from core-side masters toward the distributed L1 cache-bank slave endpoints, while responses return read data or completion information to the originating requester. Separating the two traffic directions removes cyclic dependencies between request and response messages and thereby simplifies deadlock-free operation. The separation also allows request and response buffering, arbitration, and physical routing to be handled independently.

The choice of a narrow multi-channel network is specific to fine-grained L1 data accesses. Cache refill traffic has fundamentally different characteristics: a refill transfers a complete 64 B cache line rather than an individual machine word. The separate refill network described in Section 3.8 therefore uses a 512 bit datapath, allowing one complete cache line to be transported in a single data beat. The two networks are thus specialized for different traffic granularities: multiple narrow channels provide scalable bandwidth for fine-grained core-to-cache-bank accesses, while the wide refill network efficiently transports cache-line data between the cache hierarchy and external memory.

3.7.3 Intra-Tile Access

An access targeting an L1 bank in the issuing Tile follows the direct path

$$\text{CC memory port} \rightarrow \text{local parallel Xbar} \rightarrow \text{local L1 bank}.$$

This is the shortest access path in the hierarchy and does not enter either the Group-level interconnect or the inter-Group meshes.

3.7.4 Intra-Group Access

For accesses targeting another Tile within the same Group, each Tile exposes a configurable number of ports toward the Group-local crossbar network. The default configuration provides two such ports per Tile.

The outgoing Group-local port is selected as a modulo function of the issuing core ID. This distributes requests from different cores across the available ports and increases the aggregate bandwidth available for accesses to neighboring Tiles. The number of Group-local ports and the corresponding source-to-port mapping are compile-time configurable to match the routing resources available in a particular physical implementation.

3.7.5 Inter-Group Narrow Meshes

Accesses targeting a remote Group are transported through multiple narrow 2D meshes with a topology similar to TeraNoC [45]. Each mesh hop introduces two cycles of forwarding latency due to the input and output buffering implemented at the routers. These pipeline stages are required to meet timing across the long inter-Group wires imposed by the physical dimensions of a Group.

Each Tile is associated with multiple 32 bit-wide mesh paths that connect geometrically corresponding Tile positions across neighboring Groups. In the default configuration, four parallel narrow meshes are instantiated per Tile position. This configuration has been verified to be routable in the physical design.

Using several narrow meshes in parallel increases aggregate inter-Group bandwidth without introducing a single physically wide global load/store fabric. The organization also preserves the native 32 bit granularity of scalar and vector memory-port requests throughout the long-distance interconnect.

3.7.6 Non-Geometric Destination Routing

The narrow meshes naturally terminate at the Tile occupying the same relative position in the destination Group. If a request targets a different Tile position within that Group, an additional local forwarding stage is required.

Such an access follows the path

$$\text{source Tile} \rightarrow \text{narrow mesh} \rightarrow \text{destination-Group Xbar} \rightarrow \text{destination Tile}.$$

The inter-Group mesh performs the long-distance transport between Groups, while the Group-local crossbar resolves the final Tile destination. This two-stage organization avoids requiring every narrow mesh to connect directly to every Tile position in neighboring Groups, thereby reducing global wiring complexity.

3.7.7 L1 Network Latency

The latency of an L1 access depends on the physical location of the destination bank and the corresponding interconnect resources traversed by the request and response. We characterize the zero-load round-trip latency from a scalar core to the destination L1 data SRAM, including the return path to the issuing core. The request path starts at the core and terminates at the destination data SRAM, while the response path carries the returned data back to the requester through the corresponding return-routing resources. One cycle of the request and response paths overlaps at the SRAM access, such that the round-trip latency is one cycle shorter than the sum of the two independently counted paths.

An intra-Tile access requires 4 cycles on the request path and 3 cycles on the response path, resulting in a 6-cycle round-trip latency. The request traverses the core-side pipeline cut, scalar

Table 4: Zero-load L1 access latency for representative destination classes.

Destination	**Request**	**Response**	**Round Trip**
Same Tile	4	3	6
Same Group	5	5	9
Neighbor Group, geometric Tile	9	9	17
Neighbor Group, non-geometric Tile	10	10	19

routing crossbar, local L1 crossbar, atomic unit, cache controller, and data SRAM, while the response returns through the corresponding local structures.

An access to another Tile within the same Group additionally traverses the Group-local crossbar network. This increases both the request and response paths to 5 cycles and results in a 9-cycle round-trip latency.

For an access to the geometrically corresponding Tile in a neighboring Group, the request enters the narrow 2D mesh directly from the source Tile and traverses the input and output pipeline stages of both the source and destination routers. The request and response paths each require 9 cycles, giving a total round-trip latency of 17 cycles.

If the destination is located in a different relative Tile position within the remote Group, an additional local NoC crossbar stage is required to select the appropriate narrow mesh and complete the final Tile routing. This adds one cycle to both the request and response paths, increasing the neighboring-Group round-trip latency from 17 to 19 cycles.

Each inter-Group router contains one pipeline cut at its input and one at its output. Consequently, every additional mesh hop beyond the neighboring Group adds 2 cycles to the request path and 2 cycles to the response path, corresponding to 4 additional round-trip cycles.

For an inter-Group access, the zero-load round-trip latency can therefore be expressed as

$$L_{\mathrm{RT}} = 17 + 4\,(N_{\mathrm{hop}} - 1) + 2I_{\mathrm{nongeo}},$$

where $N_{\mathrm{hop}} \geq 1$ is the number of inter-Group mesh hops and I_{nongeo} is 1 when an additional non-geometric Tile-routing stage is required and 0 otherwise. The first term represents access to the geometrically corresponding Tile in a neighboring Group, the second captures the additional router stages for longer-distance accesses, and the final term accounts for the extra crossbar traversal required by non-geometric destinations.

Figure 8 illustrates the resulting spatial latency distribution across the system. Banks within the issuing Tile provide the minimum latency, while accesses become progressively more expensive as they traverse the Group-local network and additional inter-Group mesh hops. The non-uniform latency is therefore primarily determined by physical network distance rather than by a uniform logical shared-memory abstraction.

A parallel study of possible low-latency topologies was carried out in a Master's thesis, summarized in Section B. The study's results show that a logically low-latency topology cannot guarantee low latency in the physical design when considering wire length and pin constraints. However, the increased bandwidth from other topologies can still improve overall performance, although average latency may not decrease.

3.8 Cache Refill Network and Main-Memory Connectivity

Cache misses from the distributed L1 data-cache banks are serviced through a dedicated wide refill network that is physically and logically separated from the L1 data access network. Whereas the L1 data access network carries fine-grained core-generated memory operations, the refill network is optimized for wide cache-line transfers between distributed cache controllers and external-memory endpoints.

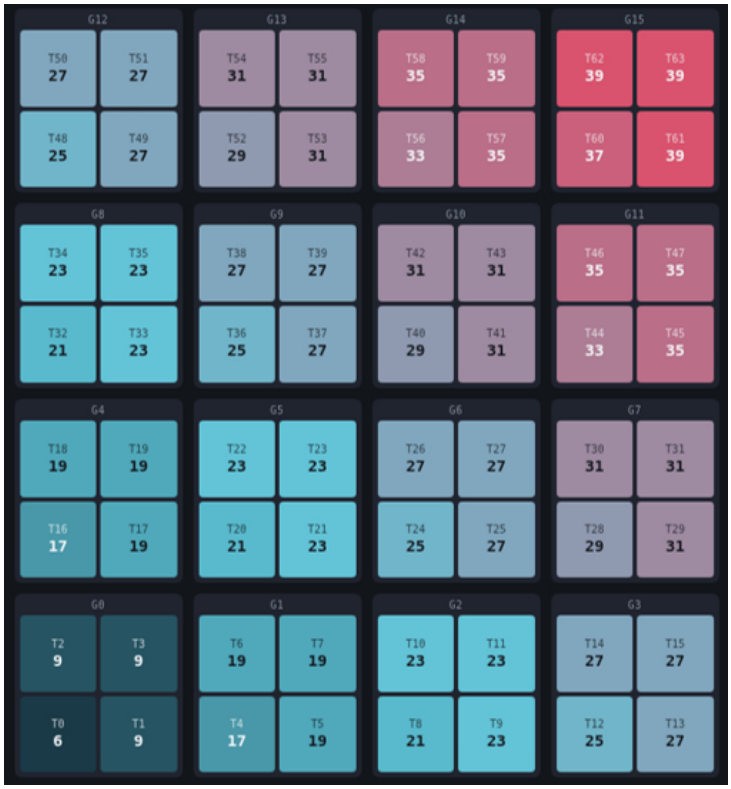


Figure 8: Zero-load round-trip L1 access latency from Core 0 to cache-bank destinations across the CachePool topology. The latency increases with the number of inter-Group mesh hops and includes an additional penalty when the target is not located at the geometrically corresponding Tile position.

The refill network uses a 512 bit datapath, matching the 64 B L1 cache-line width. This allows a complete cache line to be transported in one data beat within the refill fabric and avoids fragmenting refill traffic into multiple narrow transactions. The network is implemented as a two-dimensional mesh and can be configured with different numbers of external-memory endpoints to match the target system scale and memory configuration.

Separating the two networks allows their datapaths, topology, arbitration, and buffering to be tailored to different traffic characteristics. The narrow L1 data access network is optimized for fine-grained requests generated by scalar and vector cores, while the wide refill network is optimized for lower-frequency but bandwidth-intensive cache-line movement.

3.8.1 Wide Refill Mesh

Cache-controller refill traffic is physically separated from the narrow core-generated load/store network. All cache controllers connect to a dedicated 512 bit-wide 2D mesh NoC that is used for full cache-line transfers between the distributed L1 cache and the external-memory interfaces. The width of the refill network matches the 64 B cache-line size, allowing an entire cache line to be transferred in one network beat when sufficient downstream bandwidth is available.

Separating refill traffic from fine-grained core accesses prevents wide cache-line transfers from consuming the routing resources of the narrow load/store interconnect. It also allows the two fabrics to be optimized independently for their different traffic characteristics: the narrow network targets low-overhead word accesses, while the refill mesh is optimized for high-throughput cache-line movement.

3.8.2 HBM Connectivity

The wide refill mesh connects to a configurable number of HBM endpoints. The current CachePool configuration supports either 8 or 16 HBM channels, allowing the available off-chip bandwidth to be scaled according to the system configuration.

An example physical organization for the 16-channel configuration is shown in Figure 9. The HBM channels are distributed around the perimeter of the Group mesh, with neighboring channels placed on the same side of the system where possible. This arrangement is intended to match the physical packing constraints of typical multi-channel HBM controller IPs while keeping the average distance between Groups and external-memory endpoints short.

The placement also preserves a notion of affinity between Group identifiers and HBM channels. Channels are positioned such that Groups with matching or nearby identifiers can access a nearby HBM endpoint with a short path through the refill mesh. This organization aligns with the software convention used by bandwidth-oriented kernels, where off-chip traffic can be distributed across Groups and their preferred memory channels to reduce contention and improve aggregate bandwidth.

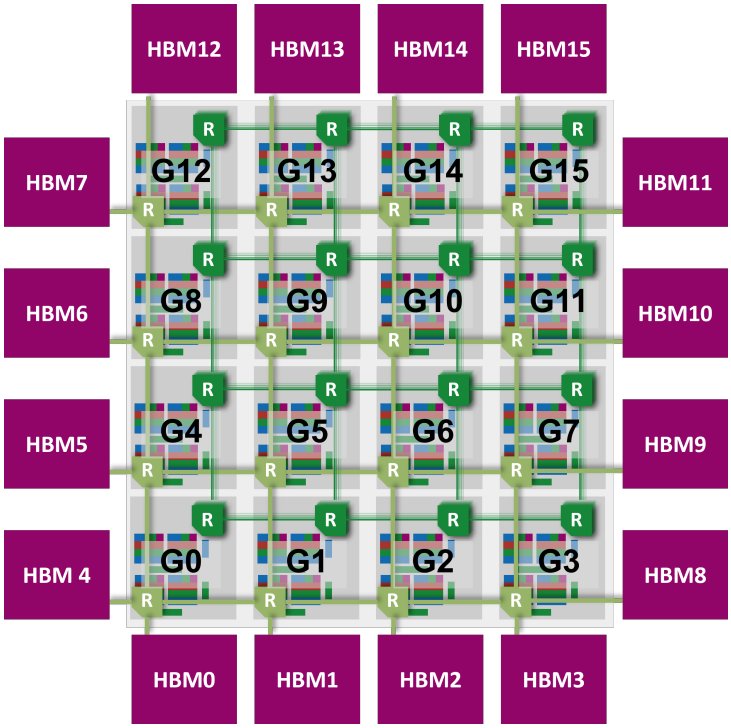


Figure 9: Example organization of the 16-channel HBM configuration and the wide refill mesh. The HBM endpoints are distributed around the perimeter of the 16-Group array to reduce communication distance and to provide a natural affinity between Groups and external-memory channels.

3.8.3 Refill Routing and Protocol Conversion

As described above, cache refill traffic is transported through a dedicated 512 bit-wide TCDM-style 2D mesh NoC. The refill network carries full cache-line requests and responses between the distributed cache controllers and the off-chip memory interfaces. Table 5 summarizes the fields used by the refill protocol.

Compared with the narrow core-access interface, several protocol fields are simplified because the refill datapath width matches the 512 bit cache-line width. For HBM-backed configurations, one cache line is transferred in a single network beat and no multi-beat burst reconstruction

is required inside the refill mesh. The burst metadata is therefore reduced to the information required by the endpoints, while unused burst-length bits are tied to constants and removed during physical implementation.

The write-strobe field is likewise simplified. Refill transactions operate on complete cache lines, so the 64 bit strobe vector is driven either entirely high or entirely low rather than carrying independent per-byte enables. The resulting constant structure can be collapsed during synthesis and physical implementation.

Atomic-operation information is not required on the refill network. The `amo` field is therefore tied to `AMONONE` and is not consumed by any refill endpoint.

The refill request metadata identifies the originating cache bank and Tile and carries the information required to route returning data back to the corresponding cache-controller transaction. The protocol also preserves a source identifier used by the next memory-hierarchy level and cache-internal miss-handling metadata required when the refill response returns.

Table 5: TCDM-style request/response interface used by the cache-refill network.

Request		
Field	**Bits**	**Description**
addr	32	Target byte address
write	1	Read/write selector
amo	Z	Atomic-operation field, tied to `AMONONE`
data	512	Cache-line write data
strb	64	Byte-enable vector, driven uniformly to all zeros or all ones
size	3	Transfer-size encoding preserved from the external-memory transaction
user	26/28	User-defined refill metadata
q_valid	1	Request-valid handshake
p_ready	1	Response-ready handshake
Response		
Field	**Bits**	**Description**
data	512	Cache-line refill data
error	1	Error indicator
write	1	Write-response indicator
user	26/28	User-defined refill metadata
p_valid	1	Response-valid handshake
q_ready	1	Request-ready handshake
User Metadata		
Field	**Bits**	**Description**
bank_id	3	Identifier of the originating cache bank within a Tile
tile_id	6	Identifier of the originating Tile
l2_src_id	5	Source identifier used by the next memory-hierarchy level
info	11	Cache miss-handling metadata associated with the refill transaction
burst	1/3	Burst metadata; reduced when the cache-line width matches the data width

When the refill network width matches the cache-line width, only a single burst-control bit is required to distinguish data-cache and instruction-cache traffic. The remaining burst-length bits are tied to constant values and are not consumed by the endpoints. In configurations requiring multi-beat refills, the full burst metadata can instead be retained.

3.9 Instruction Memory Hierarchy

ManyRVData employs a hierarchical instruction-cache organization that we refer to as a 2.5-level hierarchy:

per-core L0 iCache → Tile-shared L1 iCache → Group-shared L2 iCache → 512-bit refill mesh.

3.9.1 Per-Core L0 Instruction Cache

Each scalar core contains a small private L0 instruction cache of 128 B for the hottest instruction working set and lowest-latency instruction delivery.

3.9.2 Tile-Shared L1 Instruction Cache

The next level is shared by the cores within one Tile, reducing redundant instruction storage while keeping most instruction traffic local to the Tile. The L1 instruction cache has a default capacity of 8 KiB.

3.9.3 Group-Shared L2 Instruction Cache

A further instruction-cache level is shared across the Tiles of a Group. This level absorbs misses from the Tile-shared L1 instruction caches before they reach the refill network. The Group-shared L2 instruction cache has a default capacity of 16 KiB. Misses from this level enter the same 512 bit refill mesh used by the data-cache controllers.

3.10 System Control and Synchronization

CachePool provides lightweight hardware mechanisms for runtime configuration and synchronization across the manycore system. Memory-mapped control registers expose architectural features such as cache partitioning and address scrambling to software, while a hardware barrier provides scalable synchronization across cores and higher-level hierarchy domains.

3.10.1 Memory-Mapped Control and Status Registers

CachePool exposes a set of memory-mapped control and status registers for runtime configuration of system-level architectural features. The register interface covers synchronization, Spatz sharing, instruction-cache control, boot and execution status, L1 cache maintenance, cache partitioning, and address-mapping configuration. Most architectural mechanisms described in the previous subsections are therefore programmable without requiring a hardware rebuild.

The number of Tile-selection registers and hardware-barrier participation masks is parameterized through `NumTileSelRegs` and `NumBarrierSlots`, respectively. The default Tile-selection configuration provides two 32 bit masks, which together cover the 64 Tiles of the 16-Group system. The number of barrier slots follows the selected system configuration, as described in Section 3.10.2.

Several of the configuration registers use an explicit programming and commit sequence. This allows software to prepare a new configuration before atomically applying it to the corresponding hardware mechanism. Examples include the L1 SPM configuration, L1 maintenance operations, and crossbar address-mapping offset.

The software abstractions and runtime APIs used to program these registers are described in Section 5.

3.10.2 Hardware Barrier

CachePool provides a dedicated hierarchical hardware barrier to support low-overhead synchronization across the large number of scalar cores in the system. The barrier follows the physical organization of the architecture and is implemented as a three-level tree spanning Tile, Group, and system-wide domains. This hierarchical organization allows synchronization state to be aggregated locally before propagating to the next level, avoiding a flat global structure with large fan-in and long timing paths.

At the Tile level, participating Snitch cores report their arrival to a local barrier instance. The Tile-level state is then aggregated by the corresponding Group-level barrier, which in turn communicates with the top-level barrier for synchronization spanning the complete system. Pipeline cuts are inserted along the hierarchy to meet timing across the longer communication paths between hierarchy levels.

The barrier supports both *full* and *partial* synchronization. For a full barrier, all configured participants propagate through the complete Tile–Group–system hierarchy and are released only after the global synchronization condition has been satisfied. For a partial barrier, software can configure a participation mask such that synchronization terminates at a lower hierarchy level when all required local participants have arrived. A local partial barrier therefore does not need to propagate to higher levels of the hierarchy, reducing synchronization latency and avoiding unnecessary global traffic.

Multiple independent barrier instances are supported concurrently through a set of barrier slots. Each barrier operation is identified by a barrier ID, allowing different groups of threads or execution phases to maintain independent synchronization state. The default configuration provides 2 barrier slots for 1S1V systems and 4 slots for dual-scalar 2S1V systems, while the number of slots remains configurable at design time.

Barrier participation and partial-barrier masks are controlled through memory-mapped peripheral registers. The barrier state maintains the masks and identifiers required to track participating cores together with valid and completion information at the corresponding hierarchy levels. This provides software with a configurable synchronization primitive while keeping the latency-critical arrival and release operations in hardware.

The software interface and runtime support for configuring and invoking the hardware barrier are described in Section 5.

3.11 System Configuration and Scalability

CachePool is intentionally parameterized rather than fixed to a single microarchitectural configuration. The main architectural parameters include the number and arrangement of Groups, the number of Tiles and CCs per Group, the CC organization, optional 32 bit floating-point support, L1 cache geometry and controller options, local and remote interconnect resources, refill-network organization, and the number of external-memory channels. Several of these parameters are selected explicitly in the build configuration, while others are derived automatically from higher-level architectural choices.

The current RTL repository provides a set of predefined configurations covering different combinations of system scale, CC organization, floating-point support, and external-memory connectivity. For compactness, the common `cachepool_` prefix is omitted from the configuration names in Table 6.

3.12 DRAM/HBM Co-Simulation

A time-efficient yet cycle-accurate external-memory subsystem is required to evaluate data-transfer performance together with the RTL memory hierarchy. The simulation environment integrates *DRAMSys5.0* [50] through an RTL interface, enabling cycle-accurate modeling of

multiple external-memory technologies. The current CachePool environment supports both DDR4-2400 and HBM2E-3600 configurations, which can be selected through the system configuration files to evaluate the corresponding memory-bandwidth and latency characteristics.

Table 6: Predefined CachePool build configurations provided by the current ManyRVData RTL repository. The common `cachepool_` prefix is omitted from the configuration names.

Parameter	`4g`	`fpu_4g`	`dual_4g`	`dual_fpu_4g`	`fpu_16g`	`fpu_16g_tiny`	`dual_fpu_16g`
Groups	4	4	4	4	16	16	16
Topology	2×2	2×2	2×2	2×2	4×4	4×4	4×4
CC organization	1S1V	1S1V	2S1V	2S1V	1S1V	1S1V	2S1V
Spatz FP	No	Yes	No	Yes	Yes	Yes	Yes
HBM channels	4	4	4	4	16	8	16
Variant	Small-size	FP	Dual scalar	Dual scalar + FP	Baseline	Tiny	Dual scalar

Table 7: Summary of the memory-mapped control and status registers exposed by CachePool.

Category	**Register**	**Function**
Barrier and synchronization	`HW_BARRIER`	Load-based hardware barrier synchronization
	`HW_BARRIER_PARTICIPATION_MASK[n]`	Per-slot Group participation mask for system-wide barriers
Spatz ownership control	`SPATZ_LOCK_ACQUIRE`	Acquire Spatz ownership in the 2S1V configuration
	`SPATZ_LOCK_RELEASE`	Release Spatz ownership in the 2S1V configuration
Instruction-cache control	`ICACHE_PREFETCH_ENABLE`	Enable or disable instruction-cache prefetching
Execution status and profiling	`SPATZ_STATUS`	Indicate whether the system is executing a kernel
	`SPATZ_CYCLE`	Store software-visible kernel cycle counts
Boot and termination	`CLUSTER_BOOT_CONTROL`	Configure the post-boot program entry point
	`CLUSTER_EOC_EXIT`	Report end-of-computation and program exit status
L1 SPM/cache configuration	`CFG_L1D_SPM`	Configure the size of the L1 SPM region
	`L1D_SPM_COMMIT`	Commit the programmed SPM configuration
L1 maintenance	`CFG_L1D_INSN`	Select L1 flush or invalidation operations
	`CFG_L1D_TILE_SEL[n]`	Select Tiles targeted by private-partition maintenance operations
	`L1D_INSN_COMMIT`	Commit the selected L1 maintenance operation
	`L1D_FLUSH_STATUS`	Report whether an L1 flush or invalidation is in progress
L1 partitioning	`L1D_PRIVATE`	Configure the number of private banks and Tile/Group-private scope
	`L1D_ADDR`	Configure the address boundary of the private L1 region
L1 address mapping	`XBAR_OFFSET`	Configure the L1 crossbar address-mapping offset
	`XBAR_OFFSET_COMMIT`	Commit the programmed crossbar offset

4 Physical Implementation

This section evaluates the physical feasibility and area composition of the CachePool architecture after place and route. We implement both the baseline 1S1V Core Complex and the 2S1V configuration to quantify the physical-design overhead introduced by sharing one Spatz vector unit between two Snitch scalar cores. The analysis covers the Group, Tile, and L1 cache-controller levels, followed by the assembly of the complete 16-Group CachePool system.

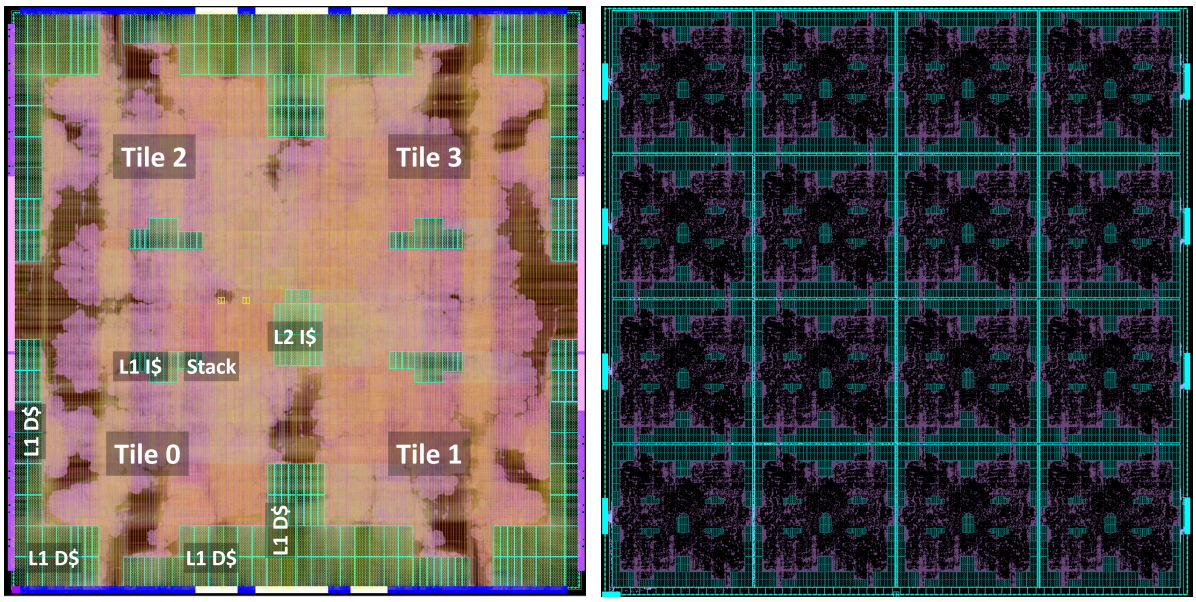
4.1 Methodology

We implement CachePool in TSMC's 7 nm technology using a bottom-up physical-design flow. Each Group is first synthesized, placed, and routed, and the resulting Group implementation is then used as a hardened building block when assembling the complete 16-Group cluster. All synthesis, placement, and routing are performed using Synopsys Fusion Compiler 2025.06.

The implementation targets 1 GHz at the worst-case process corner at 0.675 V, considering the specified temperature range from −40 °C to 125 °C. The `cachepool_fpu_16g` configuration is used as the baseline physical-design configuration, with a target placement utilization of approximately 50–60%.

4.2 Floorplan

The placed-and-routed baseline Group occupies a floorplan of 1670×1690 μm^2, with approximately 54% area utilization after routing. Sixteen Groups are subsequently assembled into the full CachePool cluster, resulting in a floorplan of 6900×6920 μm^2. A 30 µm routing channel is reserved between adjacent Groups, while slightly wider channels are left at the cluster boundaries to accommodate cluster-level interconnect and control logic.



Placed-and-routed CachePool Group. Assembled 16-Group CachePool cluster.

Figure 10: Physical implementation of the baseline CachePool configuration at the Group and full-cluster levels.

4.3 Area Breakdown

The Gate Equivalent (GE), defined as the area of a two-input NAND gate, is used to normalize the physical area of the different logic components. Figures 11 and 12 show the post-place-and-route area breakdown of the 1S1V and 2S1V configurations at the Group, Tile, and cache-controller levels.

For the baseline 1S1V configuration, one Group occupies 32.27 MGE. The four Tiles account for 28.54 MGE, or approximately 88.4% of the Group area. The L1 routing infrastructure, including the L1 router, local-Group and remote-Group crossbars, and refill router, occupies approximately 1.31 MGE, corresponding to about 4.1% of the Group. The remaining Group-level logic accounts for approximately 7.5%.

At the Tile level, the 1S1V configuration occupies approximately 7.14 MGE per Tile. The distributed L1 data-cache subsystem remains the dominant contributor, accounting for approximately 64.5% of the Tile area. The four Core Complexes together contribute approximately 29.3%, while the Tile-local L1 crossbar accounts for less than 2%.

Within the L1 subsystem, the data SRAM, metadata SRAM, and cache controller contribute approximately 540 kGE, 254 kGE, and 357 kGE per bank, respectively. The relatively large metadata SRAM is partly caused by the current implementation assumption that the SRAM macros do not support byte-enable writes. Byte-dirty information must therefore be explicitly stored in the metadata. Technologies providing suitable byte-write support could reduce this metadata overhead.

A further breakdown of the cache controller shows that the controller datapath and control logic dominate its logic area, while FIFOs account for approximately 81 kGE per controller. These FIFOs buffer requests and intermediate states across the different cache-controller operations described in Figure 5.

The 2S1V configuration increases the Group area from 32.27 MGE to 33.40 MGE, corresponding to an overhead of only approximately 3.5%. At the Tile level, the area increases from approximately 7.14 MGE to 7.37 MGE, or about 3.3%. The additional scalar core mainly affects the Core Complex and local interconnect: the Core Complex area increases from approximately 523 kGE to 571 kGE, while the Spatz vector unit itself remains almost unchanged.

The modest area increase is a direct consequence of sharing the much larger Spatz vector unit between two Snitch scalar cores. In the 1S1V configuration, one Spatz occupies approximately 482 kGE, whereas one Snitch occupies only about 18 kGE. Adding a second Snitch therefore increases scalar-control capability and enables vector-unit sharing without duplicating the dominant vector datapath.

At the Group level, the fraction occupied by the four Tiles remains nearly unchanged between the two configurations, at approximately 88%. Similarly, the Group-level routing infrastructure remains close to 4% of the total area. This indicates that moving from 1S1V to 2S1V does not introduce a disproportionate interconnect overhead and that the architectural scalability is still primarily determined by the compute and distributed L1 structures.

Overall, the post-place-and-route results demonstrate that the configurable Core Complex organization can increase scalar-core density with limited physical-area overhead. The 2S1V configuration increases the complete Group area by approximately 3.5%, while preserving the same number of Spatz vector units, L1 banks, and Group-level network resources. This makes 2S1V an area-efficient configuration when additional scalar control throughput or vector-unit sharing is desired.

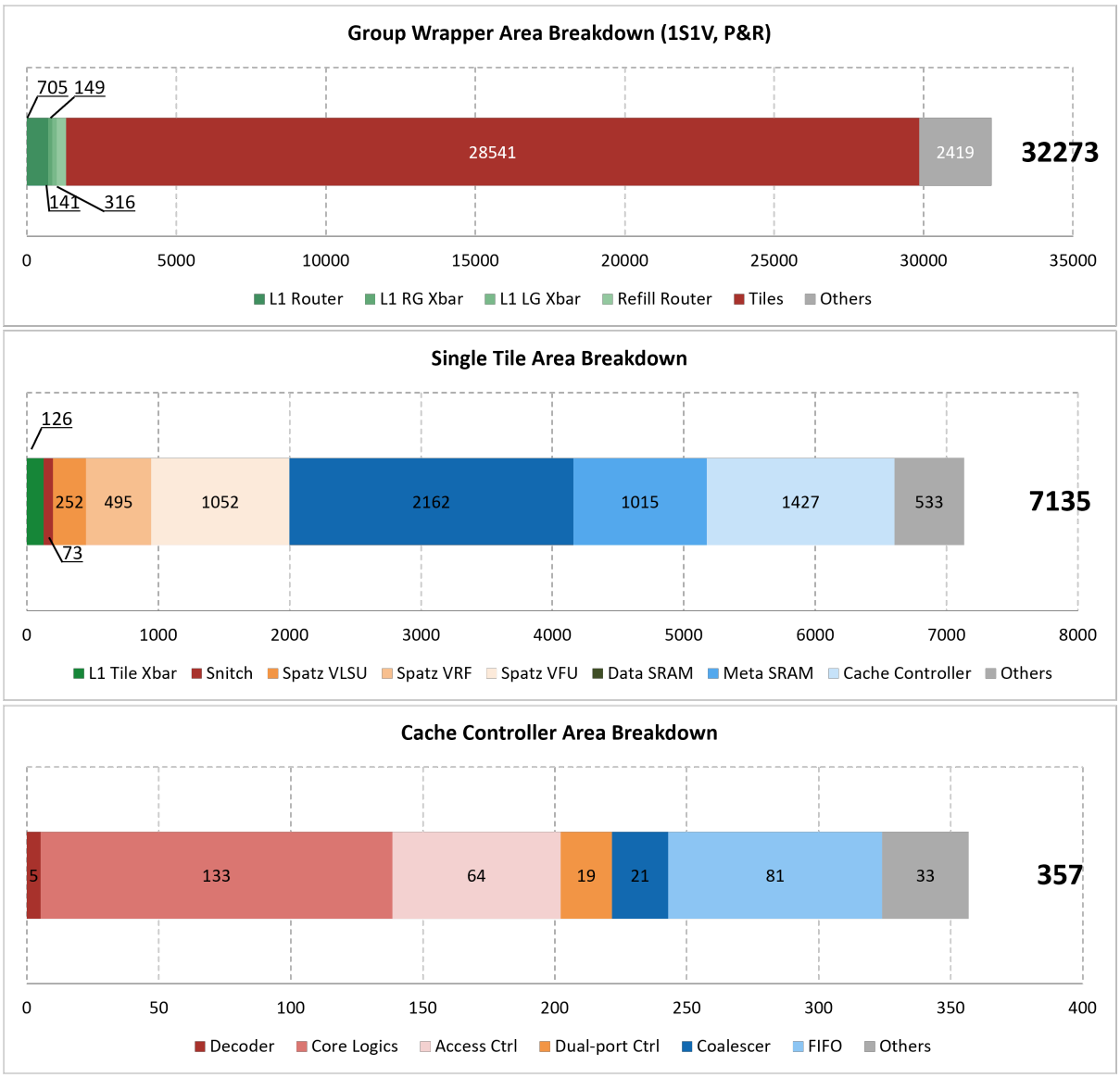


Figure 11: Post-place-and-route area breakdown of the 1S1V CachePool configuration at the Group, Tile, and cache-controller levels. Area is reported in kGE.

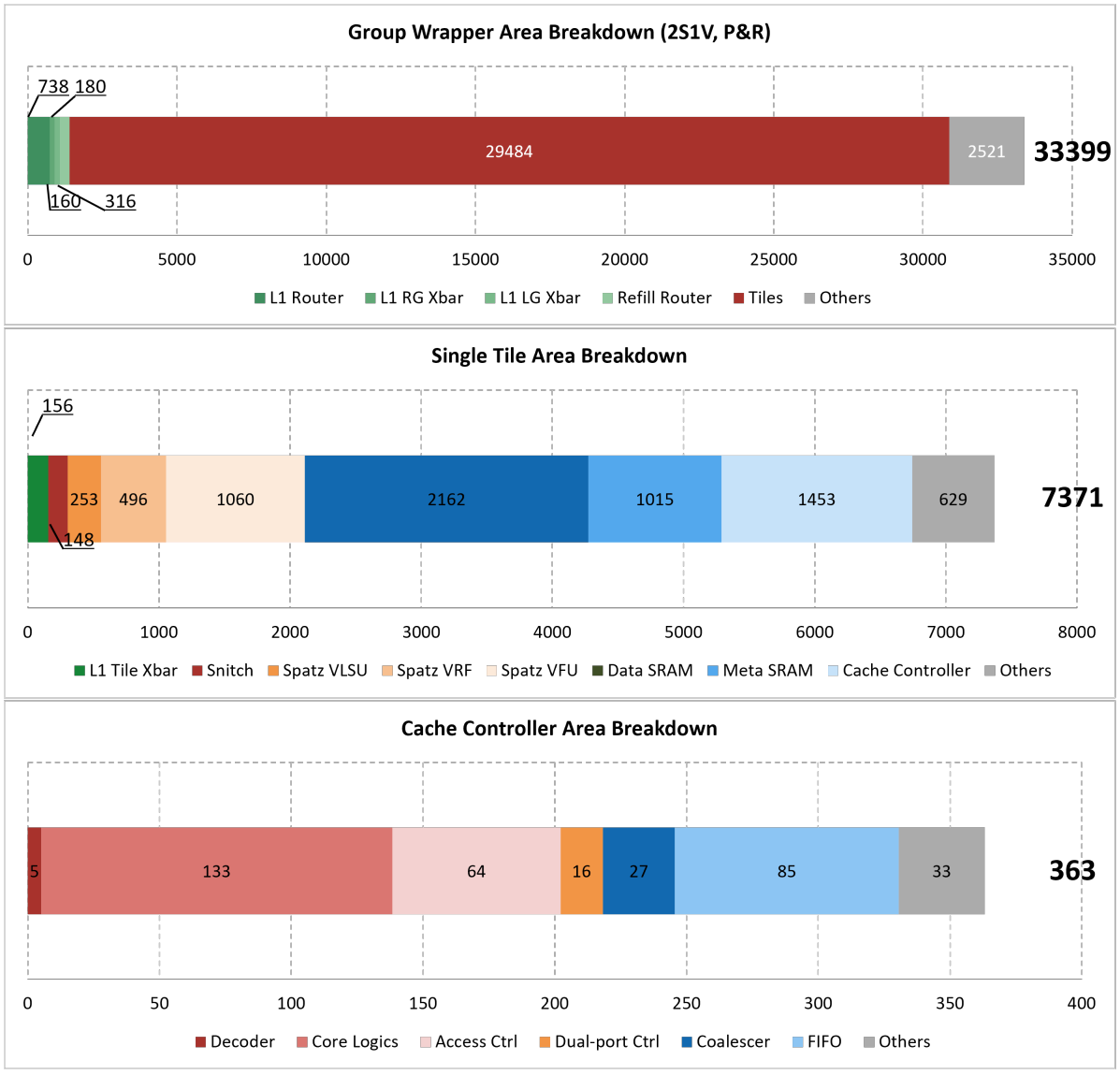


Figure 12: Post-place-and-route area breakdown of the 2S1V CachePool configuration at the Group, Tile, and cache-controller levels. Area is reported in kGE.

5 Software and Runtime Support

The CachePool runtime provides the software interface required to expose the architectural mechanisms described in Section 3. It abstracts system topology, memory placement, cache configuration, synchronization, and execution control through a set of lightweight runtime functions and memory-mapped configuration interfaces. The runtime is designed to preserve the hierarchical organization of CachePool, allowing software to reason explicitly about cores, Tiles, Groups, and memory-sharing domains when required.

5.1 Programming Model

CachePool follows a fork–join programming model in which all Snitch cores execute a common program and synchronize at explicit barriers between parallel phases. A designated core performs initialization and other sequential tasks when required, after which work is distributed across the participating cores for parallel execution. The runtime provides topology and synchronization primitives that allow applications to organize work at the core, Tile, Group, and system levels.

The scalar Snitch processors are programmed primarily in C and compiled using the LLVM RISC-V toolchain. Scalar code handles control flow, work distribution, synchronization, memory management, and orchestration of vector computation.

Spatz implements the RISC-V Vector Extension (RVV) 1.0 programming model but currently supports only a subset of the complete RVV instruction set. In particular, Spatz does not implement several vector masking, fixed-point, and permutation operations. Consequently, unrestricted compiler auto-vectorization cannot be assumed to generate instruction sequences that are directly executable by Spatz.

The current ManyRVData software flow therefore disables or avoids unrestricted LLVM auto-vectorization for performance-critical vector kernels. Instead, these kernels use explicitly controlled vector code, including hand-written assembly, so the compiler emits only instructions supported by the target Spatz configuration. A future compiler flow could enable more automatic vectorization by exposing the supported Spatz instruction subset to the compiler or by otherwise constraining RVV code generation accordingly.

5.2 Topology Discovery and Core Identification

The runtime exposes topology-query functions that allow software to identify the position of the executing hart within the CachePool hierarchy. These functions abstract the compile-time system configuration and allow the same application code to run across configurations with different numbers of Groups, Tiles, scalar cores, and vector units.

The runtime provides identifiers and counts at several hierarchy levels, including the global core index, cluster-local core index, Tile index, Group index, and the corresponding number of resources. In addition, helper functions report the number and index of physical Spatz vector units, which differ from the hart count in the 2S1V configuration because two Snitch harts share one Spatz instance. The function `snrt_cluster_is_primary()` identifies the default owner of a shared Spatz pair and is used by software targeting dual-scalar Core Complexes.

Representative topology functions include

- `snrt_global_core_idx()` and `snrt_global_core_num()` for system-wide core identification;
- `snrt_cluster_core_idx()` and `snrt_cluster_core_num()` for logical-cluster-local identification;
- `snrt_cluster_tile_idx()` and `snrt_cluster_tile_num()` for Tile-level topology information;

- `snrt_cluster_group_idx()` and `snrt_cluster_group_num()` for Group-level topology information;
- `snrt_cluster_vpu_idx()` and `snrt_cluster_vpu_num()` for physical Spatz identification.

These helpers are used throughout the runtime and application kernels to derive work partitions, select local memory regions, construct barrier masks, and map software tasks onto the physical hierarchy.

5.3 Memory Allocation and Placement

Dynamic memory allocation in CachePool is performed in off-chip memory, while allocated data are transparently cached in the shared L1 according to the configured address-mapping and partitioning policies.

The runtime provides a lightweight DRAM allocator through `snrt_malloc()` and `snrt_free()`. The allocator uses a linked-list organization in DRAM and supports block coalescing on deallocation. Both allocation metadata and payloads are aligned to the 64 B cache-line size, and requested payload sizes are rounded up to a cache-line multiple.

The allocator is intentionally not thread-safe and is expected to be used during single-core initialization phases. A typical application therefore allocates and initializes shared data structures on one designated core before synchronizing the participating cores and entering the parallel computation phase.

Once allocated, data are accessed through the normal CachePool memory hierarchy. Their effective placement within the distributed L1 is determined by the programmable address-interleaving and cache-partitioning mechanisms described in the following subsections.

In addition to dynamically allocated data, each Snitch core uses a dedicated 1024 B local stack SRAM for low-latency stack accesses. When the local stack region is insufficient, excess stack storage is mapped into a core-specific region of the shared L1 address space, as described in Section 3.2.4.

5.3.1 L1 Address Scrambling

The runtime exposes software control over the L1 address-scrambling mechanism through `l1d_xbar_config()`. The argument specifies the crossbar interleaving offset in address bits and therefore controls the granularity at which consecutive memory regions are distributed across L1 destinations.

The runtime enforces a minimum granularity equal to the cache-line size so that all bytes belonging to the same cache line remain mapped to one cache controller. For the default 64 B cache line, an offset of 6 corresponds to cache-line-granular interleaving.

Changing the scrambling configuration allows software to trade locality against aggregate bandwidth. Streaming kernels can use finer-grained interleaving to distribute traffic across more banks and Tiles, while kernels with strong spatial reuse can use coarser mappings to retain more accesses within a local region.

5.3.2 Cache Partitioning and Address Classification

The runtime exposes the programmable private/shared L1 partition through `l1d_part()` and `l1d_part_folded()`. The partition size specifies the number of private banks per Tile, ranging from zero for a fully shared configuration to four for a fully private configuration.

The default `l1d_part()` interface selects Tile-private behavior for banks assigned to the private region. The extended `l1d_part_folded()` interface additionally allows software to select Group-private operation through the `L1D_FOLD_GROUP` mode. In this mode, the private partition

is shared among the Tiles of one Group rather than replicated independently per Tile, while the shared partition remains globally visible across the system.

Group folding is particularly useful for data that is reused by all Tiles within one Group. Instead of fetching one copy per Tile, the Group can share a single cached copy through the intra-Group network. The trade-off is that accesses to nominally private data may target a sibling Tile within the same Group and therefore incur an intra-Group access rather than a purely local Tile access.

The address boundary between the shared and private regions is configured through `l1d_addr()`. In the current runtime, addresses below the configured boundary are mapped to the globally shared region, while addresses at or above the boundary are mapped to the private region. Changing either the partition organization or the address boundary requires invalidating or flushing resident cache state before the new mapping becomes active.

5.3.3 Cache Maintenance

The runtime provides cache-maintenance functions for full-cache, shared-region, and private-region operations. Application code normally uses the cluster-wide variants, which coordinate all participating cores before issuing the maintenance request.

The functions `l1d_cluster_flush()`, `l1d_cluster_shared_flush()`, and `l1d_cluster_private_flush()` respectively flush the complete L1, only the globally shared banks, or the private banks associated with selected Tiles. The cluster-wide functions internally execute a memory fence and hardware barriers, perform the actual maintenance operation on one designated core, and synchronize again before returning.

Low-level single-core variants are also provided for runtime initialization and internal use. The function `l1d_wait()` polls the hardware status register until the selected maintenance operation has completed.

Cache reconfiguration operations that alter the partitioning mode or the private/shared address boundary implicitly require a cold transition. The runtime therefore flushes the cache before applying such changes so that valid data cannot remain associated with an obsolete address mapping.

5.4 Synchronization and Work Distribution

CachePool provides several complementary synchronization mechanisms ranging from local memory ordering to system-wide collective barriers. The runtime exposes memory fences, atomic operations, software barriers, and the hierarchical hardware barrier described in Section 3.10.2.

5.4.1 Memory Ordering and Fences

The function `snrt_fence()` drains both outstanding Snitch scalar-memory operations and outstanding Spatz memory requests before subsequent instructions are allowed to proceed. This operation is required before synchronization points when earlier scalar or vector stores must be globally visible.

The runtime additionally provides `snrt_fence_snitch()` and `snrt_fence_spatz()` when only the scalar or vector memory path needs to be drained. These more selective fences can reduce unnecessary waiting in kernels that use only one of the two memory-issuing engines.

5.4.2 Hardware and Software Barriers

The default `snrt_cluster_hw_barrier()` function invokes the full hardware barrier on barrier slot 0 and synchronizes all participating cores in the logical cluster. The runtime also retains a polling-based software barrier and a higher-level global barrier for cases where a hardware collective is not appropriate.

More selective synchronization is supported through partial and Group-level barrier APIs. The function `snrt_cluster_partial_barrier()` restricts the participating cores within each Tile, while `snrt_cluster_group_barrier()` additionally allows software to select participating Tiles and optionally terminate synchronization at the Group level. A barrier ID selects one of the independent hardware barrier slots, allowing multiple synchronization contexts to remain active concurrently.

The function `snrt_barrier_set_group_mask()` configures the Group participation mask associated with a barrier slot for synchronization rounds that reach the top-level barrier. All participants in a given round must use identical masks and barrier identifiers to preserve a consistent synchronization domain.

5.4.3 Atomic Operations

The runtime exposes RISC-V atomic operations for fine-grained synchronization through the scalar Snitch memory path. For example, `snrt_mutex_lock()` and `snrt_mutex_release()` implement a blocking mutex using `amoswap.w` operations with acquire and release semantics.

Atomics complement the collective hardware barrier by providing synchronization for shared data structures and irregular producer–consumer interactions where a full collective synchronization point would be unnecessarily expensive.

5.4.4 Work Distribution

Parallel kernels derive work assignments from the topology-query functions described in Section 5.2. The global or local core identifier is commonly used to divide loop iterations or data partitions across cores, while Tile and Group identifiers can be used when the mapping should follow the physical memory hierarchy.

This explicit topology-aware work distribution allows software to align computational ownership with cache placement and interconnect locality. For example, kernels can assign data partitions to cores within one Group when Group-private cache partitioning is enabled, or distribute off-chip traffic across Groups and HBM channels according to the memory-placement policy.

5.5 Spatz Control

In the 2S1V configuration, two Snitch harts share one Spatz vector coprocessor and software must therefore coordinate vector-unit ownership. The runtime exposes blocking and non-blocking ownership primitives that operate on the hardware sharing mechanism described in Section 3.2.

The function `spatz_lock_acquire()` repeatedly attempts to acquire ownership until the request is accepted, while `spatz_lock_release()` relinquishes ownership after the current hart has completed its vector work. The underlying hardware operation itself is non-blocking and returns one of three outcomes: failure due to contention, immediate success, or success with a pending ownership transition.

When the hardware reports a pending transition, the runtime returns without waiting for the previous owner's Spatz instructions to fully drain. The accelerator multiplexer prevents the new owner from issuing vector operations until the transition completes, so the first subsequent vector or floating-point instruction naturally stalls until ownership is safe.

For applications requiring finer control, `spatz_lock_try_acquire()` and `spatz_lock_try_release()` expose the non-blocking ownership attempts directly. On 1S1V configurations, these APIs reduce to harmless operations because each Snitch already has exclusive access to its Spatz instance.

The runtime also provides host-specific barrier helpers for dual-scalar configurations. The functions `snrt_cluster_host0_barrier()` and `snrt_cluster_host1_barrier()` synchronize

only the corresponding members of each shared-Spatz pair and are implemented using the programmable hardware-barrier infrastructure.

5.6 Boot, Execution, and Profiling Support

The runtime provides the low-level startup and control functionality required to bring all Snitch harts into a consistent execution state before entering the application. At reset, one boot hart performs platform and runtime initialization while the remaining harts wait until they are released into the common application entry point.

The runtime initializes the topology descriptors, interrupt infrastructure, allocator state, and CachePool-specific memory subsystem before parallel execution begins. The L1 cache is invalidated during startup so that applications enter `main()` with a clean cache state and do not need to perform explicit cache initialization.

Memory-mapped peripheral registers additionally provide software-visible execution status, kernel cycle bookkeeping, boot-entry control, and end-of-computation signaling. These mechanisms are primarily used by the runtime, test infrastructure, and performance-evaluation environment rather than by individual kernels.

Together, the runtime abstractions provide a lightweight software layer over the configurable CachePool hardware while still exposing topology and memory-placement information when applications require architecture-aware optimization.

5.7 Hardware–Software Co-Design Guidelines

The CachePool architecture exposes several software-visible controls that allow the memory hierarchy, synchronization scope, and scalar–vector resources to be adapted to workload behavior. The runtime provides the corresponding mechanisms, while effective performance depends on selecting these mechanisms according to the access pattern, sharing structure, and compute characteristics of each kernel. This subsection summarizes the main hardware–software co-design principles used throughout the project.

Irregular accesses and hardware-managed caching. Sparse, pointer-based, and data-dependent accesses are difficult to stage efficiently through explicitly managed scratchpad memories because the accessed addresses are not known sufficiently far in advance for regular DMA-based double buffering. For these workloads, the hardware-managed shared L1 cache provides transparent demand fetching and allows independent cores and vector units to generate irregular accesses without explicit software-controlled data movement. Software can therefore operate directly on shared data structures while relying on the cache and the non-blocking memory interfaces to tolerate variable access latency.

Address scrambling for bandwidth and locality. The distributed L1 exposes a programmable address-scrambling granularity that controls how consecutive address regions are interleaved across cache banks, Tiles, and Groups. Fine-grained scrambling spreads memory traffic across a larger number of banks and interconnect paths and is therefore beneficial for bandwidth-oriented kernels with many concurrent independent accesses. Conversely, kernels with strong spatial or producer–consumer locality can use a coarser mapping to keep larger contiguous regions closer to the cores that consume them. The preferred configuration is therefore workload dependent rather than fixed globally for the complete application.

Private and shared cache partitioning. The four L1 banks associated with each Tile can be divided between globally shared and private regions. Shared banks are appropriate for data structures accessed broadly across the system, while Tile- or Group-private banks can improve locality and reduce interference when a working set is primarily consumed by a

restricted subset of cores. Software selects the private/shared bank ratio and private scope through memory-mapped configuration registers and places data in the corresponding address regions. This makes cache partitioning a software-controlled locality and isolation mechanism without requiring a different cache-controller implementation.

Synchronization scope. CachePool provides hierarchical hardware barriers at the Tile, Group, and system levels together with configurable participation masks. Software should use the smallest synchronization domain that contains all participating cores. Restricting synchronization to a Tile or Group avoids propagating barriers through the complete system and allows unrelated parts of the machine to continue independently. Multiple barrier slots additionally allow independent synchronization phases to coexist without requiring one global barrier state.

Scalar–vector resource sharing. The 2S1V Core Complex allows two Snitch cores to share one Spatz vector coprocessor, reducing vector-resource replication when vector execution is not continuously required by every scalar thread. Software can use the default free-sharing mode when vector requests from the two scalar cores are naturally intermittent and can be interleaved. For execution phases requiring stable vector context or explicit ownership, the runtime can place Spatz in lock mode and assign the coprocessor to one scalar core until the phase completes. The appropriate sharing policy therefore depends on the temporal structure of scalar and vector execution in the application.

Exploiting vector memory-level parallelism. Spatz provides four independent VLSU ports, each capable of maintaining multiple memory transactions in flight. Vector kernels should expose sufficient independent memory accesses to make use of these ports and the corresponding parallel paths through the L1 access network. Data layout and loop partitioning should therefore avoid concentrating all accesses onto the same bank or physical region when aggregate bandwidth is the limiting factor. In bandwidth-oriented kernels, combining vector-level parallelism with appropriate address scrambling helps distribute accesses across both banks and network planes.

Locality-aware work distribution. Although the L1 is logically shared, its physical distribution produces non-uniform access latency. Runtime work distribution should therefore consider the placement of frequently accessed data and, where possible, assign work to cores close to the corresponding cache banks or private regions. This is particularly useful for kernels whose working sets are reused repeatedly by the same Tile or Group. For workloads dominated by globally distributed accesses, spreading the data more broadly can instead increase usable aggregate bandwidth.

These mechanisms form the software-visible interface to the main architectural trade-offs of CachePool. The architecture does not impose one fixed memory placement or resource-allocation policy; instead, the runtime exposes controls that allow software to trade locality, sharing, bandwidth, synchronization scope, and vector-resource utilization according to workload requirements. The kernel implementations discussed later in this report apply these principles to representative dataplane and compute workloads.

6 GVSoC Simulation Framework

The CachePool GVSoC platform combines scalar/vector execution, distributed caches, and interconnect models to execute complete manycore workloads. Time-aligned hardware counters and software events connect packet-level behavior to core stalls, cache activity, and network contention. The evaluated 256-core target, `cachepool_v3`, supports end-to-end software analysis and architectural exploration through a configurable Python hierarchy and C++ component models.

Compatible CachePool ELFs execute in GVSoC and RTL with matching software interfaces, connecting software development to hardware validation. The asynchronous, cycle-approximate cache path models queued requests and contention, while synchronized instrumentation exposes their effect on application execution. RTL comparisons match topology, memory configuration, synchronization semantics, and measurement boundaries.

6.1 CachePool System Model

The measured large configuration contains 4×4 Groups, four Tiles per Group, and four CCs per Tile: 256 single-scalar CCs in total. Each CC executes Snitch scalar code and Spatz vector operations with four VLSU lanes. Four cache controllers per Tile form its distributed L1 slice. The data path contains per-port-class Tile crossbars, remote crossbars, narrow inter-Group networks, per-bank refill outputs, a Group refill multiplexer, and a separate wide refill mesh. Figure 13 and table 8 show the mapping. Remote shared accesses terminate at the destination Tile's shared banks. The figure's refill-mux row represents one mux per Group, each with 16 data-bank inputs and one priority input for Group L2 I$ miss refills.

Table 8: Architectural responsibilities of GVSoC components and their RTL counterparts.

GVSoC component	RTL counterpart	Modeled responsibility
Snitch ISS / Ara–Spatz model	`cachepool_cc`, Snitch, Spatz	Scalar/vector execution, dependencies, VLSU requests
`InsituCacheXbar`, remote xbar, AMO shim	Tile/remote TCDM interconnect, `spatz_cache_amo`	Routing, private/shared selection, atomics
`InsituCacheCore`	InSitu controller and TCDM wrapper	Tags/data, pending-line queues, misses, eviction, flush
Group I$ and refill mux	Group instruction cache and refill arbitration	Instruction priority, round-robin bank service
`FlooNoc2dMeshNarrowWide`	Group NoC wrappers and FlooNoC routers	Queuing, routing, injection, link contention
`ClusterRegisters`	Peripheral and barrier hierarchy	Boot/EOC, cache configuration, modeled barrier masks
`CachepoolV3SpatzLock`	Spatz lock and accelerator mux	Ownership and issue gating in multi-scalar CCs

The asynchronous cache path advances a per-cycle state machine with pending-line tracking, queued requests, and callback-based responses. Its refill resources are specified in Section 6.6. A separately calibrated synchronous reference path accumulates delays within a request call and bypasses some queueing structures.

6.2 Configurable Architecture Exploration

Python system elaboration configures the topology before simulation. Table 9 lists principal controls and the large-scale evaluation settings. MMIO writes separately control L1 partitioning, address-scrambling offset, and maintenance.

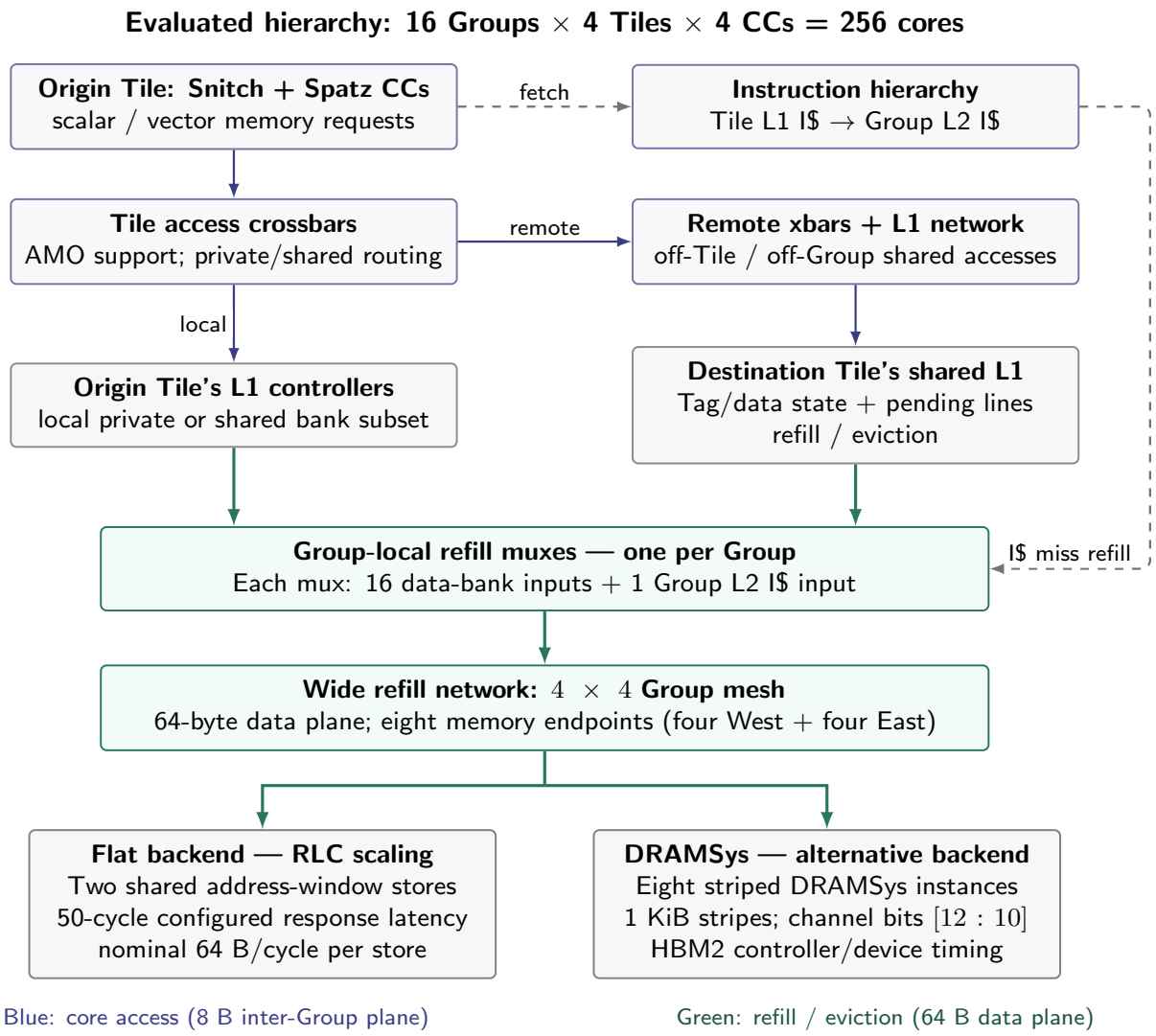


Figure 13: GVSoC request paths and backend alternatives. Local and remote shared accesses reach their respective Tile's L1 controllers. Repeated Group structures are collapsed: each controller refills through its own Group's mux, and the Group L2 instruction cache supplies that Group's instruction input. Responses use asynchronous return paths. The evaluated mesh has eight endpoints; flat mode shares two backing stores, while DRAMSys mode uses eight striped instances. These are alternative backend configurations.

Table 9: Principal model controls. Environment names have the prefix `CACHEPOOL_V3_`. Values specify the evaluation configuration.

Control suffix	RTL correspondence	Evaluation setting
`NB_X_GROUPS`, `NB_Y_GROUPS`	`NumGroupsX`, `NumGroupsY`	4×4
`TILES_PER_GROUP`	`NumTilesPerGroup`	4
`CORES_PER_TILE`	CCs/Tile, `NumCC/NumTiles`	4 CCs
`SCALAR_PER_CC`	`NumScalarPerCC`	1; optional 2
`BANKS_PER_TILE`	L1 controllers/Tile	4
Cache factory geometry	Ways, entries, line width	4 ways, 256 sets, 64 B
`MEM_LATENCY`	No direct RTL constant	50 cycles, flat backend
`DRAMSYS`, `DRAM_TYPE`	Backend selection	Off for RLC scaling; HBM2 smoke test
`PERIPH_MAP`	Software register layout	`rlc_next` for evaluated RLC ELFs
`PROBE`	Instrumentation	Enabled for measured RLC runs

This cache geometry provides 64 KiB per controller, 256 KiB per Tile, and 16 MiB over 64 Tiles. The boot image supplies the modeled core and Tile counts to the runtime. Topology exploration combines Python parameters, generated RLC role lists, and the corresponding topology/backend model build. Shared-ELF execution uses compatible ISA/ABI, register maps, cache assumptions, and barriers, as described in Section 6.4.

6.3 Memory-System Modeling

The measured 16-Group model has a 4×4 refill-router mesh and eight memory endpoints: four West and four East. Endpoints do not add router hops. The modeled narrow request plane is 8 B wide and the wide plane is 64 B wide. These widths specify interconnect transfer sizes; sustained memory service additionally depends on backend scheduling and contention.

Configurable memory service. For the scaling experiments, the flat backend has a response latency of 50 model cycles and nominal width of 64 B per cycle *per store*. The eight endpoint paths converge on two shared flat address-window stores, as shown in Figure 13. Observed miss latency additionally includes network traversal, arbitration, pending-line handling, and installation.

DRAMSys backend. The optional backend is selected with `CACHEPOOL_V3_DRAMSYS=1`. The contiguous windows `0x80000000--0xbfffffff` form a 1 GiB SoC address space striped over eight separate DRAMSys instances. For byte offset a from `0x80000000`, channel and compact address are

$$h = \lfloor a/1024 \rfloor \bmod 8, \qquad a_h = 1024 \lfloor a/8192 \rfloor + (a \bmod 1024). \tag{3}$$

Bits $[12 : 10]$ therefore select a channel at a 1 KiB granularity. Mesh and interleaver use the same mapping; an aligned 64-byte line cannot cross a stripe.

The verified configuration is `hbm2-example.json`, using the supplied HBM2 specification, pseudo-channel address mapping, and open-page FR-FCFS controller. The specification sets $t_{CK} = 1$ ns, read latency to 17 DRAM clock periods, and read activate-to-column delay to 12 periods. Row state, conflicts, refresh, scheduling, and queues determine the resulting request latency. The mapped SoC address window is 1 GiB. DDR3, DDR4, and LPDDR4 backend configurations are also supplied. Comparisons with RTL select and calibrate matching device timing, including its HBM2E or DDR4-2400 settings where applicable.

The SystemC bridge supports asynchronous striped transfers, preserves pending request data, and synchronizes model time before grants and responses. Posted writes acknowledge acceptance into the request path; physical DRAM completion follows backend scheduling. End-to-end HBM2 integration is validated by a 16-Group RLC regression. Application timing uses in-kernel boundaries to separate packet processing from loading and boot.

The evaluated GVSoC topology uses eight memory endpoints; RTL also provides a 16-channel configuration. Endpoint placement, routing, and striping define these as separate system configurations.

6.4 Software Compatibility and End-to-End Simulation

The platform models boot, program/data loading, private stack SPMs, scalar/vector accesses, synchronization, maintenance, and EOC status. Compatible non-instrumented ELFs can be shared with RTL using matching ISA/ABI, peripheral maps, and barrier semantics. The `legacy`, `rlc_next`, and `multi_scalar` maps specify the boot, EOC, and L1-control offsets; barrier configuration specifies slot encoding and release behavior.

The evaluation uses a common model-cycle time base for:

- Per-hart retirement and stall counters, PC samples, and Spatz issue, queue, dependency, execution-unit, and vector-memory state.
- Cache read/write accesses, hit/miss counts and response latencies, refills, evictions, stalls, and occupancy; crossbar and refill-mux contention.
- NoC router/interface traffic, queues, and backpressure; memory activity and per-hart barrier waits.
- Software kernel boundaries, core roles and phases, SDU arrival and final completion, PDU, grant and STATUS events, and entity queues.

Time-sliced CSVs and per-packet records feed comparative dashboards. Paced arrivals are back-dated to scheduled arrival, including producer backlog in latency. Passive PC/queue gauges support diagnosis alongside component and instruction traces; software probes provide explicit kernel and packet events, with their executed instructions included in measured time. RLC `_probe` builds use GVSoC-specific instrumentation addresses that overlap RTL control space. Unchanged-binary RTL comparisons therefore use compatible non-probe builds.

6.5 Functional Validation and RTL Compatibility

GVSoC connects application development to RTL validation through shared software interfaces. Compatible non-probe ELFs exercise the boot path, scalar/vector execution, memory accesses, and synchronization in both environments. Matching ISA/ABI, peripheral maps, and barrier semantics allows software mappings to transfer between the architectural model and the hardware implementation.

Application checks validate observable results: exit status records completion, transport-block self-checks compare headers and payloads, and scalar-reference comparisons check vectorized planning. The RLC validation in Section 8.4 combines these checks with segmentation and uplink RTL regressions.

Architectural exploration uses GVSoC's cycle-approximate resource model to relate software phases to cache, network, and synchronization activity. RTL simulation supplies implementation-level timing. Comparisons align the executable, topology, partition policy, memory assumptions, and in-kernel measurement boundaries; the abstraction choices below define the resource behavior represented in GVSoC.

6.6 Modeling Scope and Abstraction Choices

- **Miss concurrency and cache internals.** Pending lines support multiple waiters, with one outstanding refill per controller in the deployed model. RTL provides additional in-cacheline miss-resource concurrency. SRAM skew arbitration, forwarding, and macro timing are represented at an abstract level; the default per-cell coalescer is disabled. Physical implementation and PPA are evaluated through the RTL/backend flow.
- **Network and memory timing.** Narrow L1 traffic is aggregated by port class rather than reproducing every per-Tile RTL parallel mesh. Refill routing, request/response transport, arbitration, and Group I$ timing use architectural abstractions. Flat memories share backing resources; the evaluated DRAMSys mesh has eight endpoints. The 16-endpoint RTL configuration is a separate evaluation point.
- **Synchronization and partition scope.** Independent-slot Tile/Group/cluster barrier semantics require case-specific matching to the modeled subset. The evaluated partitioning uses Tile-private and globally shared banks. Group-private routing is outside this setup; software maintains ownership and visibility for noncoherent private data.
- **Shared Spatz and ISA.** Multi-scalar sharing uses gated per-hart vector models rather than one physical VRF. Drain/writeback gating, scalar-FP routing, and some vector operations differ from RTL. Software treats ownership transfer as a vector-state boundary and uses the ISA subset supported by the intended hardware configuration.

End-to-end execution and synchronized instrumentation connect software mapping decisions to core stalls, cache activity, and network contention. This provides a common framework for analyzing packet-processing phases and their architectural resource use.

7 Vector Kernel Characterization and Evaluation

7.1 Vector Micro-Kernels

To characterize the compute and memory behavior of CachePool independently of the more complex control and RLC workloads, we evaluate three representative FP32 vector micro-kernels: dot product, matrix–vector multiplication, and matrix multiplication. The kernels span different levels of data reuse and therefore exercise different regions of the memory hierarchy, from streaming bandwidth-dominated execution to compute-dominated execution with substantial reuse.

All measurements in this subsection use the 16-Group CachePool configuration at 1 GHz, with all 256 Spatz vector cores active. Each Spatz instance provides four 32-bit vector lanes, corresponding to an aggregate peak throughput of 2048 FP32 FLOP/cycle. The kernels are evaluated under both cold- and hot-cache conditions to separate the cost of initially fetching the working set from main memory from the steady-state behavior once the relevant data are resident in the distributed L1 cache.

7.1.1 Kernel Selection and Mapping

Dot product. The `fdotp-32b` kernel computes the dot product of two FP32 vectors and represents a streaming workload with very limited temporal reuse. Each input element contributes one multiplication and one accumulation, while the input vectors are consumed only once. The kernel therefore provides a simple stress case for the memory system and is expected to remain bandwidth sensitive even when many vector cores execute concurrently. For the reported measurements, the L1 cache remains globally shared across the full CachePool system.

Matrix–vector multiplication. The `fgemv-32b` kernel multiplies a dense FP32 matrix by a vector and represents an intermediate point between streaming and strongly reuse-oriented computation. The matrix is partitioned across the participating cores, while the input vector is reused by many output rows. To exploit this locality, the reported configuration uses fully Tile-private L1 banks. Each Tile therefore operates primarily on its local matrix partition while retaining a local copy of the input vector, reducing unnecessary remote accesses during the kernel execution. The measured problem uses a matrix of size 16384×64.

Matrix multiplication. The `fmatmul-32b` kernel evaluates a dense FP32 matrix multiplication and provides the most compute-intensive micro-kernel in this study. The implementation partitions the output rows across the vector cores and processes multiple rows together while streaming rows of the shared input matrix through the vector registers. This organization increases data reuse and allows multiple independent vector multiply–accumulate operations to remain in flight.

The reported measurements use $M = 1024$, $K = 128$, and $N = 128$. The kernel uses fully private L1 banks folded at Group granularity, such that the private banks of the four Tiles within each Group form one Group-local memory domain. Matrix A is naturally partitioned with the output rows, while matrix B is shared among the cores of a Group. Consequently, matrix B is fetched once into each Group-private domain rather than once per Tile. Across the 16-Group configuration, this results in replication of B across the Groups and increases the off-chip traffic relative to the idealized GEMM data footprint. This implementation-specific communication cost is included when calculating the effective arithmetic intensity used in the roofline analysis.

The three kernels therefore cover complementary execution regimes. Dot product emphasizes streaming bandwidth, GEMV combines moderate data reuse with distributed memory access, and GEMM stresses the vector arithmetic units while reusing data within Group-local working sets. The selected cache configurations are not intended to establish a universal optimal mapping

for each kernel, but rather represent the mappings used for the reported measurements and reflect the dominant sharing scope of each implementation. Their measured performance is analyzed in the roofline model in Section 9.1.1.

7.2 Control Algorithm

The second workload considered in this section is a control-oriented resource-allocation algorithm derived from the iterative and greedy uplink OFDMA allocation method proposed by Kim et al. Compared with the standalone vector micro-kernels, this workload combines matrix and vector computation with iterative scalar control, conditional updates, nonlinear score evaluation, selection, and synchronization. It therefore serves as a more representative case study for the mixed scalar–vector execution model targeted by CachePool.

7.2.1 Reference Algorithm and Benchmark Formulation

The benchmark is based on the greedy subcarrier and power-allocation procedure described in the project dataplane specification. The algorithm is executed once per transmission time interval (TTI), with a target interval of 0.5 ms. At the 1 GHz target frequency used in this work, this corresponds to a budget of 500 000 cycles per invocation. All input values may change from one TTI to the next, so the benchmark does not assume persistent reuse across consecutive invocations.

The algorithm considers M users and N subcarriers. The specified input range is $1 \leq M \leq 80$ and $1 \leq N \leq 100$. The main input matrices and vectors are $R_1 \in \mathbb{Z}^{N\times M}$, $R_2 \in \{0,1\}^{M\times N}$, $A \in \mathbb{Z}^{M\times N}$, $C \in \mathbb{Z}^{N\times N}$, and $D \in \mathbb{Z}^{N}$. The outputs are the global subcarrier-allocation vector ResAllocMap, the per-user allocation matrix UserResAllocMap, and the power-allocation matrix P.

The benchmark extends the original resource-allocation procedure with synthetic matrix stages intended to increase the fraction of vectorizable computation and the frequency of switching between scalar and vector execution. These additions are used solely as project-level workload components and are not intended to model a production resource-allocation implementation. The resulting benchmark combines the greedy control structure of the original algorithm with repeated matrix, vector, element-wise, and selection operations that stress both the compute pipeline and the memory hierarchy.

At the start of an invocation, all entries of P and the per-user water-level vector Δ are initialized to a scaled value of 100, while both allocation maps are cleared. The algorithm then iterates while unallocated subcarriers remain. In each iteration, the first stage computes

$$U = R_1 R_2 + C, \qquad R = \mathrm{clamp}(U, \mathrm{MinR}, \mathrm{MaxR}),$$

followed by

$$V = AR + D, \qquad G = \mathrm{clamp}(V \gg 7, \mathrm{MinG}, \mathrm{MaxG}).$$

The current benchmark uses

$$\mathrm{MinR} = -4096, \qquad \mathrm{MaxR} = 4096,$$

and

$$\mathrm{MinG} = 1, \qquad \mathrm{MaxG} = 2^{20}.$$

For each user and subcarrier, the score input is computed element-wise as

$$S = P \odot G,$$

and the corresponding gain is

$$\mathrm{Gain} = \ln(1 + S).$$

Among all currently unallocated subcarriers, the algorithm selects the user–subcarrier pair (w, k^*) with the largest gain. Ties are resolved deterministically by preferring the lowest subcarrier index within a user and then the lowest user index.

After selecting (w, k^*), subcarrier k^* is assigned to user w, the corresponding allocation maps are updated, and the power-allocation row $P[w, :]$ is recomputed. The selected user's row of R_2 is then conditionally modified according to the current iteration index. This update changes the gains used in subsequent iterations and introduces a sequential dependency between consecutive allocation decisions.

Once all required subcarriers have been allocated, a finalization step recomputes the power allocation for every user that owns at least one subcarrier. The complete benchmark therefore consists of an initialization phase, repeated product and selection stages, iterative power and state updates, and a final power-allocation pass.

The main benchmark configuration used in this work is $M = 64$ and $N = 128$. Although $N = 128$ exceeds the specified upper bound of 100, this configuration is intentionally used as a stress case. It increases the initial matrix-product work beyond the largest valid $M = 80$, $N = 100$ input while also providing power-of-two dimensions that distribute evenly across 1, 2, 4, 8, and 16 participating cores. This allows the implementation to be evaluated under a regular work partition while placing a slightly higher computational load on the architecture than the nominal input range.

7.2.2 Fixed-Point Gain Evaluation

The power-allocation matrix P and the per-user water-level vector Δ use fixed-point values with a scale factor of 100, with integer division applied consistently with the reference implementation. The main non-linear operation in the allocation loop is

$$\text{Gain} = \ln(1 + S), \qquad S = P \odot G.$$

Because the current Spatz implementation does not provide a native logarithm instruction, the gain is evaluated using a compact software approximation.

The input is decomposed as

$$1 + S = 2^e m, \qquad m \in [1, 2),$$

and the mantissa is centered around 1.5 using

$$u = (m - 1.5)\frac{2}{3}, \qquad |u| \leq \frac{1}{3}.$$

The logarithm of the mantissa is then approximated by the degree-three polynomial

$$\ln m \approx \ln 1.5 + u - \frac{u^2}{2} + \frac{u^3}{3}.$$

The polynomial is evaluated in fixed point, while an integer-to-floating-point conversion is used only to extract the exponent because the current Spatz ISA does not provide a count-leading-zeros instruction. Across the supported input range, the approximation error remains below 4.3×10^{-3} compared with the exact value of $\ln(1 + S)$.

The complete implementation is checked bit-exactly against the scalar reference model for P, ResAllocMap, UserResAllocMap, and the winning user–subcarrier pair of every iteration. This ensures that the subsequent parallelization and algorithmic restructuring preserve the deterministic behavior of the reference algorithm.

7.2.3 Restructured Algorithm and Kernel Chain

A direct implementation of the benchmark would recompute the matrix products for R and G in every allocation iteration. For the $M = 64$, $N = 128$ benchmark, this would require approximately 2.1 million multiply–accumulate operations per iteration and would dominate the execution time. However, the update of R_2 has a highly structured form that allows most of this repeated work to be eliminated.

We first define the unclamped intermediate matrices

$$U = R_1 R_2 + C$$

and

$$V = AR + D,$$

with

$$R = \text{clamp}(U) \qquad \text{and} \qquad G = \text{clamp}(V \gg 7).$$

After each allocation, only the row of R_2 corresponding to the winning user w can change. Moreover, the update

$$R_2[w, :] = (R_2[w, :] + j) \bmod 2$$

leaves the row unchanged for even j and flips it for odd j. An odd iteration therefore modifies R_2 only through one row, with each changed element differing by a sign $s_k \in \{-1, +1\}$.

This structure converts the update of U into the rank-1 correction

$$U \leftarrow U + R_1[:, w]\, s^T.$$

As long as the affected elements remain within the unclamped range, the change in R is identical to the change in U, allowing the update to be propagated through the second matrix product. Defining

$$AR_1 = AR_1,$$

the update becomes

$$V \leftarrow V + (AR_1[:, w])\, s^T.$$

The matrix AR_1 is therefore computed once during initialization and reused throughout the allocation loop.

When an updated element of U approaches or crosses the clamp boundary, the corresponding relationship between U and R is no longer linear. In this case, only the affected columns of V are recomputed from the updated R rather than rebuilding the complete matrix. Rows of U that can never reach the clamp boundary are identified once after initialization and excluded from these checks in subsequent iterations.

For the $M = 64$, $N = 128$ benchmark, the two original dense products alone require approximately 2.1 million multiply–accumulate operations. The additional AR_1 product is computed only once and then reused throughout the iterative phase. Afterwards, an odd iteration requires only about 33 thousand update operations, instead of repeating the complete matrix products. This restructuring changes the execution profile from repeated dense matrix multiplication to a one-time initialization followed by substantially lighter rank-1 updates and selection steps.

The resulting implementation is organized into four computational stages.

1. *Initial products.*

   The initialization stage computes U, R, V, and G from the input matrices. It also computes AR_1, which is reused by the iterative rank-1 updates.

2. *Rank-1 update.*

   After an odd iteration, the winning user's contribution is propagated through U and V using the rank-1 corrections described above. Columns whose R values are affected by clamping are selectively recomputed.

3. *Gain and winner selection.*

   The current power allocation and gain matrix are used to identify the best still-unallocated user–subcarrier pair. The implementation avoids recomputing the complete gain matrix in every iteration and instead maintains the information required for candidate selection incrementally.

4. *Power update and finalization.*

   The power-allocation row of the winning user is updated after each allocation. Once the allocation loop completes, the final power values are recomputed for all users that received at least one subcarrier.

These stages form the kernel chain used in the implementation. The full control kernel executes the initialization once and then alternates between lightweight selection, power updates, and rank-1 corrections until all required subcarriers are allocated. The resulting decomposition exposes both highly vectorizable matrix operations and latency-sensitive iterative control, making it a useful workload for evaluating the interaction between Spatz vector execution, distributed L1 placement, and inter-core synchronization.

7.2.4 CachePool Mapping and Software Optimizations

The control algorithm is mapped onto one CachePool Group using up to 16 Spatz vector cores. All working data are placed in the Group-private L1 region, and no data movement outside the Group is required during the iterative phase. This mapping matches the communication structure of the algorithm: the participating cores repeatedly access common matrices and exchange a small amount of selection state, while no other Group needs to observe the intermediate data. Consequently, the execution behavior is determined primarily by the local Group architecture and is largely independent of the total number of Groups instantiated in the surrounding CachePool system.

The choice of Group-private placement is critical for the dense product stages. With Tile-private L1 placement, the shared matrices are effectively replicated or accessed across Tile boundaries, reducing locality and increasing pressure on the local interconnect. For the $M = 64$, $N = 128$ product phase using 16 cores, Tile-private placement requires approximately 157 k cycles and reaches only 20.8% IPU utilization. Using Group-private L1 reduces the same phase to approximately 40 k cycles while increasing IPU utilization to 81.9%. This nearly fourfold reduction demonstrates that the relevant sharing scope of the benchmark is the Group rather than the individual Tile.

Beyond data placement, several software transformations are applied to reduce the serial and synchronization overhead of the iterative phase.

Packed-key selection. The current Spatz implementation does not provide the masking support required by a straightforward masked maximum reduction. The selection stage therefore packs the score and tie-breaking information into an integer key and performs element-wise maximum operations on these keys. This allows the vector datapath to perform the comparison while preserving the deterministic ordering required by the reference model. The packed representation also avoids maintaining a separate vector mask during the reduction.

Lazy gain evaluation. Evaluating $\ln(1 + S)$ for every user–subcarrier pair in every iteration would introduce unnecessary work because most entries do not change between successive allocations. Instead, each core maintains a per-column summary of the current score and recomputes the nonlinear gain only for candidate columns whose score can still compete with the current maximum. The gain is therefore evaluated lazily, while the complete winner remains bit-exact with respect to the reference implementation. This substantially reduces the amount of nonlinear computation performed inside the iterative loop.

Single-barrier iteration. A naive parallel implementation requires one synchronization point to publish the current winner and a second synchronization point after updating the allocation state. The optimized implementation removes the second barrier by allowing each core to derive the winner from the per-core keys produced during the previous iteration and to update its local allocation state directly. The full algorithm therefore requires only one Group-level barrier per iteration.

Top-two candidate reduction. The candidate-selection stage is further reduced by computing both the best and second-best column candidates with a vector slide tree. If the second-best candidate is below a conservative threshold derived from the best candidate, only the best column can contain the global winner and the exact candidate search can be skipped for all other columns. This optimization reduces both the number of exact gain evaluations and the amount of scalar candidate-processing work.

Clamp-aware incremental updates. The rank-1 update is exact while the affected values remain within the unclamped range. Rows of U that provably cannot reach the clamp limit are identified once after initialization and subsequently bypass the clamp-crossing checks. For the remaining rows, only columns affected by a possible clamp transition require recomputation. This avoids repeatedly applying conservative checks to rows whose behavior is known in advance.

Overlapped power update. The power update lies on the critical path of the core owning the winning user. The optimized implementation recomputes the required G values directly from the maintained V state and overlaps the two vector strips of the power update. This reduces the number of store-fence waits required by the owner and shortens the serial portion of each allocation iteration.

Per-core selection state. Frequently accessed selection metadata is replicated into per-core storage. Each core writes its candidate key to a separate cache-line-aligned slot and maintains local copies of the allocation map and index tables. This avoids repeated contention on a single shared cache line when all 16 participating cores perform the same selection bookkeeping at the end of each iteration.

The resulting implementation combines architectural locality with software restructuring. Group-private cache placement keeps the working set close to the participating cores, while the rank-1 update, lazy gain evaluation, packed-key reduction, reduced barrier count, and replicated per-core metadata remove much of the repeated computation and synchronization from the original formulation. The remaining execution time is therefore dominated by candidate reduction, load imbalance around the winning user, and the serial portion of the power update rather than by the initial dense matrix products.

8 RLC/MAC Dataplane Implementation

The RLC dataplane combines ordered per-UE protocol state with parallel transport-block assembly. An owner computes segmentation and output placement, while cooperating consumers construct PDUs in disjoint output ranges. Vector operations accelerate planning arithmetic and payload movement; Tile-aware placement aligns UE state with the cores that process it. A complementary uplink harness exercises header scanning, segment accounting, and ordered completion.

8.1 Target Workloads and Execution Model

The partner scenarios define nominal payload size, aggregate packet rate, and user count. The evaluated downlink arrival clock uses 1,600 DL opportunities/s, corresponding to 625 µs between DL opportunities. This differs from the nominal 500 µs radio slot; the packet-processing budget used below is 350 µs.

Table 10: Downlink scenarios and finite datasets used in GVSoC. Rates are decimal payload rates; descriptor sizes are explained below.

Scenario	UEs	Payload (B)	Gbit/s	Mpacket/s	Packets
TC1: single-user peak	1	1,350	20	1.85	9,256
TC2: multi-user peak	48	800	50	7.81	19,528
TC3: multi-user average	4,800	800	20	3.13	15,704

For payload length S, the nominal requirement is $R_{\text{pkt}} = R_{\text{bit}}/(8S)$. At 1,600 opportunities/s, the cases correspond to approximately 1,157.41, 4,882.81, and 1,953.125 packets per opportunity. TC3 contains 15,704 descriptors and uses a paced arrival rate of 3.125 Mpacket/s, covering slightly more than eight DL opportunities.[1] The three scenarios evaluate downlink assembly; the uplink harness is described separately below.

Descriptors contain UE ID, source/destination address, and length. Reported bit rates use nominal payload sizes. Descriptor lengths include a ten-byte dataset allowance, giving 1,360/810 bytes for nominal 1,350-/800-byte payloads. AM assembly adds separate 3-byte headers, or 5 bytes with a segment offset, at the default 18-bit SN. The tables therefore report nominal payload rates, with header and dataset bytes accounted for separately from these rates.

Burst runs expose descriptors without a prescribed external arrival rate; paced runs release descriptor i at $t_0 + i/R_{\text{pkt}}$. Latency starts at that scheduled instant, including delay before the producer enqueues it. At the 1 GHz model clock, 350,000 cycles represent 350 µs.

8.2 Software Architecture and Implementation

Figure 14 separates the consumer assembly phases from producer-side acknowledgment and recycling. Fully sent SDUs leave the input queue at commit and retain their storage until STATUS acknowledges their final segment. Partial SDUs remain queued for a subsequent grant. The stop check concerns completed ingress and drained assembly; storage reclamation remains a separate ACK-driven operation.

8.2.1 Cooperative AM Downlink Pipeline

The AM implementation separates ordered work from parallel execution:

[1] The generator uses 1,963 as its finite-dataset sizing constant. The target-rate formula sets the pacing interval independently.

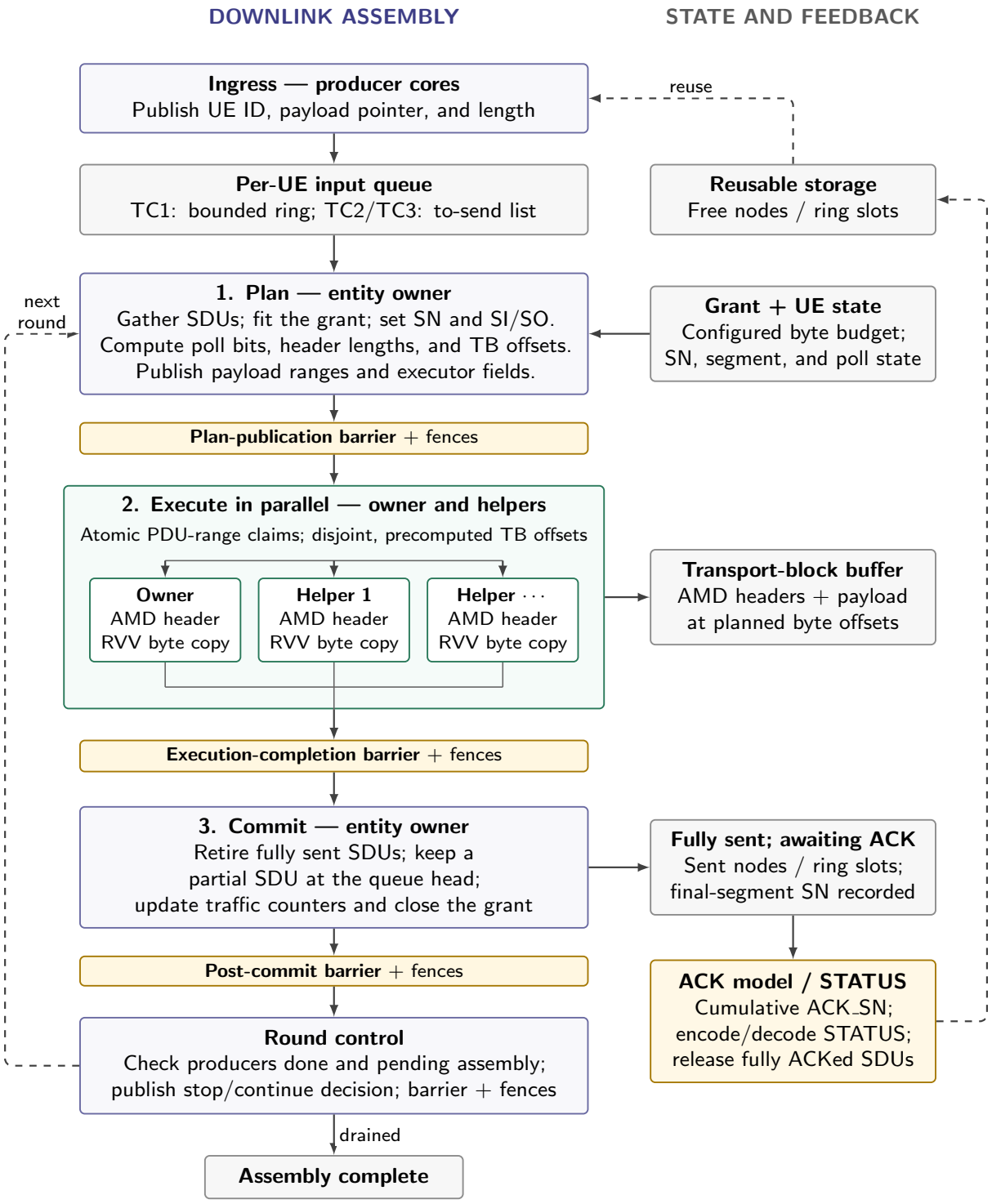


Figure 14: Cooperative AM downlink assembly with parallel header and payload construction. Each entity has one planning/commit owner; helper count is illustrative. Solid arrows show processing and data dependencies; dashed arrows show next-round and storage-reuse paths. Fenced barriers synchronize the participating consumers. Producer-side STATUS models cumulative ACKs independently of these phases. TC1 shares its ring, published plan, and TB across Tiles, with a private owner working plan. TC2/TC3 keep queues, plans, and TBs Tile-private. STATUS runs on TC1's designated producer or each worker Tile's producer in TC2/TC3.

1. **Gather and plan.** One owner gathers a bounded batch from an entity's SDU queue and computes grant fit, sequence numbers, segmentation indicators/offsets, polling decisions, and transport-block byte offsets. Plans use a structure-of-arrays layout. Vector map and prefix-sum primitives accelerate arithmetic, while scalar compare/search operations handle control decisions. A scalar planner provides a checked reference.

2. **Execute.** The owner and helper consumers atomically claim disjoint PDU ranges, writing headers and payload to predetermined offsets without a shared output cursor. Byte-element RVV copying supports the unaligned destinations created by 3-/5-byte headers. An alternative scatter–gather backend emits payload descriptors. Scaling results use the copy backend and include its payload memory traffic.

3. **Commit.** Once execution finishes, the owner retires fully sent SDUs, keeps a partial SDU at the queue head, updates traffic counters, and closes the grant. Fully sent SDUs retain their storage until acknowledged.

Producer-side STATUS processing runs separately from the consumer phases. It builds/parses cumulative ACK information, advances the acknowledged SN, and recycles nodes/ring slots. Default polling thresholds are 32 PDUs and 25,000 bytes.

Header helpers support 12- and 18-bit SNs with modular comparisons; the partner downlink cases use 18 bits. Cumulative ACK feedback controls storage recycling in the evaluated assembly path; retransmissions and protocol timers are disabled for these experiments. Each entity has one owner and can distribute execution across multiple consumers. Conversely, a consumer can own multiple entities and help others. A *helper core* is a standard Snitch/Spatz consumer core executing a published plan.

A selectable copy baseline isolates queue synchronization and data movement. It receives a PDCP descriptor, allocates a fixed-pool node, and enqueues it to a per-UE to-send list. A consumer copies the payload, updates sequence/polling state, and moves the node to a sent list; acknowledgment releases the node. The cooperative AM path additionally performs grant-dependent segmentation and PDU header construction.

8.2.2 Queues, Allocation, and Synchronization

The linked-list variant uses intrusive nodes containing links, UE/payload metadata, and destination information. Fixed-size pages provide bounded allocation/recycling without a general-purpose heap. Shared queue and allocator operations use locks; independent entities have independent queue locks. Padding separates frequently written locks, flags, entity records, and plans onto cache lines to reduce false sharing. TC1's optimized path replaces pointer-heavy ingress with a bounded ring: producers reserve entries atomically, publish filled slots, and wait for acknowledgment before reusing them.

TTI-style execution separates plan, execute, and commit with participant barriers instead of continuously polling every entity. Tile-affine mode uses Tile-local barriers; STATUS visits a dirty bitmap instead of rescanning every UE after each packet. The non-Tile-affine phase barrier uses a dedicated hardware slot, distinct from full-cluster and host barriers, with fences on both sides. The barrier synchronizes execution; fences enforce publication ordering between phases. As shown in Figure 14, plan publication and execution completion each have a fenced consumer barrier. After the post-commit barrier, a leader checks completed ingress and pending assembly; a fourth barrier publishes the stop/continue decision to the participating consumers.

8.2.3 Private/Shared Placement and Work Mapping

The large-scale runs use two private and two shared banks per Tile. With the boundary at `0xA0000000`, upper addresses select Tile-private capacity; lower addresses use the globally shared

subset. Vector payload loads/stores traverse L1 controllers. Private capacity is shared within its owner Tile, with software-managed ownership and visibility across noncoherent private copies. Per-hart stack SPMs are separate.

For TC2/TC3, UE u is assigned to logical worker index $u \bmod 48$ among the 48 selected Tiles, three per Group. Its worker Tile's producer and consumers share private UE state, queues, plans, and transport-block arena. Same numerical arena addresses may denote different private cached copies in different Tiles; Tile-affine ownership keeps each UE's working state within the corresponding private domain. These runs use Tile-private and globally shared regions.

TC1 has one entity and hence one serial planning/commit owner. Helpers span Tiles, so the queue ring, published execution plan, and transport block must be shared. The owner constructs a private working plan and publishes executor fields; the shared TB base lies below the private boundary. This mapping combines owner-local planning with shared cross-Tile execution.

Figure 15 contrasts the cross-Tile TC1 mapping with Tile-affine multi-UE execution, including the bank partition and the scope of shared execution data.

Table 11: Producer (P) and consumer (C) software-role assignments in the measured 256-core configuration.

Case	Active mapping	Consumers and placement	Unassigned
TC1	16P + 48C; Tile 0 of each Group	One owner plus 47 helpers; shared execution data	192
TC2	48P + 144C; three Tiles per Group	One producer and three consumers per Tile; 48 UEs, Tile-owned	64
TC3	48P + 144C; three Tiles per Group	Same roles; 4,800 UEs over 48 Tiles	64

Unassigned cores initialize and synchronize but do not process packets. STATUS is assigned within the producer set; TC1 dedicates one producer to STATUS and the remaining 15 to ingress in the multi-stream path. Workload-aware provisioning matches parallelism to entity ownership. TC1 concentrates ordered planning and commit in one owner, with helpers executing its plan. TC2/TC3 use 48 worker Tiles, each with one producer and three consumers. This mapping provides 1.60× and 3.40× burst-processing headroom over the TC2 and TC3 nominal packet rates, respectively, as quantified in Section 9.2.

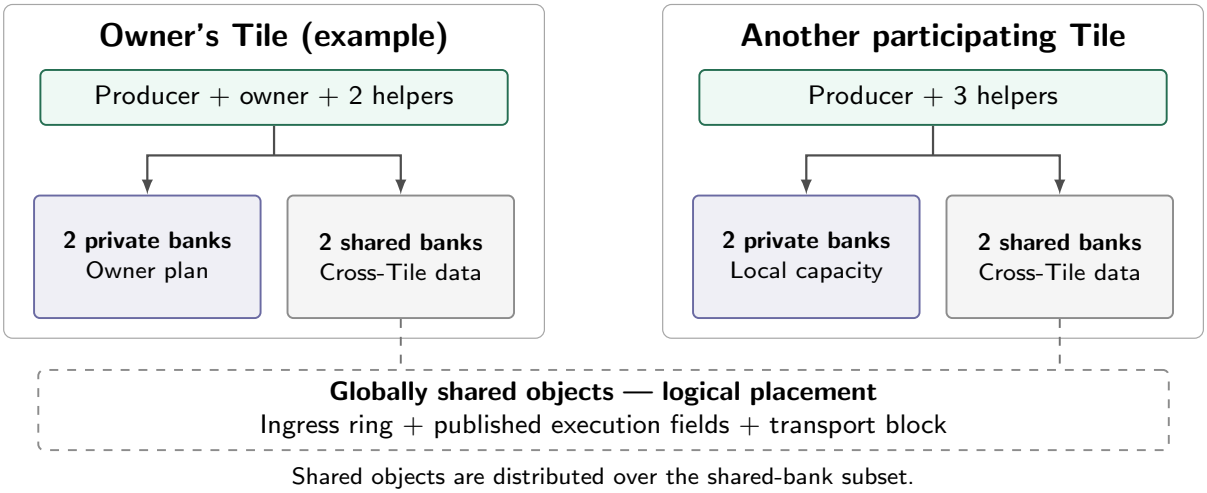
8.3 Uplink Receive Harness

The uplink path implements *scan TB → parallel segment accounting → ordered delivery and STATUS*. MAC is assumed to have assigned a TB to a UE; the harness uses a two-byte PDU-length framing field at its input. Consumer 0 scans the TB, validates header bounds, and publishes SN, SI, SO, polling, and payload ranges. All participating consumers process fixed, disjoint PDU slices and atomically accumulate received lengths. Consumer 0 then advances `rx_next_highest`, accounts complete SDUs in order, advances `rx_next`, and builds ACK-only STATUS when polling requests feedback. The three fenced barriers in Figure 16 publish the scan, await accounting completion, and share the round/stop decision. In the target harness, consumer 0 also generates the input TB stream for entity 0; scan, accounting, and ordered completion are timed as distinct receive phases.

The receive window defaults to 64 SDUs. The `RLC_UL_EXEC_COPY` backend reads and sums payload bytes to exercise receive-side memory traffic; a byte-sum sink keeps those loads observable. COUNT isolates segment bookkeeping without payload accesses. These modes separate receive-side metadata processing from payload-read traffic while sharing the same ordered-completion pipeline. The measured operation is segment accounting, with payload reads in COPY mode; destination-payload reconstruction, loss/duplicate-range recovery, retransmission, and protocol

(a) TC1: shared execution across Tiles

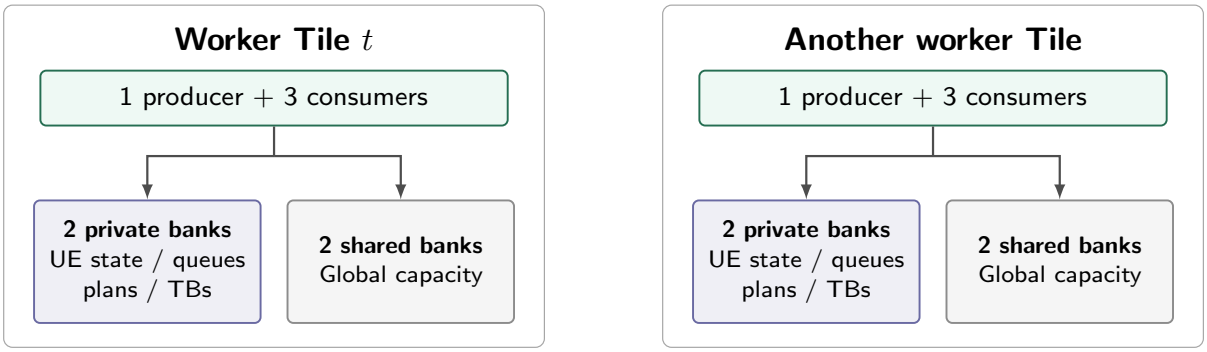
16 participating Tiles: 16 producers + 48 consumers; one entity owner.



Shared objects are distributed over the shared-bank subset.

(b) TC2/TC3: private working sets within owner Tiles

48 worker Tiles: 48 producers + 144 consumers; one producer and three consumers per Tile.



UE $u \rightarrow$ worker index $u \bmod 48$; three worker Tiles per Group. The remaining 16 Tiles are unassigned.
Both mappings use **L1-cached payload accesses**; partition boundary: `0xA0000000`.

Figure 15: Work and data placement in the 256-core RLC experiments. Each pictured Tile has four cores and four L1 banks, split equally between Tile-private and globally shared capacity. TC1 combines a private working plan with shared cross-Tile execution; its producer set includes the designated STATUS core. TC2/TC3 keep UE queues, state, plans, and TBs within owner Tiles. Worker indices identify the 48 selected Tiles, rather than a contiguous range of physical Tile IDs. Solid arrows show cached accesses; dashed links associate shared objects with shared banks, rather than an additional cache level or DMA transfers.

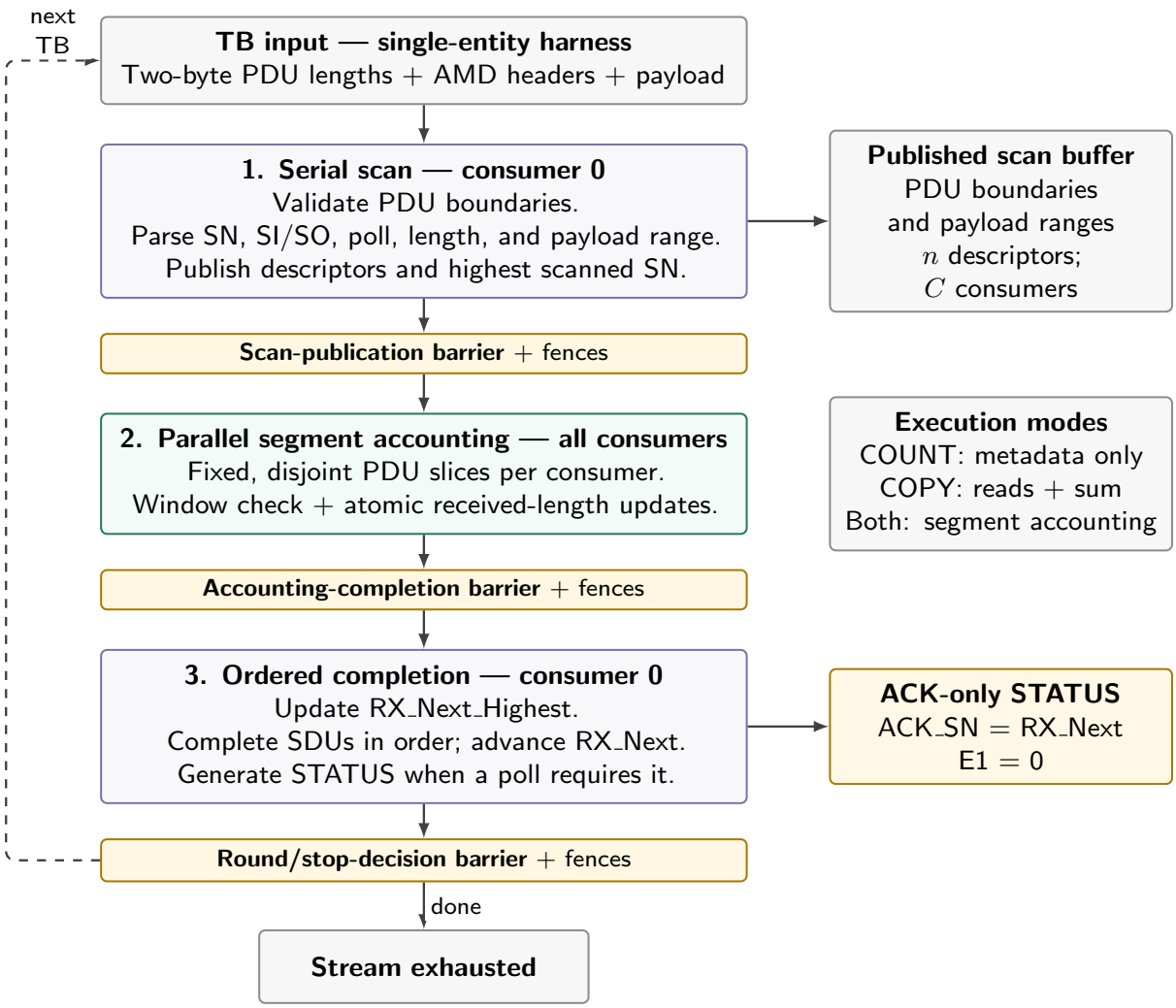


Figure 16: Uplink pipeline combining serial header scanning, parallel segment accounting, and ordered completion. Consumer 0 scans the UE-assigned TB stream and performs ordered completion. For n PDUs and C consumers, consumer i accounts slice $[is, \min((i+1)s, n))$, with $s = \lceil n/C \rceil$. Three fenced consumer barriers separate the phases and publish the stop decision. COPY generates payload-read traffic and a byte sum, while COUNT isolates bookkeeping. The serial receive phase generates ACK-only STATUS in response to polling.

timers are separate receive functions outside this harness.

8.4 Functional Validation

Host planner tests check grant-fill, segmentation, polling, and structural invariants. Target builds can compare the vectorized planner to its scalar reference. TB self-checks decode assembled headers and compare payload with the source. Uplink tests compare serial and sliced processing and check ordered delivery.

The 18-bit planner/uplink host suites passed 589,728/16,109 checks; their 12-bit variants passed 539,658/16,085 checks. These host suites validate planner and receiver logic; target self-checks exercise vector execution and assembled output. RTL regression results separately cover an AM segmentation test with six grants, zero TB mismatches, and successful EOC; the short uplink test also reports PASS and successful EOC. Self-check failures propagate to the application exit status.

Functional validation combines planner invariants, scalar-reference comparisons, transport-block self-checks, and RTL regressions. The following evaluation characterizes packet-processing performance at cluster scale. Its timing datasets track descriptor completion; bytewise payload correctness is checked separately using initialized self-test patterns.

9 Evaluation

This section evaluates the performance and scalability of the complete CachePool architecture across representative vector and dataplane workloads. The evaluation covers configurations ranging from individual Groups to the full 16-Group system and considers both the distributed L1 hierarchy and the connection to external memory. Cycle-accurate RTL simulation with QuestaSim is used for the vector micro-kernels and control-algorithm evaluation, while GVSoC is employed for larger-scale workload studies where faster simulation is required. External-memory behavior is modeled with *DRAMSys* where applicable, using the HBM-based memory configuration of the HBM2e-3600 model. The RLC evaluation in Section 9.2 measures cooperative packet assembly, throughput headroom, and paced packet latency on the 256-core system. The evaluated workloads include the FP32 vector micro-kernels introduced in Section 7.1, the control algorithm described in Section 7.2, and the RLC workload presented in the following subsection.

9.1 Vector Kernel Benchmarking

9.1.1 Roofline Analysis

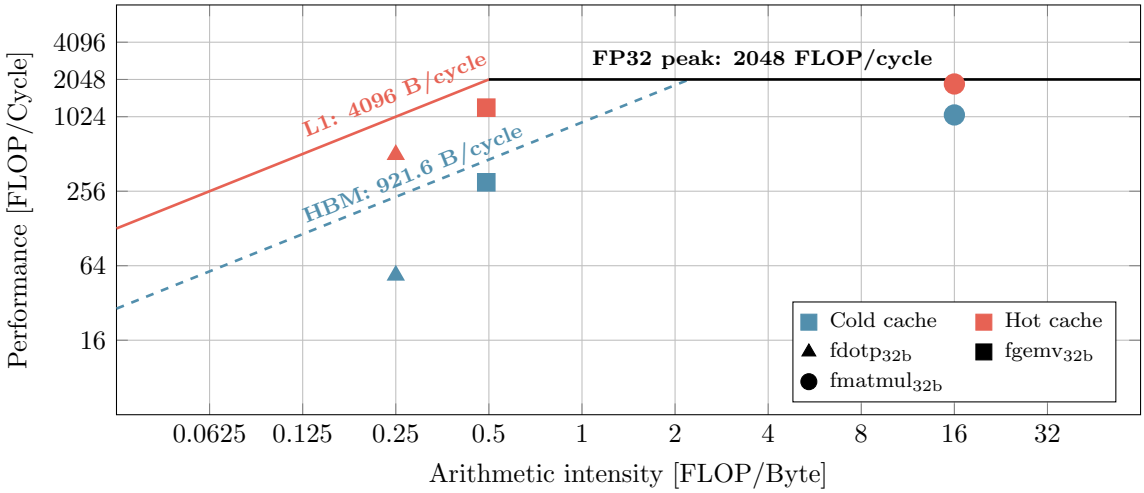


Figure 17: Roofline model of the 16-Group CachePool configuration for the FP32 vector micro-kernels. The compute roof assumes 256 Spatz vector cores, each with four 32-bit lanes, giving a peak throughput of 2048 FLOP/cycle at 1 GHz. The off-chip roof corresponds to 16 HBM2E-3600 channels (921.6 B/cycle), while the on-chip roof corresponds to the aggregate L1 bandwidth of 4096 B/cycle. Arithmetic intensity is calculated from the compulsory off-chip traffic of the implemented kernel mapping. For `fmatmul-32b`, this includes replication of matrix B once per Group under the Group-private cache configuration.

Figure 17 evaluates the three FP32 micro-kernels against the compute, distributed-L1, and off-chip-memory limits of the 16-Group configuration. The 256 Spatz instances provide an aggregate peak throughput of

$$256 \times 4 \times 2 = 2048 \text{ FP32 FLOP/cycle},$$

where a multiply–accumulate is counted as two floating-point operations.

The aggregate core-to-L1 bandwidth is

$$256 \times 4 \times 32 \text{ bit/cycle} = 4096 \text{ B/cycle}.$$

The off-chip memory system consists of 16 HBM2E-3600 channels and provides an aggregate theoretical bandwidth of 921.6 GB/s, corresponding to 921.6 B/cycle at 1 GHz. The dedicated

refill network provides 16×512 bit per cycle of aggregate interconnect bandwidth and therefore does not impose a lower bandwidth limit than the HBM interfaces. The corresponding FP32 roofline bounds are therefore

$$P_{\mathrm{L1}}(I) = \min(2048, 4096I)$$

and

$$P_{\mathrm{HBM}}(I) = \min(2048, 921.6I),$$

where I denotes arithmetic intensity in FLOP/byte. The L1 and HBM ridge points are 0.5 and approximately 2.22 FLOP/byte, respectively.

Arithmetic intensity is derived from the compulsory off-chip data movement of the actual software mapping rather than from an idealized algorithmic data footprint. For `fdotp-32b`, the two input vectors are streamed once, giving an arithmetic intensity of 0.25 FLOP/byte. For `fgemv-32b`, the matrix is distributed among the Tiles while the input vector is replicated into the Tile-private L1 regions, resulting in an effective arithmetic intensity of approximately 0.49 FLOP/byte. For `fmatmul-32b`, rows of matrix A are distributed across the participating cores, whereas matrix B is shared by the cores and fetched once into each Group-private L1 domain. Consequently, the 16-Group execution replicates B across the 16 Groups. For the measured $M = 1024$, $K = 128$, and $N = 128$ problem, this results in an effective off-chip arithmetic intensity of 16 FLOP/byte, substantially below the arithmetic intensity obtained by assuming that each matrix crosses the off-chip interface only once. The arithmetic-intensity calculation considers the application-visible compulsory transfers and does not include additional protocol-dependent cache refill or writeback traffic.

Figure 17 reports both cold- and hot-cache performance using the same arithmetic intensity for each kernel. The arithmetic intensity characterizes the mapped kernel and its compulsory off-chip data footprint, while the two measurements expose the performance difference between initially fetching that working set from main memory and executing with the data already resident in L1. This convention allows the impact of the memory hierarchy to be visualized without assigning an artificially unbounded arithmetic intensity to the hot-cache measurements.

The dot-product kernel represents the most strongly bandwidth-sensitive case. With an arithmetic intensity of 0.25 FLOP/byte, its cold-cache execution reaches only 53.52 FLOP/cycle, compared with the theoretical HBM roof of 230.4 FLOP/cycle at this arithmetic intensity. Once the operands are resident in L1, performance increases to 505.09 FLOP/cycle, corresponding to a vector-unit utilization of 24.66%. The large difference between the cold and hot measurements confirms that the kernel is dominated by data delivery rather than arithmetic throughput.

The matrix–vector multiplication kernel occupies an intermediate regime. Its Tile-private mapping yields an effective arithmetic intensity of approximately 0.49 FLOP/byte, close to the L1 ridge point. Cold-cache performance reaches 302.14 FLOP/cycle, while the hot-cache execution improves to 1211.53 FLOP/cycle and 59.16% vector-unit utilization. Compared with dot product, GEMV benefits from greater reuse of the input vector and exposes enough independent work to use the distributed L1 bandwidth more effectively. Nevertheless, the substantial remaining difference between cold and hot execution shows that external-memory latency and bandwidth still have a significant impact.

Matrix multiplication exhibits a different behavior. The kernel performs substantial data reuse after the Group-local working set has been established, despite the additional off-chip traffic required to replicate matrix B across Groups. Its effective arithmetic intensity of 16 FLOP/byte places it well inside the compute-bound region of both rooflines. The cold-cache execution reaches 1058.13 FLOP/cycle, or 51.67% of peak vector throughput, while the hot-cache case reaches 1883.07 FLOP/cycle and 91.95% utilization. The hot result is therefore close to the 2048 FLOP/cycle architectural compute roof, demonstrating that the full 16-Group system can sustain high vector utilization when sufficient data reuse is available.

The three kernels therefore expose complementary operating points of the CachePool memory hierarchy. Streaming DOTP is primarily limited by memory traffic, GEMV lies close to the

transition between bandwidth- and compute-dominated execution, and GEMM approaches the compute roof once its distributed working set is resident in the Group-private L1 regions. The measurements also illustrate the role of the programmable L1 organization: the reported configurations use a globally shared L1 for DOTP, fully Tile-private banks for GEMV, and fully Group-private banks for GEMM, matching the sharing and locality characteristics of each software mapping. These results motivate the workload-aware cache-placement mechanisms discussed in Section 5.7.

9.1.2 Control Kernel Characterization

We first characterize the main computational stages independently using the $M = 64$, $N = 128$ benchmark. The measurements use up to 16 Spatz cores within one Group and the Group-private L1 configuration described in Section 7.2.4. All results in this subsection correspond to hot-cache execution in order to focus on the compute, synchronization, and intra-Group memory behavior of the kernels.

Table 12 summarizes the main stages. The initialization product stage computes the dense intermediate matrices required before entering the iterative allocation loop. Using eight cores, this stage completes in 73 383 cycles with 89.3% IPU utilization. Increasing the core count to 16 reduces the execution time to 39 964 cycles, corresponding to a speedup of approximately 1.84×, while maintaining 81.9% IPU utilization. The high utilization confirms that the dense product phase maps efficiently onto the Spatz vector units and benefits strongly from the Group-private working set. The isolated initial-product kernel reported here computes the U/R and V/G stages. The complete control kernel additionally constructs AR_1 and the initial per-column selection state, accounting for the larger initialization cost reported in the end-to-end evaluation.

Table 12: Cycle count and IPU utilization of the main control-algorithm kernels for $M = 64$ and $N = 128$.

Kernel	Cores	Cycles	IPU utilization
Initial products	8	73 383	89.3%
Initial products	16	39 964	81.9%
Gain and selection	1	45 181	95.1%
Gain and selection	4	13 172	89.9%
Gain and selection	8	7 552	86.7%
Gain and selection	16	5 974	61.7%
Update and gain	4	22 692	74.4%
Update and gain	8	14 012	63.9%
Power update	1	533–539	–

The gain-and-selection kernel also scales well at low to moderate core counts. Execution time decreases from 46 303 cycles on one core to 13 172 cycles on four cores and 7 552 cycles on eight cores. At 16 cores, the execution time reaches 5 974 cycles, but IPU utilization drops to 61.7%. This reduction indicates that the fixed problem size no longer provides enough independent vector work to maintain near-linear scaling once the reduction and selection overhead become significant.

The update kernel shows a similar trend. Increasing the core count from four to eight reduces the execution time from 22 692 to 14 012 cycles, while IPU utilization decreases from 74.4% to 63.9%. Unlike the dense initialization phase, the update stage contains less regular work and combines vector operations with clamp checking, column repair, and selection-state maintenance. The available vector parallelism is therefore increasingly limited by control and synchronization overhead as the number of participating cores grows.

The power update is considerably smaller, requiring approximately 535 cycles for one winning user when executed by its owner core. Although this operation contributes relatively little arithmetic work, it lies directly on the dependency chain between consecutive allocation decisions. Its latency therefore has a larger effect on the end-to-end execution time than its isolated cycle count might suggest.

The measurements show a clear transition in the dominant bottleneck. The initial dense products are compute-intensive and sustain more than 80% IPU utilization even at 16 cores. In contrast, the iterative kernels become increasingly dominated by candidate reduction, control flow, synchronization, and load imbalance. As a result, further scaling of the control algorithm depends less on increasing raw vector throughput and more on reducing the serial and synchronization components of each allocation iteration.

9.1.3 Control Algorithm End-to-End Results

The end-to-end control benchmark combines the initialization products, iterative winner selection, power update, rank-1 matrix updates, synchronization, and finalization described in the previous subsections. All performance results reported in this subsection are obtained from cycle-accurate RTL simulation of the CachePool system. The target is to complete one invocation within the 0.5 ms TTI budget. At the 1 GHz operating frequency used in this work, this corresponds to 500 000 cycles.

The optimized implementation executes on up to 16 Spatz cores within one Group using Group-private L1 placement. For the $M = 64$, $N = 128$ stress configuration, the complete 128-iteration algorithm requires approximately 482 k cycles on 16 cores and 565 k cycles on eight cores. The 16-core configuration therefore meets the TTI target with approximately 18 k cycles of margin, corresponding to about 3.6% of the available execution budget. This result is obtained for the stress configuration with 128 subcarriers, which is larger than the nominal maximum of 100 subcarriers in the target specification.

Table 13: End-to-end execution time of the $M = 64$, $N = 128$ control benchmark.

Cores	Execution cycles	TTI budget	TTI ratio
8	∼565 k	500 k	1.13×
16	∼482 k	500 k	0.96×

The 16-core execution consists of approximately 58 k cycles for the initial products, followed by alternating update and non-update iterations. The measured 32-iteration run requires 3.62 k cycles for iterations containing the rank-1 update pass and 2.92 k cycles for iterations without the update pass. The corresponding 128-iteration execution therefore remains below the TTI budget even though half of the iterations require the more expensive update path. The similar per-iteration costs observed in the 16- and 32-iteration runs indicate that the iterative cost remains stable as the allocation progresses.

Several optimizations are responsible for closing the remaining gap from the earlier implementation. First, the V and P matrices are placed on different L1 cache controllers and the column mapping is staggered across cores, reducing contention during the update pass. The late U update is distributed across half rows, while allocated columns are marked directly in the per-core column-index tables to reduce candidate-selection work. Finally, idle cores speculatively compute the next power update before update-pass iterations, overlapping part of the owner-dependent critical path. Together, these changes reduce the update-pass latency to approximately 3.62 k cycles on 16 cores.

The candidate-reduction path was also improved at the microarchitectural level. The current VFU includes a parallel reduction tree for `vredmax`, reducing the latency of a reduction at vector

length 64 from approximately 276 cycles in the previous VFU implementation to 75 cycles. This removes a major bottleneck from the candidate-selection stage and allows the optimized implementation to use the vector reduction directly rather than relying on a software slide-tree workaround.

The result demonstrates the importance of combining algorithmic restructuring, data placement, software scheduling, and targeted microarchitectural support. The original formulation repeatedly recomputed dense matrix products, whereas the final implementation performs these products once and maintains the state through incremental rank-1 updates. Group-private L1 placement keeps the complete working set within one Group, while the optimized memory layout and speculative execution reduce contention and owner-side serialization. As a result, the complete control workload meets the 0.5 ms TTI using only the 16 Spatz cores of a single Group.

Further improvements remain possible even after meeting the target. Better scheduling of owner-dependent work could further reduce synchronization and load imbalance, while additional nonlinear-function support could reduce the software cost of the gain computation. More generally, the control workload highlights the benefit of combining programmable vector execution with small targeted architectural extensions for reductions, nonlinear operations, and irregular selection patterns.

9.2 RLC Kernel Benchmarking

Cooperative AM assembly provides burst-processing capacities of 12.466 and 10.620 Mpacket/s for TC2 and TC3 using 48 worker Tiles. These rates correspond to 1.60× and 3.40× the scenarios' nominal packet rates. Processing capacity, paced delivery, and arrival-to-completion latency expose how cooperative assembly handles concurrent packet traffic. Single-Tile measurements identify the cache-line choices that favor payload copying and complete data management. Figure 18 summarizes processing headroom and paced tail latency; the tables retain the exact measurements and deadline fractions. Panel (a) normalizes both burst capacity and whole-window paced delivery to each case's nominal packet rate. Panel (b) shows full-run and after-startup p99 latency against the 350 µs budget.

256-core GVSoC evaluation. The evaluation uses the asynchronous `cachepool_v3` model, 1 GHz, $16 \times 4 \times 4 = 256$ single-scalar CCs, 64-byte lines, 16 MiB aggregate L1, two private/two shared banks per Tile, and an eight-endpoint refill mesh. Memory-service parameters are specified in Section 6.3. TC1 uses the ring/private-plan cross-Tile AM variant; TC2/TC3 use Tile-affine AM, as in Table 11.

Every descriptor in the three scenario datasets completes processing. Table 10 specifies the packet populations used for the throughput and latency measurements.

For K completions, first recorded arrival a_0, and last completion $d_{\max}$, whole-window delivered rate is $K/(d_{\max} - a_0)$. The middle-window estimate is the slope between the 10th and 90th percentiles of sorted completion times. Burst results use this as a capacity indicator, separating processing from startup and drain. Under paced arrivals, the same slope characterizes middle-window delivery, including catch-up when a backlog is present. Whole-window rate measures delivery over the complete arrival-to-final-completion interval.

Table 14: Packet-completion rates, Mpacket/s. Burst capacity and paced middle-window rate use the central 80%; paced delivery includes startup.

Case	Target	Burst capacity	Paced delivered	Paced middle-window
TC1	1.851851	1.907	1.840	1.835
TC2	7.812500	12.466	7.741	9.291
TC3	3.125000	10.620	3.116	3.224

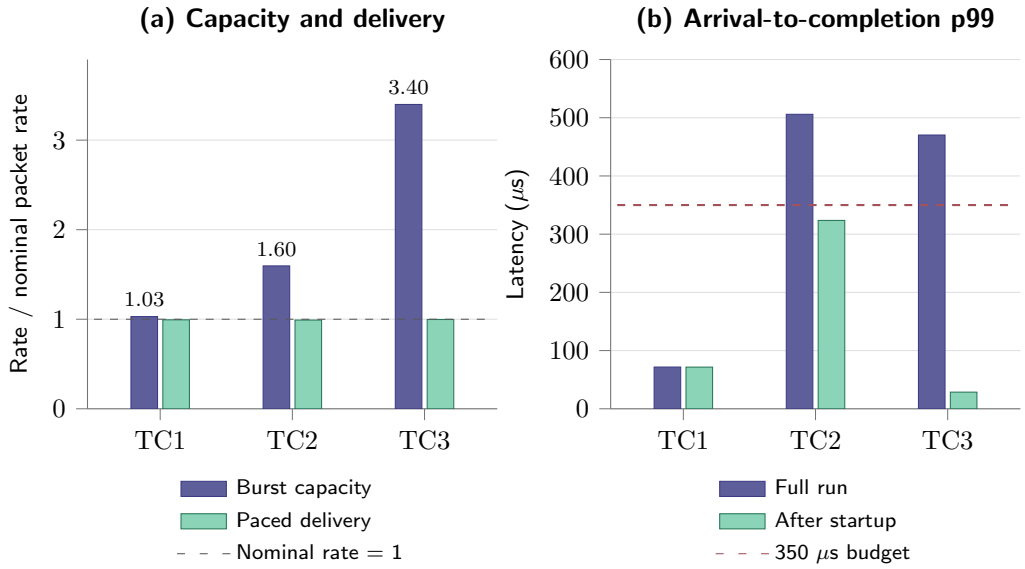


Figure 18: Cooperative RLC processing headroom and paced latency on the 256-core system. (a) Burst capacity uses the central 80% of completions; paced delivery uses the complete arrival-to-final-completion window. Each is normalized to that case's nominal packet rate; burst capacities are 1.03×, 1.60×, and 3.40× nominal. (b) Full-run p99 includes all packets; after-startup p99 selects arrivals at or after $a_0 + 625$ μs, with a_0 the first recorded arrival. Exact rates and deadline fractions are reported in Tables 14 and 15.

TC2 and TC3 provide 1.60× and 3.40× burst-capacity headroom relative to nominal packet rates. Whole-window paced delivery is 7.741 and 3.116 Mpacket/s, respectively, including startup and drain. TC1 delivers 1.840 Mpacket/s, approximately 99.4% of its nominal rate, with a burst-processing capacity of 1.907 Mpacket/s. The latency distributions in Table 15 complement these rates by accounting for packet waiting time. Throughout this evaluation, "after startup" selects arrivals at or after $a_0 + 625\,\mu$s, where a_0 is the first recorded arrival used in the whole-window rate definition.

Table 15: Paced arrival-to-final-completion latency for all packets and for arrivals at or after $a_0 + 625$ μs, with a_0 the first recorded arrival. Deadline fractions count individual packets; per-entity TTI deadlines are a distinct metric.

	All packets		After startup		
Case	p99 (μs)	$\leq 350\,\mu$s	p50 (μs)	p99 (μs)	$\leq 350\,\mu$s
TC1	71.7	100.0%	48.5	71.6	100.0%
TC2	505.9	88.3%	17.6	323.6	99.5%
TC3	470.3	96.7%	9.7	28.6	100.0%

TC1 keeps every measured packet within 350 µs, with full-run p99 latency of 71.7 µs. After startup, TC2 and TC3 deliver p99 latencies of 323.6 µs and 28.6 µs, with 99.5% and 100% of packets within budget. The full-run distributions in Table 15 include initialization and packet processing from the first arrival; role/phase profiling associates much of the startup delay with producer role discovery. Reporting both populations connects initialization behavior to paced-processing latency.

Single-Tile memory-system characterization. The single-Tile baseline separates payload copying from the full data-management path to examine how line width affects each workload.

This queue/copy baseline and the cooperative multi-UE AM mappings above exercise different software organizations. For a 1,350-byte payload at 1 GHz, the payload rate is $8 \cdot 1350 \cdot 10^9/C$ bit/s, where C is cycles per packet. All bandwidth values use decimal Gbit/s.

Table 16: Single-packet copy throughput at 1 GHz.

Line width (bit)	Cycles/packet	Payload rate (Gbit/s)
128	330	32.73
256	259	41.70
512	243	44.44

Table 17: Full data-management single-Tile baseline. Miss/stall columns give the reported averages.

Line (bit)	Cycles/packet	Gbit/s	Miss rate	Miss-stall cycles
128	1,552	6.96	59.8%	132,667
256	620	17.42	44.4%	2,448
512	1,051	10.28	43.9%	47,848

Copy throughput increases with line width, whereas the complete data-management path achieves its highest measured throughput at 256 bits. The 256- and 512-bit full-path cases have similar miss rates and distinct miss-stall costs. Line-width selection therefore depends on complete packet-processing behavior as well as payload-copy bandwidth.

Architectural implications. The RLC mappings align parallel execution with state ownership. TC1 combines a bounded ingress ring and private owner planning with shared cross-Tile execution. TC2/TC3 colocate UE state and transport-block assembly in owner Tiles, use Tile-local synchronization, and visit dirty UEs for STATUS processing. Vector payload movement traverses the L1 hierarchy in both mappings. Together, these mechanisms connect cooperative packet assembly to software-controlled private/shared placement, with the multi-UE cases providing measured burst-processing headroom on 48 worker Tiles.

10 Future Work

The current CachePool architecture establishes a scalable shared-L1 many-core platform with configurable scalar–vector organization, cache partitioning, and a hierarchical interconnect. Several directions remain open to further improve compute utilization, specialization, and communication efficiency.

Fine-Grained Scalar–Vector Sharing. The 2S1V Core Complex increases scalar-core density by allowing two Snitch cores to share one Spatz vector processor with limited area overhead. The current sharing mechanism, however, relies on explicit ownership of Spatz, while both scalar cores ultimately contend for the same vector execution resources. This can leave vector resources underutilized when the owning thread encounters a long-latency dependency and can also stall the second scalar core while waiting for access to Spatz.

A natural extension is to support multiple hardware contexts inside Spatz. The vector register file could be partitioned into independent architectural contexts, allowing instructions from two scalar threads to remain active concurrently while sharing the VLSU and vector functional units. A lightweight scheduler could switch between contexts when one thread is blocked by a long-latency operation, or arbitrate between ready contexts using a fair scheduling policy. Such fine-grained multithreading could hide memory and dependency latency, increase vector-processing-element utilization, and improve the effective IPC of the associated scalar cores.

Heterogeneous Vector–Tensor Compute. While Spatz provides flexible programmable vector execution, dense matrix multiplication and related tensor operations can benefit substantially from more specialized datapaths. A future CachePool configuration could therefore combine general-purpose vector processors with dedicated matrix or tensor engines within the same Tile. One possible design point consists of multiple Snitch scalar cores, one shared Spatz4/8 vector processor, and one tensor engine.

The main challenge in such a heterogeneous organization is efficient resource scheduling. Scalar, vector, and tensor execution should be coordinated such that specialized engines accelerate regular matrix operations without sacrificing the flexibility required by control-dominated and irregular kernels. Future work can therefore investigate lightweight scheduling support for dispatching operations to the appropriate execution engine, overlapping independent work, and balancing contention for the shared L1 memory system. Compiler and runtime support could further automate accelerator selection, data placement, and resource allocation.

Low-Latency and High-Bandwidth Interconnects. As the number of Groups increases, remote shared-L1 access latency and interconnect bandwidth become increasingly important to overall system efficiency. The topology exploration presented in Section B shows that low-diameter networks can reduce communication distance, while also exposing the physical cost of long planar interconnects and additional pipeline stages. Future work can extend this exploration toward physically optimized NoC organizations and 3D integration, where alternative placement and vertical connectivity can shorten critical communication paths and increase available bandwidth.

At a finer granularity, the current TCDM-style memory interfaces rely primarily on independent word-level transactions. Introducing burst support for regular vector accesses could combine consecutive requests and responses into larger transfers, reducing arbitration and protocol overhead. This can improve effective L1 bandwidth utilization while lowering pressure on the large number of narrow physical channels and arbitration structures. Combining burst-aware memory accesses with the existing vector load/store units and distributed L1 organization therefore

represents a complementary direction for improving both architectural performance and physical scalability.

Overall, these directions move CachePool toward a more dynamically utilized and heterogeneous architecture. Fine-grained sharing can improve the utilization of existing vector resources, tensor acceleration can increase throughput for regular compute-intensive kernels, and topology and burst-access improvements can reduce the communication overhead required to feed these engines. An important longer-term challenge is to coordinate these mechanisms through increasingly automated hardware and software scheduling rather than exposing each architectural degree of freedom directly to the programmer.

11 Conclusion

This work presents CachePool, a scalable many-core architecture built around a distributed shared L1 data cache, programmable vector processing, and a hierarchical interconnect. The final system scales to 16 Groups and supports configurable Core Complex organizations, including both 1S1V and 2S1V arrangements, while preserving a globally addressable shared-memory programming model. The architecture further exposes software-controlled cache partitioning, address scrambling, hierarchical synchronization, and configurable communication structures to adapt the memory system to different workload characteristics.

A central focus of the project has been the interaction between compute organization and the shared L1 memory hierarchy. The distributed cache provides high aggregate bandwidth while allowing software to select globally shared, Tile-private, or Group-private organizations according to the locality and sharing behavior of each kernel. The vector-kernel evaluation demonstrates the resulting range of operating points. Streaming kernels remain strongly bandwidth sensitive, while more reuse-intensive workloads progressively approach the compute roof. For dense matrix multiplication, the full 256-Spatz configuration reaches approximately 1.88 kFLOP/cycle in the hot-cache case, corresponding to about 92% of the architectural FP32 peak throughput.

The control-algorithm case study further demonstrates the importance of hardware–software co-design. A direct formulation based on repeated dense matrix products was restructured into one-time initialization followed by incremental rank-1 updates. Combined with Group-private data placement, lazy gain evaluation, reduced synchronization, and optimized candidate selection, this transformation substantially reduces the iterative workload. The resulting implementation approaches the 0.5 ms TTI target using only the 16 Spatz cores of a single Group, while retaining bit-exact behavior with respect to the reference algorithm. These results show that the architecture can support not only regular dense linear algebra, but also mixed scalar–vector control workloads with iterative dependencies and irregular selection operations.

The final physical implementation confirms that the architecture is practical at scale. The placed-and-routed 1S1V Group occupies approximately 32.3 MGE, while the 2S1V organization increases this to only approximately 33.4 MGE. The additional scalar core per Core Complex therefore introduces only about 3.5% Group-level area overhead while keeping the number of Spatz vector processors unchanged. This result demonstrates that scalar-core density can be increased efficiently by sharing the substantially larger vector datapath. The complete 16-Group system is assembled hierarchically from reusable Group implementations, providing a physically realizable path toward large-scale shared-L1 many-core integration.

The interconnect studies also highlight the importance of physical awareness when scaling such architectures. Low-diameter topologies can reduce remote-memory communication distance and improve application performance, but long physical links and the pipeline stages required to meet timing can offset part of these architectural gains. Similarly, the use of separate narrow request/response networks and wide refill paths in CachePool reflects the need to match interconnect organization to both traffic characteristics and physical implementation constraints.

Overall, CachePool demonstrates that a large shared-L1 many-core architecture can combine programmability, vector throughput, configurable locality, and physical scalability within a unified design. The project moves beyond a fixed compute or memory organization toward an architecture in which compute sharing, memory placement, and communication scope can be adapted to the workload. The resulting platform provides a strong foundation for future exploration of fine-grained vector sharing, heterogeneous vector–tensor execution, burst-oriented memory access, and more advanced low-latency interconnect organizations.

References

[1] Samuel Riedel et al. "MemPool: A Scalable Manycore Architecture with a Low-Latency Shared L1 Memory". In: *IEEE Transactions on Computers* 72.12 (2023), pp. 3561–3575. DOI: 10.1109/TC.2023.3307796. URL: https://arxiv.org/abs/2303.17742.

[2] Yichao Zhang et al. "TeraPool: A Physical Design Aware, 1024 RISC-V Cores Shared-L1-Memory Scaled-up Cluster Design with High Bandwidth Main Memory Link". In: *IEEE Transactions on Computers* (2025), pp. 1–14. DOI: 10.1109/TC.2025.3603692.

[3] Ampere Computing. *AmpereOne Family Product Brief*. May 2023. URL: https://amperecomputing.com/briefs/ampereone-family-product-brief (visited on 09/15/2025).

[4] *AMD EPYC® 9004 Series Processors: Architecture Overview*. Tech. rep. Advanced Micro Devices, Inc. (AMD), 2024. URL: https://www.amd.com/system/files/documents/epyc-9004-series-processors-architecture-overview.pdf (visited on 09/15/2025).

[5] Intel Corporation. *The Next-Generation Efficient-core Xeon Processor (Sierra Forest)*. Hot Chips 2023 presentation. 2023. URL: https://www.intel.com/content/www/us/en/events/hot-chips-2023/briefings.html?presentation=787431 (visited on 09/15/2025).

[6] NVIDIA. *NVIDIA Grace Hopper Superchip Architecture In-Depth*. Nov. 2022. URL: https://developer.nvidia.com/blog/nvidia-grace-hopper-superchip-architecture-in-depth/ (visited on 09/15/2025).

[7] Daniel Sorin, Mark Hill, and David Wood. *A primer on memory consistency and cache coherence*. Morgan & Claypool Publishers, 2011.

[8] Milo MK Martin, Mark D Hill, and David A Wood. "Token coherence: Decoupling performance and correctness". In: vol. 31. 2. ACM New York, NY, USA, 2003, pp. 182–193.

[9] Xiangyao Yu and Srinivas Devadas. "Tardis: Time traveling coherence algorithm for distributed shared memory". In: *2015 International Conference on Parallel Architecture and Compilation (PACT)*. IEEE. 2015, pp. 227–240.

[10] Xiaowei Ren et al. "Hmg: Extending cache coherence protocols across modern hierarchical multi-gpu systems". In: *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE. 2020, pp. 582–595.

[11] Johnathan Alsop, Matthew Sinclair, and Sarita Adve. "Spandex: A flexible interface for efficient heterogeneous coherence". In: *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE. 2018, pp. 261–274.

[12] Xiangyao Yu et al. "Tardis 2.0: Optimized time traveling coherence for relaxed consistency models". In: (2016), pp. 261–274.

[13] Arm Ltd. *Memory attributes*. Available at: https://developer.arm.com/documentation/100941/0101/Memory-attributes. 2023. URL: https://developer.arm.com/documentation/100941/0101/Memory-attributes.

[14] Arm Ltd. *AMBA AXI and ACE Protocol Specification*. AXI Coherency Extensions (ACE). 2013. URL: https://developer.arm.com/documentation/ihi0022/latest.

[15] Arm Ltd. *CoreLink CMN-600 Coherent Mesh Network - Technical Reference Manual*. Document Number: 100614. 2020. URL: https://developer.arm.com/documentation/100180/latest/.

[16] HSA Foundation. *HSA System Architecture Specification v1.2*. Available at: https://hsafoundation.com/wp-content/uploads/2021/02/HSA-SysArch-1.2.pdf. 2020. URL: https://hsafoundation.com/wp-content/uploads/2021/02/HSA-SysArch-1.2.pdf.

[17] Marc S Orr et al. “Synchronization using remote-scope promotion”. In: *ACM SIGARCH Computer Architecture News* 43.1 (2015), pp. 73–86.

[18] Compute Express Link Consortium. *What is CXL?* https://www.computeexpresslink.org/about-cxl. Available at: https://www.computeexpresslink.org/about-cxl. 2023.

[19] Rambus Inc. *Compute Express Link (CXL): All you need to know*. https://www.rambus.com/blogs/compute-express-link/. Available at: https://www.rambus.com/blogs/compute-express-link/. 2024.

[20] Arm Ltd. *Accelerating Cloud Workloads with CCIX*. https://www.arm.com/-/media/global/events/techcon/2017/pdf/303_armtechcon2017_acceleratingcloudworkloads.pdf. Presented at Arm TechCon 2017. 2017.

[21] Cadence Design Systems. *CCIX Verification Solution from Cadence*. https://community.cadence.com/cadence_blogs_8/b/ip/posts/ccix-verification-solution. Available at: https://community.cadence.com/cadence_blogs_8/b/ip/posts/ccix-verification-solution. 2019.

[22] Rebecca Lewington. *An AI Chip With Unprecedented Performance To Do the Unimaginable*. Aug. 2021. URL: https://www.cerebras.ai/blog/an-ai-chip-with-unprecedented-performance-to-do-the-unimaginable (visited on 09/15/2025).

[23] Yichao Zhang et al. “TeraPool-SDR: An 1.89 TOPS 1024 RV-Cores 4 MiB Shared-L1 Cluster for Next-Generation Open-Source Software-Defined Radios”. In: *Proceedings of GLSVLSI 2024*. 2024. DOI: 10.1145/3649476.3658735. URL: https://arxiv.org/abs/2405.04988.

[24] Intel Labs. *The SCC Platform Overview*. Tech. rep. Intel Corporation, May 2010. URL: https://www.intel.com/content/dam/www/public/us/en/documents/technology-briefs/intel-labs-single-chip-platform-overview-paper.pdf (visited on 09/15/2025).

[25] Kalray. *COOLIDGE™ MPPA® DPU Whitepaper*. Tech. rep. Kalray, 2022. URL: https://www.kalrayinc.com/wp-content/uploads/2023/10/WP_Kalray_MPPA_DPU_Coolidge_june2022.pdf (visited on 09/15/2025).

[26] Xiaowei Ren et al. “HMG: Extending Cache Coherence Protocols Across Modern Hierarchical Multi-GPU Systems”. In: *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 2020, pp. 582–595. DOI: 10.1109/HPCA47549.2020.00056. URL: https://ericrxw.github.io/xiaoweiren/docs/HPCA2020/HMG-HPCA2020.pdf.

[27] César Fuguet. “HPDcache: Open-source high-performance L1 data cache for RISC-V cores”. In: *Proceedings of the 20th ACM International Conference on Computing Frontiers*. 2023, pp. 377–378.

[28] Mikhail Asiatici and Paolo Ienne. “Stop crying over your cache miss rate: Handling efficiently thousands of outstanding misses in fpgas”. In: *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 2019, pp. 310–319.

[29] Yongbin Gu and Lizhong Chen. “Dynamically linked MSHRs for adaptive miss handling in GPUs”. In: *Proceedings of the 33rd International Conference on Supercomputing*. ICS ’19. Phoenix, Arizona: Association for Computing Machinery, 2019, pp. 510–521. ISBN: 9781450360791. DOI: 10.1145/3330345.3330390.

[30] Mazen Kharbutli and Yan Solihin. “Counter-based cache replacement and bypassing algorithms”. In: *IEEE Transactions on Computers* 57.4 (2008), pp. 433–447.

[31] Jorge Albericio et al. “The reuse cache: Downsizing the shared last-level cache”. In: *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture*. 2013, pp. 310–321.

[32] Konstantinos Kanellopoulos et al. “Victima: Drastically Increasing Address Translation Reach by Leveraging Underutilized Cache Resources”. In: *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 2023, pp. 1178–1195.

[33] Shaoxian Xu et al. “MiCache: An MSHR-inclusive Non-blocking Cache Design for FPGAs”. In: *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. FPGA ’24. Monterey, CA, USA: Association for Computing Machinery, 2024, pp. 22–32. ISBN: 9798400704185. DOI: 10.1145/3626202.3637571.

[34] NVIDIA. *NVIDIA H100 Tensor Core GPU Architecture*. en-us. URL: https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c. URL: https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c (visited on 09/30/2024).

[35] Samuel Riedel et al. “MemPool: A Scalable Manycore Architecture with a Low-Latency Shared L1 Memory”. In: *IEEE Transactions on Computers* 72.12 (2023), pp. 3561–3575. DOI: 10.1109/TC.2023.3307796. URL: https://arxiv.org/abs/2303.17742.

[36] Yichao Zhang et al. “TeraPool: A Physical Design Aware, 1024 RISC-V Cores Shared-L1-Memory Scaled-up Cluster Design with High Bandwidth Main Memory Link”. In: *IEEE Transactions on Computers* (2025), pp. 1–14. DOI: 10.1109/TC.2025.3603692.

[37] Taehyun Kim et al. “Scalable Bandwidth Shaping Scheme via Adaptively Managed Parallel Heaps in Manycore-Based Network Processors”. In: *ACM Trans. Des. Autom. Electron. Syst.* 22.4 (2017). ISSN: 1084-4309. DOI: 10.1145/3065926. URL: https://doi.org/10.1145/3065926.

[38] Yang Hu and Tao Li. “Enabling Efficient Network Service Function Chain Deployment on Heterogeneous Server Platform”. In: *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 2018, pp. 27–39. DOI: 10.1109/HPCA.2018.00013.

[39] Bowen Wang et al. *A Dynamic Allocation Scheme for Adaptive Shared-Memory Mapping on Kilo-core RV Clusters for Attention-Based Model Deployment*. 2025. arXiv: 2508.01180 [cs.AR]. URL: https://arxiv.org/abs/2508.01180.

[40] Scott Davidson et al. “The Celerity Open-Source 511-Core RISC-V Tiered Accelerator Fabric: Fast Architectures and Design Methodologies for Fast Chips”. In: *IEEE Micro* 38.2 (2018), pp. 30–41. DOI: 10.1109/MM.2018.022071133.

[41] Dai Cheol Jung et al. “Scalable, Programmable and Dense: The HammerBlade Open-Source RISC-V Manycore”. In: *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 2024, pp. 770–784. DOI: 10.1109/ISCA59077.2024.00061.

[42] Dai Cheol Jung and Michael Taylor. “Evaluating Ruche Networks: Physically Scalable, Cost-Effective, Bandwidth-Flexible NoCs”. In: *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. ISCA ’25. Association for Computing Machinery, 2025, pp. 1035–1048. ISBN: 9798400712616. DOI: 10.1145/3695053.3731010. URL: https://doi.org/10.1145/3695053.3731010.

[43] Jasmina Vasiljevic et al. “Compute Substrate for Software 2.0”. In: *IEEE Micro* 41.2 (2021), pp. 50–55. DOI: 10.1109/MM.2021.3061912.

[44] David R. Ditzel and the Esperanto team. “Accelerating ML Recommendation With Over 1,000 RISC-V/Tensor Processors on Esperanto’s ET-SoC-1 Chip”. In: *IEEE Micro* 42.3 (2022), pp. 31–38. DOI: 10.1109/MM.2022.3140674.

[45] Yichao Zhang et al. *TeraNoC: A Multi-Channel 32-bit Fine-Grained, Hybrid Mesh-Crossbar NoC for Efficient Scale-up of 1000+ Core Shared-L1-Memory Clusters*. 2025. arXiv: 2508.02446 [cs.DC]. URL: https://arxiv.org/abs/2508.02446.

[46] Florian Zaruba et al. “Snitch: A Tiny Pseudo Dual-Issue Processor for Area and Energy Efficient Execution of Floating-Point Intensive Workloads”. In: *IEEE Transactions on Computers* 70.11 (Nov. 2021), pp. 1845–1860. ISSN: 1557-9956. DOI: 10.1109/TC.2020.3027900.

[47] Samuel Riedel et al. “MemPool: A Scalable Manycore Architecture With a Low-Latency Shared L1 Memory”. In: *IEEE Transactions on Computers* 72.12 (2023), pp. 3561–3575. DOI: 10.1109/TC.2023.3307796.

[48] Matteo Perotti et al. “Spatz: Clustering Compact RISC-V-Based Vector Units to Maximize Computing Efficiency”. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44.7 (2025), pp. 2488–2502. DOI: 10.1109/TCAD.2025.3528349.

[49] Chi Zhang. *Pulp-Platform-Insitu-Cache*. URL: https://github.com/pulp-platform/Insitu-Cache.

[50] Lukas Steiner, Matthias Jung, Felipe S. Prado, et al. “DRAMSys4.0: An Open-Source Simulation Framework for In-depth DRAM Analyses”. In: *International Journal of Parallel Programming* 50 (April 2022 2022), pp. 217–242. DOI: 10.1007/s10766-022-00727-4. URL: https://doi.org/10.1007/s10766-022-00727-4.

A Private Write-Through L0.5 Cache Exploration

A.1 Motivation and Scope

The baseline CachePool architecture provides a logically shared L1 data cache that is physically distributed across Tiles and Groups. This organization avoids the need for a hardware cache-coherence protocol because cache lines are not replicated in independent private coherent copies. However, the physical location of a cache line still determines the interconnect distance required to access it. For workloads with strong per-core temporal locality, repeatedly accessing a remote shared-L1 bank can therefore incur unnecessary latency and interconnect traffic.

As a side exploration, we investigated the addition of a small private write-through cache between each core and the distributed shared L1. To keep the terminology of the main report unchanged, this additional level is referred to as the *L0.5 data cache*, while the existing distributed CachePool cache remains the L1.

Two approaches were explored. The first augments the private cache hierarchy with a hardware-managed directory-based coherence mechanism. The second retains the private write-through cache but removes hardware coherence and instead relies on explicit software synchronization and cache-management operations. The latter approach is the main focus of this appendix.

A.2 Hardware-Coherent Exploration

The first exploration introduced an HPDcache-based private data cache in front of the original shared CachePool data cache. The private cache operates in write-through mode, while the shared lower level remains logically shared and physically banked. A directory-based coherence mechanism was added to track private-cache ownership and sharers and to coordinate invalidations and ownership transitions.

This exploration demonstrated that a coherent private-cache hierarchy can be integrated into CachePool without fundamentally changing the surrounding scalar–vector organization. At the same time, the evaluation showed that the additional private-cache level introduces a non-negligible miss penalty, while coherence management adds further metadata accesses, arbitration, control flow, and coherence traffic. These observations motivated the exploration of a simpler alternative that retains the locality benefits of a private cache while eliminating the distributed hardware-coherence protocol.

The accompanying Master's thesis documents the complete hardware-coherent design, including the cache organization, coherence protocol, message network, functional verification, performance analysis, and physical-design results.

A.3 Software-Managed Coherence

The second exploration retains the private write-through L0.5 cache but removes hardware-managed coherence. Each private cache operates independently, while the existing distributed shared L1 remains the common lower-level memory visible to all cores.

Write-through operation simplifies software-managed consistency because stores are propagated toward the shared L1 rather than retained indefinitely as dirty private copies. However, write-through alone is insufficient to guarantee coherence. A consumer may retain a stale private copy after another core updates the same shared data, while producer writes buffered inside the private cache may not yet have reached the shared L1 when synchronization occurs.

The explored design therefore couples synchronization with explicit cache-maintenance operations. At communication boundaries, software publishes modified data from the producer and invalidates potentially stale private copies at the consumer. This avoids hardware sharer tracking, coherence directories, and invalidation networks, at the cost of making data visibility an explicit software responsibility.

A.3.1 Private L0.5 Organization

The private L0.5 is based on HPDcache and is inserted between the Snitch–Spatz memory ports and the existing CachePool L1 interconnect. Both scalar and vector memory traffic therefore pass through the private cache before reaching the distributed shared L1.

The four Spatz memory lanes are coalesced into one wider HPDcache request, while the Snitch scalar memory port is presented to HPDcache as a separate requester. The corresponding response path scatters the vector response back to the individual Spatz lanes while returning scalar responses directly to Snitch. On the downstream side, HPDcache's separate read and write memory channels are translated and merged back into the CachePool request/response protocol before entering the existing shared-cache interconnect.

The private cache is configured in write-through mode and does not maintain hardware coherence with other private caches. Consequently, ordinary cacheable data can be replicated and become stale across cores. Correctness is established only at explicit software synchronization boundaries.

Atomic memory operations are treated differently from ordinary cacheable accesses. AMOs are forwarded through the private level as uncacheable operations and are executed at the shared-cache side. This preserves a common serialization point for atomic synchronization even though ordinary data may be replicated in the private L0.5 caches.

A.3.2 Synchronization and Data Visibility

Software-managed coherence follows an acquire–release discipline around synchronization-protected shared data. On acquisition, a consuming core invalidates its private L0.5 so that subsequent reads cannot reuse stale lines and must obtain the current value from the shared hierarchy. Before releasing ownership, a producing core first waits for outstanding scalar and vector memory operations to complete and then drains the write-through buffer of its private cache.

Conceptually, a lock-protected critical section follows the sequence

```
acquire lock
lp1_inval()

    access protected shared data

snrt_fence()
lp1_wt_flush()
release lock
```

The invalidate operation provides the acquire-side visibility action by removing potentially stale private copies. The fence ensures that outstanding Snitch and Spatz memory operations complete before publication. The subsequent write-through flush drains buffered stores toward the shared L1 before the synchronization object is released.

This mechanism does not require cache maintenance for data that remain private to one core. Explicit maintenance is required only when data ownership or visibility crosses a synchronization boundary. The software overhead therefore depends on the communication structure of the workload rather than on the total number of memory accesses.

A.3.3 Runtime Cache-Management Operations

The software-managed scheme is supported by per-core Cache Management Operation (CMO) interfaces. Each core has an independent CMO injection path into its private cache so that cache maintenance does not serialize through one global control port.

The runtime exposes three principal operations:

- `lp1_wt_flush()` issues a write-through fence and drains the private-cache write buffer toward the shared L1;
- `lp1_inval()` invalidates the complete private L0.5 cache;
- `lp1_inval_line(addr)` invalidates only the cache line containing the selected address.

Each operation programs the calling core's CMO control registers, commits the request, and waits until the private cache reports completion. The register interface is replicated per core so that independent cores can perform maintenance without contending for a shared trigger.

The implementation uses the historical prefix `lp1` for these functions and registers. In this report, the same private level is referred to as L0.5 to avoid renaming the existing shared CachePool L1.

A.3.4 Kernel Integration

The required cache-management sequence depends on how shared data are exchanged between cores. Private data require no additional operations, while data communicated through barriers or locks must be explicitly published by the producer and refreshed by the consumer.

For the FDOTP reduction, each producer completes its outstanding accesses and flushes the private write-through buffer before the synchronization point. The reducing core invalidates its private cache before reading the partial results so that the reduction observes the values published to the shared L1.

For spin-lock-based workloads, `lp1_inval()` is executed after lock acquisition and `snrt_fence()` followed by `lp1_wt_flush()` is executed before lock release. The same discipline is applied to multiple independent spin locks and to the MCS queue lock. These tests are particularly useful because repeated entry into the same critical section exposes stale private copies if the invalidate operation is omitted.

FFT requires cache-maintenance operations at inter-core communication boundaries between stages, while data that remain local to one core can stay resident in its private cache without additional maintenance. GEMV similarly applies explicit producer/consumer protection when results are exchanged between cores.

In addition to application kernels, dedicated tests were added for private-cache bring-up, capacity and eviction behavior, CMO liveness, atomic operations, LR/SC behavior, and cross-core synchronization. These tests provide functional coverage of the new private-cache datapath and the software-managed visibility mechanism.

A.4 Preliminary Performance Observations

Preliminary RTL experiments indicate that the usefulness of the private L0.5 is strongly workload dependent. Kernels with substantial temporal reuse can benefit from serving repeated accesses locally, while streaming kernels can suffer from the additional private-cache miss latency.

In an initial experiment on the `cachepool_fpu_16g_tiny` configuration with a 4 KiB private cache per core, FMATMUL showed a substantial increase in FPU utilization from 13.1% to 71.5%. GEMV improved from 9.0% to 17.4%. In contrast, FDOTP degraded because its low-reuse streaming access pattern repeatedly exercises the private-cache miss path.

Detailed waveform analysis indicates that a significant part of the FDOTP degradation originates from the HPDcache miss-handling pipeline. Directory lookup, MSHR allocation, refill processing, and replay can temporarily prevent new requests from entering the cache. The additional private level is therefore particularly costly when the hit rate is low and misses dominate execution.

These results suggest that the private L0.5 should be viewed as a workload-dependent optimization rather than a universally beneficial additional cache level. The potential gain is determined by the balance between local-hit reuse, miss penalty, synchronization frequency, and the cost of explicit cache-maintenance operations.

A.5 Discussion and Future Work

The software-managed approach substantially reduces hardware complexity compared with transparent cache coherence. It removes directory state, sharer tracking, coherence-message networks, and coherence transient states from the memory hierarchy. The corresponding cost is transferred to software, which must correctly identify communication boundaries and issue the required cache-maintenance operations.

This trade-off is particularly attractive for workloads whose sharing is structured around explicit barriers or locks. In these cases, synchronization points already exist and can also serve as data-visibility boundaries. The approach is less attractive for fine-grained or unsynchronized sharing, where frequent invalidation and write-buffer draining may eliminate the benefit of the private cache.

A broader design-space exploration is required to determine when the private L0.5 provides a net benefit. Relevant parameters include cache capacity and associativity, hit latency, miss penalty, write-buffer depth, CMO granularity, data-sharing frequency, and the physical distance to the corresponding shared-L1 banks.

A natural future step is to integrate the write-through L0.5 and its software-managed CMO behavior into the GVSoC framework described in Section 6. The higher simulation speed would allow cache size, topology, locality, and synchronization frequency to be explored over larger CachePool configurations and longer application workloads. Such an exploration could identify the workload classes and system scales for which reduced remote-access traffic outweighs the private-cache miss and maintenance overheads.

B Physically-Aware 2D NoC Topology Exploration

This appendix summarizes an exploratory study of low-diameter 2D NoC topologies for latency-sensitive core-to-L1-memory communication. The study uses the TeraNoC shared-memory many-core architecture as an exploration platform and focuses on the trade-off between logical network latency and physical implementation cost. Only planar 2D implementations are considered here.

B.1 Motivation

The baseline inter-Group network uses a regular 4×4 2D mesh. Mesh connectivity provides a simple physical structure and low router radix, but the number of hops grows with the distance between source and destination Groups. For latency-sensitive accesses to a shared L1 memory, each additional router and inter-Group link directly increases the load-response latency observed by the cores. This can translate into additional RAW stalls and increase the amount of memory-level parallelism required to hide the NoC latency.

Low-diameter topologies reduce this communication distance by adding non-local links. However, the resulting logical improvement does not come for free in a physical implementation. Long links must cross intermediate physical blocks, consume additional routing tracks, and may require pipeline registers to meet the target clock frequency. Consequently, topology selection must consider both conventional network metrics and the physical cost of realizing the additional connections.

B.2 Topology Exploration

Several candidate topologies were considered for a 4×4 Group organization, including the conventional mesh, cylindrical mesh, torus, Ruche variants, and flattened butterfly. Table 18 summarizes the principal architectural and physical-connectivity metrics.

Table 18: Comparison of candidate 2D NoC topologies for a 4×4 Group organization. Router radix includes the local injection/ejection port, and the physical channel count includes feedthrough connections.

Metric	Mesh	Cylindrical	Torus	2-hop Half-Ruche	2-hop Full-Ruche	Flattened Butterfly
Diameter	6	5	4	5	4	2
Avg. hops, incl. local	2.50	2.25	2.00	2.13	1.75	1.50
Avg. hops, excl. local	2.67	2.40	2.13	2.27	1.87	1.60
Min. bisection BW [links]	4	4	8	4	12	16
Router radix	5	5	5	7	9	7
Longest link [Group pitches]	1	2	2	2	2	3
Physical channels/node	4	6	8	7	10	14

The more aggressive topologies provide lower diameter or larger bisection bandwidth, but also introduce substantially more express connections and higher router or physical-interface complexity. This is particularly problematic for TeraNoC, where a large number of narrow NoC channels are instantiated in parallel and routing resources at Group boundaries are already highly utilized.

The torus provides a useful intermediate design point. Compared with the baseline mesh, it reduces the network diameter from six to four hops and the average hop count from 2.50 to 2.00 when local traffic is included. The minimum bisection bandwidth also increases from four to eight links. At the same time, the router radix remains unchanged at five, avoiding the significantly larger switches required by the more aggressive Ruche and flattened-butterfly alternatives. For this reason, the torus was selected for further physical and performance exploration.

B.3 Planar Torus Implementation

A 4×4 torus extends the mesh by adding wraparound connections between the two boundaries of each dimension. Logically, these connections are equivalent to short additional links. Physically, however, a wraparound wire cannot follow a "rounded" path outside the chip boundary. Instead, it must traverse the planar floorplan and pass through one or more intermediate Group blocks.

To avoid routing these long links around the Groups, feedthrough paths are introduced through the hierarchical physical blocks. The physical placement of the logical Groups is additionally shuffled so that a wraparound connection crosses fewer intermediate blocks. A direct shuffle is insufficient, however, because the original router ports have fixed physical directions. After shuffling, a logical north, south, east, or west connection may no longer align with the corresponding physical boundary of the Group.

To resolve this mismatch, the router interfaces are made direction-flexible at the physical boundary. The logical directional ports can be statically remapped to the appropriate physical side of the Group, allowing straight feedthrough connections while retaining the same logical torus topology. This also allows one reusable Group implementation to be used throughout the hierarchical design rather than requiring a different physical block for each position.

B.4 Physical Cost of the 2D Torus

The additional wraparound and feedthrough connections impose a non-negligible routing cost. In the planar physical implementation, the baseline mesh occupies approximately 4.84×4.84 mm^2, whereas the corresponding 2D torus requires approximately 5.16×5.16 mm^2. The resulting die-area increase is approximately 13.7%, primarily because the additional horizontal and vertical connections require wider routing channels between the hierarchical blocks.

Wire delay is an equally important constraint. The feedthrough paths span more than 1 mm in the planar implementation. Without additional pipeline stages, these paths limit the achievable clock frequency. Introducing one pipeline cut on each long feedthrough path improves physical feasibility, but adds latency to every request and response that traverses the corresponding link. The resulting implementation therefore exposes a direct trade-off between the reduced logical diameter of the torus and the additional latency required to physically realize its long-distance links.

B.5 Impact of Interconnect Pipelining

To quantify this trade-off, the different 2D network configurations are evaluated using an FP32 matrix-multiplication workload with dimensions $512 \times 256 \times 512$. The benchmark uses fully interleaved memory placement and involves all cores, generating substantial remote L1 traffic and making execution sensitive to inter-Group NoC latency.

Figure 19 compares the baseline mesh against three torus configurations: a logical torus without additional feedthrough pipeline cuts, a torus with one pipeline cut in both the X and Y directions, and a torus with two cuts in both dimensions.

The corresponding IPC values are summarized in Table 19.

Table 19: Impact of feedthrough pipelining on the performance of the 2D torus.

Topology	IPC	Relative to mesh
2D Mesh	0.58	1.00×
2D Torus, no cut	0.68	1.17×
2D Torus, $x1y1$ cut	0.64	1.10×
2D Torus, $x2y2$ cut	0.61	1.05×

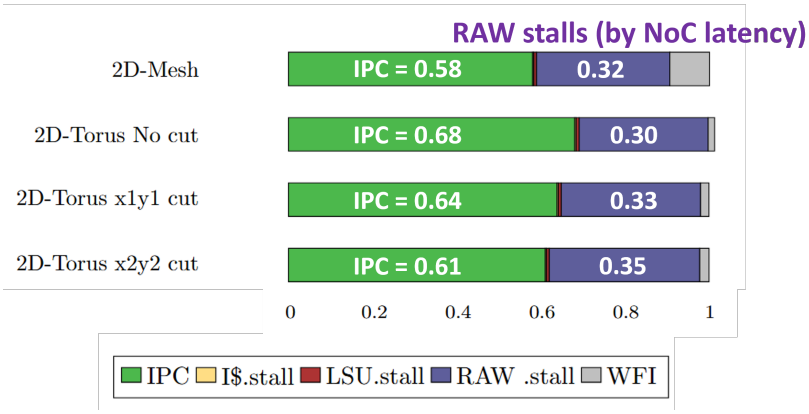


Figure 19: Execution breakdown of the FP32 matrix-multiplication benchmark for the baseline 2D mesh and 2D torus configurations with different numbers of feedthrough pipeline cuts.

Without additional pipeline cuts, the lower-diameter torus improves IPC from approximately 0.58 to 0.68, corresponding to an improvement of about 17%. The stall breakdown shows that the shorter logical communication distance reduces RAW stalls associated with load-response latency and substantially reduces synchronization-related idle time.

The benefit decreases as pipeline stages are inserted into the long feedthrough connections. With one pipeline cut in each dimension, the IPC falls to approximately 0.64. The torus nevertheless remains faster than the mesh, demonstrating that the reduction in network diameter still compensates for one additional stage of physical-link latency. With two cuts in each dimension, IPC decreases further to approximately 0.61 and approaches the mesh baseline.

These results show that logical network diameter alone is insufficient for evaluating latency-sensitive shared-memory NoCs. The no-cut torus provides the highest architectural performance, but its long planar wires are difficult to realize at high frequency. Conversely, aggressive pipelining improves the physical timing of these links but progressively removes the latency advantage that motivated the low-diameter topology. A physically effective topology must therefore minimize the number and length of pipeline cuts required to realize its non-local links.

The exploration nevertheless demonstrates that a moderate low-diameter topology can provide useful performance gains without fundamentally changing the hierarchical organization of the shared-memory system. For the studied 4×4 network, the torus offers a favorable logical compromise between hop count, bandwidth, and router complexity, while the physical results highlight interconnect distance and pipelining as the principal limitations of a planar implementation.

C CachePool Runtime API Reference

This appendix summarizes the user-facing functions provided by the CachePool runtime on the `dev/16ch` branch. The functions are grouped by purpose to complement the higher-level programming-model discussion in Section 5. Low-level register-pointer helpers and implementation-only interfaces are omitted, while low-level runtime functions that may be useful to application developers are retained for completeness.

Table 20: Summary of the user-facing CachePool runtime API. Functions are grouped by purpose; low-level register-pointer helpers and implementation-only interfaces are omitted.

Function	Scope	Description	Notes / Requirements
Topology and Identification			
`snrt_hartid()`	Per hart	Return the hardware hart identifier from `mhartid`.	Direct CSR read.
`snrt_global_core_base_hartid()`	System	Return the base hart ID of the current logical cluster.	Topology query.
`snrt_global_core_idx()`	Per core	Return the global core index.	System-wide identifier.
`snrt_global_core_num()`	System	Return the total number of cores.	Topology query.
`snrt_cluster_core_base_hartid()`	Cluster	Return the base hart ID of the current logical cluster.	Topology query.
`snrt_cluster_core_idx()`	Per core	Return the core index within the current logical cluster.	Common work-distribution index.
`snrt_cluster_core_num()`	Cluster	Return the number of cores in the current logical cluster.	Topology query.
`snrt_cluster_tile_idx()`	Per core	Return the Tile index of the executing core.	Cluster-wide Tile index.
`snrt_cluster_tile_num()`	Cluster	Return the number of Tiles in the current logical cluster.	Topology query.
`snrt_cluster_group_idx()`	Per core	Return the Group index of the executing core.	Cluster-wide Group index.
`snrt_cluster_group_num()`	Cluster	Return the number of Groups in the current logical cluster.	Topology query.
`snrt_cluster_core_per_tile()`	Cluster	Return the number of Snitch harts per Tile.	Depends on 1S1V/2S1V configuration.
`snrt_cluster_idx()`	Cluster	Return the logical-cluster index.	Relevant when multiple logical clusters are present.
`snrt_cluster_num()`	System	Return the number of logical clusters.	Topology query.
`snrt_cluster_is_primary()`	Per hart	Test whether the hart is host 0 of its shared-Spatz pair.	Always identifies the default owner in 2S1V.
`snrt_cluster_vpu_idx()`	Per VPU	Return the physical Spatz index.	Hart pairs map to one VPU in 2S1V.
`snrt_cluster_vpu_num()`	Cluster	Return the number of physical Spatz units.	Half the hart count in 2S1V.
`snrt_cluster_vpu_per_tile()`	Cluster	Return the number of Spatz units per Tile.	Configuration dependent.
`snrt_global_memory()`	System	Return the address slice describing global memory.	Returns `snrt_slice_t`.

Continued on next page

Table 20 – continued from previous page

Function	Scope	Description	Notes / Requirements
`snrt_cluster_memory()`	Cluster	Return the address slice associated with cluster-local memory metadata.	Returns `snrt_slice_t`.
Barrier Synchronization			
`snrt_cluster_hw_barrier()`	Cluster	Synchronize all cores using the hierarchical hardware barrier.	Uses slot 0 and full participation.
`snrt_cluster_sw_barrier()`	Cluster	Synchronize cores using the polling-based software barrier.	Software fallback.
`snrt_global_barrier()`	System	Synchronize across logical clusters.	Higher-level collective barrier.
`snrt_barrier(barr,n)`	Software	Execute a generic software barrier over `n` participants.	Uses a software barrier object.
`snrt_cluster_partial_barrier_mask(cids,n)`	Per core	Build a Tile-local participant mask from a list of global core IDs.	Result should be cached outside hot loops.
`snrt_cluster_partial_barrier(mask)`	Cluster	Restrict core participation within each Tile while retaining wider barrier scope.	Uses slot 0.
`snrt_cluster_group_barrier(core_mask,tile_mask, local_only,id)`	Group/System	Synchronize selected cores and Tiles on a chosen hardware-barrier slot.	All participants must use identical arguments.
`snrt_barrier_set_group_mask(id,mask)`	System	Configure the Group participation mask for a hardware-barrier slot.	Program from one core, then resynchronize before use.
`snrt_cluster_host0_barrier(id)`	Cluster	Synchronize only host 0 harts of each Core Complex.	Intended for 2S1V work sharing.
`snrt_cluster_host1_barrier(id)`	Cluster	Synchronize only host 1 harts of each Core Complex.	Intended for 2S1V work sharing.
Memory Ordering, Atomics, and Waiting			
`snrt_fence()`	Per hart	Drain outstanding Snitch and Spatz memory operations.	Use before synchronization or result readback when required.
`snrt_fence_snitch()`	Per hart	Drain outstanding Snitch scalar memory operations only.	Selective fence.
`snrt_fence_spatz()`	Per hart	Drain outstanding Spatz memory operations only.	Selective fence.

Continued on next page

Table 20 – continued from previous page

Function	Scope	Description	Notes / Requirements
`snrt_mutex_lock(pmtx)`	Shared memory	Acquire a blocking mutex using `amoswap.w.aq`.	Mutex word must be shared and initialized to zero.
`snrt_mutex_release(pmtx)`	Shared memory	Release a mutex using `amoswap.w.rl`.	Called by the lock owner.
`snrt_wfi()`	Per hart	Put the hart into wait-for-interrupt state.	Executes `wfi`.
L1 Address Mapping and Partitioning			
`l1d_xbar_config(offset)`	Cluster	Configure the L1 crossbar address-interleaving offset.	Granularity is clamped to at least one cache line.
`l1d_xbar_commit()`	Cluster/low level	Commit the programmed crossbar offset.	Normally used by the higher-level configuration path.
`l1d_part(size)`	Cluster	Set the number of Tile-private banks per Tile.	`0`=all shared; `4`=all private in the standard configuration.
`l1d_part_folded(size,fold)`	Cluster	Set the private-bank count and choose Tile- or Group-private folding.	Shared banks remain globally shared.
`l1d_addr(addr)`	Cluster	Set the shared/private address boundary.	Current runtime maps addresses below the boundary shared and addresses at/above it private.
L1 Cache Maintenance			
`l1d_cluster_flush()`	Cluster	Flush all L1 banks across the cluster.	Called by all cores; runtime fences and synchronizes internally.
`l1d_cluster_shared_flush()`	Cluster	Flush only the globally shared L1 banks.	Called by all cores.
`l1d_cluster_private_flush(tile)`	Cluster	Flush private banks selected by the Tile mask.	Called by all cores.
`l1d_flush()`	Single core	Issue a low-level full L1 flush/invalidate operation.	Primarily for runtime/internal use.

Continued on next page

Table 20 – continued from previous page

Function	Scope	Description	Notes / Requirements
`l1d_shared_flush()`	Single core	Issue a low-level shared-bank flush.	Primarily for runtime/internal use.
`l1d_private_flush(tile)`	Single core	Issue a low-level private-bank flush for selected Tiles.	Primarily for runtime/internal use.
`l1d_wait()`	Single core	Poll until the current L1 maintenance operation completes.	Used by low-level maintenance functions.
`l1d_init(size)`	Boot/runtime	Initialize the L1 cache state.	Startup code performs cache invalidation before `main()`.
`l1d_commit()`	Cluster/low level	Commit the programmed L1 cache configuration.	Runtime helper.
Spatz Ownership Control			
`spatz_lock_acquire()`	Per hart	Acquire ownership of the shared Spatz unit.	Retries in software until accepted; relevant to 2S1V.
`spatz_lock_release()`	Per hart	Release ownership of the shared Spatz unit.	Must be called by the current owner.
`spatz_lock_try_acquire()`	Per hart	Perform one immediate ownership-acquire attempt.	Returns an encoded outcome without blocking in hardware.
`spatz_lock_try_release()`	Per hart	Perform one immediate ownership-release attempt.	Returns an encoded outcome.
`spatz_lock_outcome(raw)`	Local helper	Decode a raw Spatz-lock response into `FAIL`, `SUCCESS`, or `SUCCESS_WAIT`.	Inline helper.
Dynamic Memory Allocation and Data Movement			
`snrt_alloc_init(l3off)`	Single core	Initialize the DRAM heap starting at the configured offset.	Runtime initialization.
`snrt_malloc(size)`	Single core	Allocate a cache-line-aligned payload from the DRAM heap.	Not thread-safe; payload rounded to 64 B.
`snrt_free(ptr)`	Single core	Free a DRAM allocation and coalesce adjacent free space when possible.	Not thread-safe.

Continued on next page

Table 20 – continued from previous page

Function	Scope	Description	Notes / Requirements
`snrt_memcpy(dst,src,n)`	Per core	Copy `n` bytes between memory regions.	Runtime-optimized memory copy.
Interrupt and Execution Support			
`snrt_int_init(team)`	Runtime	Initialize the interrupt subsystem for the current team.	Startup/runtime use.
`snrt_interrupt_global_enable()`	Per hart	Globally enable machine-mode interrupts.	Sets `mstatus.MIE`.
`snrt_interrupt_global_disable()`	Per hart	Globally disable machine-mode interrupts.	Clears `mstatus.MIE`.
`snrt_interrupt_enable(irq)`	Per hart	Enable an individual interrupt source.	Programs `mie`.
`snrt_interrupt_disable(irq)`	Per hart	Disable an individual interrupt source.	Programs `mie`.
`snrt_interrupt_cause()`	Per hart	Return the current interrupt cause code.	Derived from `mcause`.
`snrt_int_sw_set(hartid)`	System	Raise a software interrupt for a selected hart.	Low-level interrupt primitive.
`snrt_int_sw_clear(hartid)`	System	Clear a software interrupt for a selected hart.	Low-level interrupt primitive.
`snrt_int_sw_get(hartid)`	System	Read software-interrupt state for a selected hart.	Low-level interrupt primitive.
`snrt_int_sw_poll()`	Per hart	Poll for a software interrupt.	Low-level interrupt primitive.
`snrt_int_cluster_set(mask)`	Cluster	Raise software interrupts for selected cluster harts.	Bit-mask operation.
`snrt_int_cluster_clr(mask)`	Cluster	Clear software interrupts for selected cluster harts.	Bit-mask operation.
`snrt_int_clint_set(reg_off,mask)`	System	Write a CLINT interrupt-control mask at the selected register offset.	Low-level interrupt primitive.
`snrt_printf_float(val)`	Per hart	Print a single-precision floating-point value without promoting it to double.	Avoids unsupported compiler-generated double-precision operations.
`set_eoc(value)`	System control	Write the end-of-computation status value.	Used by runtime/test infrastructure.

D Spatz Supported Vector Instructions

The tables in this appendix summarize the vector instructions decoded by the `cachepool-32b` branch of Spatz. The decoder is used as the primary implementation-level reference. Instructions that remain only as legacy internal operation identifiers, but are not decoded by the current `spatz_decoder.sv`, are not listed as supported instructions. In particular, legacy MXU/matrix-related operation identifiers remain in `spatz_pkg.sv.tpl`, but no corresponding matrix instruction is decoded by the current decoder and they are therefore omitted here.

The `cachepool-32b` implementation has a maximum vector element width of 32 bits unless double-precision support is enabled. The decoder still recognizes 64-bit vector memory encodings, but explicitly marks them illegal when the configured maximum element width is below 64 bits. Vector floating-point instructions are available only when Spatz is configured with floating-point support. The Spatz-specific widening dot-product instructions are listed separately from standard RVV instructions.

Table 21: Integer, memory, reduction, and permutation instructions supported by the Spatz `cachepool-32b` decoder.

Class	Operation	Supported instruction forms	Condition
Vector memory operations			
Memory	Unit-stride load	`vle8.v`, `vle16.v`, `vle32.v`, `vle64.v`	64-bit width requires `MAXEW=EW_64`
	Strided load	`vlse8.v`, `vlse16.v`, `vlse32.v`, `vlse64.v`	64-bit width requires `MAXEW=EW_64`
	Unordered indexed load	`vluxei8.v`, `vluxei16.v`, `vluxei32.v`, `vluxei64.v`	64-bit index width requires `MAXEW=EW_64`
	Ordered indexed load	`vloxei8.v`, `vloxei16.v`, `vloxei32.v`, `vloxei64.v`	64-bit index width requires `MAXEW=EW_64`
	Unit-stride store	`vse8.v`, `vse16.v`, `vse32.v`, `vse64.v`	64-bit width requires `MAXEW=EW_64`
	Strided store	`vsse8.v`, `vsse16.v`, `vsse32.v`, `vsse64.v`	64-bit width requires `MAXEW=EW_64`

Class	Operation	Supported instruction forms	Condition
	Unordered indexed store	`vsuxei8.v`, `vsuxei16.v`, `vsuxei32.v`, `vsuxei64.v`	64-bit index width requires `MAXEW=EW_64`
	Ordered indexed store	`vsoxei8.v`, `vsoxei16.v`, `vsoxei32.v`, `vsoxei64.v`	64-bit index width requires `MAXEW=EW_64`
Integer arithmetic and logic			
Integer arithmetic	Add	`vadd.vv`, `vadd.vx`, `vadd.vi`	–
	Subtract	`vsub.vv`, `vsub.vx`	–
	Reverse subtract	`vrsub.vx`, `vrsub.vi`	–
Integer logic	AND	`vand.vv`, `vand.vx`, `vand.vi`	–
	OR	`vor.vv`, `vor.vx`, `vor.vi`	–
	XOR	`vxor.vv`, `vxor.vx`, `vxor.vi`	–
Carry/borrow	Add with carry	`vadc.vvm`, `vadc.vxm`, `vadc.vim`	–
	Add with carry-out	`vmadc.vv`, `vmadc.vx`, `vmadc.vi`, `vmadc.vvm`, `vmadc.vxm`, `vmadc.vim`	–
	Subtract with borrow	`vsbc.vvm`, `vsbc.vxm`	–
	Subtract with borrow-out	`vmsbc.vv`, `vmsbc.vx`, `vmsbc.vvm`, `vmsbc.vxm`	–
Shifts	Logical left	`vsll.vv`, `vsll.vx`, `vsll.vi`	–
	Logical right	`vsrl.vv`, `vsrl.vx`, `vsrl.vi`	–
	Arithmetic right	`vsra.vv`, `vsra.vx`, `vsra.vi`	–
Min/max	Signed minimum	`vmin.vv`, `vmin.vx`	–
	Unsigned minimum	`vminu.vv`, `vminu.vx`	–
	Signed maximum	`vmax.vv`, `vmax.vx`	–
	Unsigned maximum	`vmaxu.vv`, `vmaxu.vx`	–
Integer comparisons			
Comparison	Equal	`vmseq.vv`, `vmseq.vx`, `vmseq.vi`	–
	Not equal	`vmsne.vv`, `vmsne.vx`, `vmsne.vi`	–
	Unsigned less-than	`vmsltu.vv`, `vmsltu.vx`	–
	Signed less-than	`vmslt.vv`, `vmslt.vx`	–
	Unsigned less-or-equal	`vmsleu.vv`, `vmsleu.vx`, `vmsleu.vi`	–

Class	Operation	Supported instruction forms	Condition
	Signed less-or-equal	`vmsle.vv`, `vmsle.vx`, `vmsle.vi`	–
	Unsigned greater-than	`vmsgtu.vx`, `vmsgtu.vi`	–
	Signed greater-than	`vmsgt.vx`, `vmsgt.vi`	–
Integer multiply, divide, and multiply-accumulate			
Multiply	Low product	`vmul.vv`, `vmul.vx`	–
	High signed product	`vmulh.vv`, `vmulh.vx`	–
	High unsigned product	`vmulhu.vv`, `vmulhu.vx`	–
	High signed–unsigned product	`vmulhsu.vv`, `vmulhsu.vx`	–
Widening multiply	Signed	`vwmul.vv`, `vwmul.vx`	–
	Unsigned	`vwmulu.vv`, `vwmulu.vx`	–
	Signed–unsigned	`vwmulsu.vv`, `vwmulsu.vx`	–
Divide/remainder	Unsigned divide	`vdivu.vv`, `vdivu.vx`	Need div-sqrt unit
	Signed divide	`vdiv.vv`, `vdiv.vx`	Need div-sqrt unit
	Unsigned remainder	`vremu.vv`, `vremu.vx`	–
	Signed remainder	`vrem.vv`, `vrem.vx`	–
Multiply-add	Multiply-accumulate	`vmacc.vv`, `vmacc.vx`	–
	Negated multiply-subtract accumulate	`vnmsac.vv`, `vnmsac.vx`	–
	Multiply-add	`vmadd.vv`, `vmadd.vx`	–
	Negated multiply-subtract	`vnmsub.vv`, `vnmsub.vx`	–
Widening multiply-add	Signed accumulate	`vwmacc.vv`, `vwmacc.vx`	–
	Unsigned accumulate	`vwmaccu.vv`, `vwmaccu.vx`	–
	Signed–unsigned accumulate	`vwmaccsu.vv`, `vwmaccsu.vx`	–
	Unsigned–signed accumulate	`vwmaccus.vx`	–
Integer reductions			
Reduction	Sum	`vredsum.vs`	–
	AND	`vredand.vs`	–
	OR	`vredor.vs`	–
	XOR	`vredxor.vs`	–

Class	Operation	Supported instruction forms	Condition
	Signed minimum	`vredmin.vs`	–
	Unsigned minimum	`vredminu.vs`	–
	Signed maximum	`vredmax.vs`	–
	Unsigned maximum	`vredmaxu.vs`	–
Moves, merge, and slide operations			
Merge	Masked merge	`vmerge.vvm`, `vmerge.vxm`, `vmerge.vim`	–
Move	Vector/scalar/immediate move	`vmv.v.v`, `vmv.v.x`, `vmv.v.i`	–
	Scalar to vector element	`vmv.s.x`	–
	Vector element to scalar	`vmv.x.s`	–
Slide	Slide up	`vslideup.vx`, `vslideup.vi`	–
	Slide one up	`vslide1up.vx`	–
	Slide down	`vslidedown.vx`, `vslidedown.vi`	–
	Slide one down	`vslide1down.vx`	–
Vector configuration and CSR access			
Configuration	Set vector length/type	`vsetvl`, `vsetvli`, `vsetivli`	–
CSR access	Vector CSR access through standard CSR instructions	`csrrw`, `csrrs`, `csrrc`, `csrrwi`, `csrrsi`, `csrrci`	Only vector CSRs are accepted

Table 22: Floating-point and Spatz-specific vector instructions supported by the Spatz `cachepool-32b` decoder.

Class	Operation	Supported instruction forms	Condition
Basic floating-point arithmetic			
Floating-point arithmetic	Add	`vfadd.vv`, `vfadd.vf`	FPU
	Subtract	`vfsub.vv`, `vfsub.vf`	FPU
	Reverse subtract	`vfrsub.vf`	FPU
	Minimum	`vfmin.vv`, `vfmin.vf`	FPU
	Maximum	`vfmax.vv`, `vfmax.vf`	FPU
	Sign inject	`vfsgnj.vv`, `vfsgnj.vf`	FPU
	Sign inject negated	`vfsgnjn.vv`, `vfsgnjn.vf`	FPU
	Sign inject XOR	`vfsgnjx.vv`, `vfsgnjx.vf`	FPU
	Multiply	`vfmul.vv`, `vfmul.vf`	FPU
Floating-point fused multiply-add			
Fused multiply-add	Multiply-add	`vfmadd.vv`, `vfmadd.vf`	FPU
	Negated multiply-add	`vfnmadd.vv`, `vfnmadd.vf`	FPU
	Multiply-subtract	`vfmsub.vv`, `vfmsub.vf`	FPU
	Negated multiply-subtract	`vfnmsub.vv`, `vfnmsub.vf`	FPU
	Multiply-accumulate	`vfmacc.vv`, `vfmacc.vf`	FPU
	Negated multiply-accumulate	`vfnmacc.vv`, `vfnmacc.vf`	FPU
	Multiply-subtract accumulate	`vfmsac.vv`, `vfmsac.vf`	FPU
	Negated multiply-subtract accumulate	`vfnmsac.vv`, `vfnmsac.vf`	FPU
Floating-point reductions			
Reduction	Ordered sum	`vfredosum.vs`	FPU
	Unordered sum	`vfredusum.vs`	FPU
	Minimum	`vfredmin.vs`	FPU
	Maximum	`vfredmax.vs`	FPU
Floating-point conversions and moves			

Class	Operation	Supported instruction forms	Condition
Conversion	Signed integer to floating point	`vfcvt.f.x.v`	FPU
	Unsigned integer to floating point	`vfcvt.f.xu.v`	FPU
	Floating point to signed integer	`vfcvt.x.f.v`, `vfcvt.rtz.x.f.v`	FPU
	Floating point to unsigned integer	`vfcvt.xu.f.v`, `vfcvt.rtz.xu.f.v`	FPU
Narrowing conversion	Floating point to unsigned integer	`vfncvt.xu.f.w`, `vfncvt.rtz.xu.f.w`	FPU
	Floating point to signed integer	`vfncvt.x.f.w`, `vfncvt.rtz.x.f.w`	FPU
	Unsigned integer to floating point	`vfncvt.f.xu.w`	FPU
	Signed integer to floating point	`vfncvt.f.x.w`	FPU
	Floating point to narrower floating point	`vfncvt.f.f.w`	FPU
Move	Scalar FP to vector	`vfmv.v.f`, `vfmv.s.f`	FPU
	Vector FP element to scalar FP	`vfmv.f.s`	FPU
Widening floating-point operations			
Widening add	Vector/vector and wide-vector/vector	`vfwadd.vv`, `vfwadd.wv`	FPU
	Vector/scalar and wide-vector/scalar	`vfwadd.vf`, `vfwadd.wf`	FPU
Widening subtract	Vector/vector and wide-vector/vector	`vfwsub.vv`, `vfwsub.wv`	FPU
	Vector/scalar and wide-vector/scalar	`vfwsub.vf`, `vfwsub.wf`	FPU
Widening multiply	Multiply	`vfwmul.vv`, `vfwmul.vf`	FPU
Widening fused multiply-add	Multiply-accumulate	`vfwmacc.vv`, `vfwmacc.vf`	FPU

Class	Operation	Supported instruction forms	Condition
	Negated multiply-accumulate	`vfwnmacc.vv`, `vfwnmacc.vf`	FPU
	Multiply-subtract accumulate	`vfwmsac.vv`, `vfwmsac.vf`	FPU
	Negated multiply-subtract accumulate	`vfwnmsac.vv`, `vfwnmsac.vf`	FPU
Floating-point slide operations			
Slide	Slide one up with FP scalar	`vfslide1up.vf`	FPU
	Slide one down with FP scalar	`vfslide1down.vf`	FPU
Spatz-specific extension			
Dot product	Widening floating-point dot product	`vfwdotp.vv`, `vfwdotp.vf`	FPU; custom instruction